LEARNING TEXTBOOK · TECHNICAL MASTERY

PROOFPAY MASTER GUIDE

From zero blockchain knowledge to defending every design decision in a viva or placement interview

What this book is	A teaching text for the ProofPay project: blockchain escrow, an ERC-20 payment token, a Chainlink CRE workflow, a local execution bridge, a React front end, an Express shipment service and a Hardhat test suite.
Who it is for	A student who can program in JavaScript/TypeScript, has used AI coding agents on this repository, and has little formal blockchain background.
How to study it	Read Modules 1–3 in order. Then read Module 10 (the end-to-end walkthrough) before Modules 4–9, so that each technical module has a place to attach to. Finish with Modules 14–17 for interview drilling.
Verified against	Solidity 0.8.28 · Hardhat 2.29.1 · ethers 6.17.0 · OpenZeppelin 5.6.1 · React 19.3.1 · TypeScript 5.7.2 · Vite 8.3.5 · Tailwind 4.3.3 · viem 2.57.14 · Express 4.21.2 · @chainlink/cre-sdk 1.23.0 · CRE CLI 1.39.0 · Hardhat chain ID 31337
Companion document	*I_QUIT_BLOCKCHAIN.pdf* — the formal academic report. This guide is separate and is not a report; it teaches.

Every code excerpt in this book is taken from the ProofPay repository. Where a feature is incomplete or simulated, the text says so explicitly.

CONTENTS

Contents	Page
HOW TO USE THIS BOOK	05
The learning path	05
MODULE 1 UNDERSTANDING THE INTERNET AND BLOCKCHAIN FROM ZERO	06
1.1 How the internet works, and where ProofPay sits in it	06
1.2 What a blockchain is, and why it exists	07
1.3 How a blockchain transaction works	08
1.4 Ethereum and the EVM	10
1.5 Cryptographic foundations you will actually be asked about	10
MODULE 2 METAMASK, EXPLAINED COMPLETELY	12
2.1 What MetaMask actually is	12
2.2 Private keys, seed phrases, importing versus connecting	12
2.3 Accounts, networks, chain IDs and our local configuration	13
2.4 How MetaMask talks to React: EIP-1193 in the actual code	13
2.5 The real transaction lifecycle in ProofPay	14
2.6 Practical MetaMask walkthrough for this project	15
MODULE 3 SMART CONTRACTS AND SOLIDITY FROM ZERO	16
3.1 What a smart contract is, precisely	16
3.2 Solidity in one page, using ProofPay's own constructs	16
3.3 ERC-20, from first principles	17
3.4 ProofPayUSD.sol, line by line	18
3.5 ProofPayEscrow.sol, the heart of the system	19
3.6 ProofPayCREReceiver.sol, the policy boundary	22
3.7 Solidity security, using your own code as the case study	24
MODULE 4 HARDHAT AND LOCAL BLOCKCHAIN DEVELOPMENT	26
4.1 What Hardhat does, and why this project uses it	26
4.2 The local node, accounts and test ETH	26
4.3 Compilation, artifacts, ABI and bytecode	27
4.4 Deployment scripts and what they prove	27
4.5 Testing: how 31 tests are organised, and what they actually assert	28
MODULE 5 FRONTEND DEVELOPMENT AND BLOCKCHAIN INTEGRATION	30
5.1 React in the vocabulary this project uses	30
5.2 TypeScript: why the types are not decoration	30
5.3 ABI, selectors and how the browser actually calls a contract	31
5.4 Discovering escrows, and why logs are not the source of truth	31
5.5 Transaction state in the UI, and why two receipts matter	32
5.6 Error handling that users can act on	32
MODULE 6 THE BACKEND AND THE SHIPMENT API	33
6.1 Node, Express and the shape of a REST API	33
6.2 Middleware, validation and the five-layer write path	34

Contents **Page**

6.3 Registering a shipment: the real sequence 34
6.4 State transitions, and why the API is strict 34
6.5 Persistence: one file, written carefully 35
6.6 Why seller registration is separate from escrow creation 35
MODULE 7 CHAINLINK, ORACLES AND CRE FROM ZERO 36
7.1 The oracle problem, stated simply and then precisely 36
7.2 What Chainlink is, and which parts this project actually uses 36
7.3 The CRE concepts you must be able to define 37
7.4 main.ts, stage by stage, with the real code 37
MODULE 8 THE LOCAL EXECUTION BRIDGE 41
8.1 Why a bridge exists at all 41
8.2 The seventeen steps, and where each lives in the code 41
8.3 Preflight: the part that saves you from yourself 42
8.4 Validation: the bridge repeats the contract's checks, in TypeScript 43
8.5 Submission without a private key 43
8.6 Idempotency, retries and state 43
8.7 What would change for a real DON deployment 44
MODULE 9 THE AUTOMATIC COURIER SCHEDULER 45
9.1 Configuration and timeline 45
9.2 Status progression and the LOST convention 45
9.3 One tick, in order 45
9.4 The MacBook demonstration, step by step 46
9.5 What changes for a LOST shipment 46
MODULE 10 THE COMPLETE WALKTHROUGH: 3,499 pUSD FOR A MACBOOK 47
Stage 1 — Purchase 47
Stage 2 — Seller registration 47
Stage 3 — Courier progress 47
Stage 4 — CRE evaluation 47
Stage 5 — Local relay 48
Stage 6 — Settlement 48
MODULE 11 SECURITY AND DESIGN DECISIONS, DEFENDED 49
11.1 Threat model first 49
11.2 Thirteen decisions, each defended 49
11.3 The limitations to volunteer, not hide 52
MODULE 12 DEBUGGING AND TROUBLESHOOTING HANDBOOK 53
12.1 How to read an error 53
12.2 Wallet, network and gas 53
12.3 Token and escrow problems 53
12.4 Receiver and oracle problems 54
12.5 Shipment API problems 54
12.6 CRE and bridge problems 55

Contents **Page**

12.7 "I restarted Hardhat and everything vanished" 55
MODULE 13 YOUR PROJECT, FILE BY FILE 57
13.1 The repository at a glance 57
13.2 ProofPayUSD.sol 57
13.3 ProofPayEscrow.sol 58
13.4 ProofPayCREReceiver.sol 58
13.5 chainlink/cre/shipment-workflow/main.ts 58
13.6 chainlink/cre/shipment-workflow/local-runner.ts 59
13.7 chainlink/cre/shipment-workflow/courier-scheduler.ts 59
13.8 The shipment service (backend) 60
13.9 The checkout page 60
13.10 The oracle panel (admin) 60
13.11 lib/receiver.ts 61
MODULE 14 · PART A INTERVIEW AND VIVA BANK: LEVELS 1–3 62
14.1 Level 1 — foundations 62
14.2 Level 2 — mechanics 64
14.3 Level 3 — your architecture 67
MODULE 14 · PART B INTERVIEW AND VIVA BANK: LEVELS 4–5 70
14.4 Level 4 — advanced technical 70
14.5 Level 5 — the challenging questions 73
14.6 Rapid fire 75
MODULE 15 OWNERSHIP, RESUME AND DEFENCE 78
15.1 Resume entry 78
15.2 The 30-second introduction 78
15.3 The two-minute version 78
15.4 The five-minute architecture answer 78
15.5 Defending each component 79
15.6 Explaining your contribution honestly 79
MODULE 16 HANDS-ON EXERCISES AND MOCK INTERVIEW 80
16.1 The exercises 80
16.2 Answers and expected reasoning 80
16.3 Final mock interview: 25 questions 82
MODULE 17 GLOSSARY 83
17.1 How to keep learning after this book 86

HOW TO USE THIS BOOK

This book has one job: after reading it, you should be able to open a blank page and explain ProofPay from the payment button to the settlement transaction, justify each design choice, name the alternatives you rejected, and admit the system's limits without being caught out. That is a different goal from writing a report. A report proves work was done; a textbook builds understanding.

Three habits make the material stick. First, read with the repository open. Every module names real files, and reading the actual source for two minutes after each section is worth more than re-reading the prose. Second, after each module, close the book and explain the module's main idea out loud in sixty seconds. If you cannot, you have read words rather than learned a mechanism. Third, do Module 16's exercises honestly before looking at the answer section; the exercises are designed to expose exactly the gaps that interviewers find.

The learning path

The modules are ordered so that each one only depends on what came before. Modules 1 and 2 build the world: how the web talks, what a blockchain is, what a transaction does, and what MetaMask actually does when it pops up a window. Module 3 is the heart of the project: Solidity, ERC-20, and the three contracts read line by line. Module 4 explains the tool that compiles, deploys and tests everything. Module 5 covers the browser application; Module 6 covers the off-chain shipment service. Module 7 is Chainlink and the oracle problem, the intellectual centre of the project. Module 8 explains the part that is unique to this codebase, the local execution bridge, and why it exists. Module 9 covers the courier scheduler that makes the demonstration self-driving. Module 10 then replays the entire system as one continuous story, which is the version you will actually tell in an interview. Modules 11 and 12 turn the design into arguments and the failures into a troubleshooting manual. Module 13 is a file-by-file reference. Module 14 is 125 interview questions with full answers. Module 15 teaches you to describe the work on a resume and defend it. Module 16 is practice. Module 17 is a glossary of every term the project uses.

> **THE HONESTY RULE**
>
> Throughout this book, "implemented" means the code exists in this repository and was executed. "Simulated" means the mechanism is real but the data source is fake: the courier is a script, not DHL. "Not implemented" means exactly that. You will be asked about these distinctions, and the correct answer is always the precise one. Claiming a deployed Chainlink DON would be both false and easy to disprove, because the CRE configuration in this project has `dryRun: true` and the installed SDK has no selector for a local Hardhat chain.

MODULE 1

UNDERSTANDING THE INTERNET AND BLOCKCHAIN FROM ZERO

You cannot understand escrow until you understand what it means for a program to hold money, and you cannot understand that until you understand where programs live and who can change their data. This module builds those foundations in the order a newcomer needs them: the web first, then the ledger, then the transaction, then the machine that runs contracts, then the cryptography that makes any of it trustworthy.

1.1 How the internet works, and where ProofPay sits in it

Almost every interaction in ProofPay is one of two shapes: a request to a server, or a transaction to a blockchain. Both are messages; the difference is who is allowed to change the record afterwards.

A **client** is a program that asks for something: your browser, or inside it, the React application. A **server** is a program that answers: here, the Express service on port 5001, or the Hardhat node on port 8545. Between them travel **requests** and **responses**. In the browser-to-backend case the protocol is HTTP; in the browser-to-blockchain case it is JSON-RPC, which is HTTP with a fixed message format for chain operations.

Frontend means the code running inside your browser tab: React components, TypeScript, CSS, and an injected wallet provider. **Backend** means code running in a separate process that the browser cannot inspect or edit: the shipment API with its JSON file. The split matters because a frontend is under the user's control. Anything the frontend decides can be changed by a determined user with developer tools. That is exactly why ProofPay's rule, "the buyer's funds may only move when the delivery condition is satisfied", is enforced in Solidity and not in React.

An **API** (application programming interface) is an agreed set of addresses and message shapes for talking to a program. ProofPay's shipment API, for example, defines five endpoints. A **database** is durable storage. Big systems use PostgreSQL or MongoDB; this project deliberately uses a single JSON file at `backend/data/shipments.json`, written atomically, because the teaching goal is the oracle pipeline rather than database engineering.

JSON (JavaScript Object Notation) is the text format both sides agree on. A real shipment record from this project looks like this:

```
{
  "trackingId": "PPD-DEMO-DELIVERED",
  "orderId": "0xf694b593b682ac612e6f5c784a6801deab4f4c0b7420272b555d8ce669c26434",
  "carrier": "ProofPay Auto Courier",
  "sellerAddress": "0x3c44cdddb6a900fa2b585dd299e03d12fa4293bc",
  "status": "DELIVERED",
  "simulated": true,
  "events": [ { "status": "CREATED", "timestamp": "2026-11-02T00:54:12.201Z",
                "description": "Shipment registered with ProofPay Auto Courier (simulated courier
data)." } ]
}
```

Authentication answers "who is allowed to do this?". ProofPay uses three different answers in three places, and confusing them is a classic interview mistake. The shipment API uses a shared secret header on mutating routes (`requireMutationAccess`), because only the seller client and the demo scheduler should write. The blockchain uses cryptographic signatures: a transaction is only valid if it is signed by the private key behind the sending address. MetaMask is where that signature happens. And the contracts use role checks (`onlyOwner`, `onlyOracle`, `onlyAuthorized`), which are authorisation rules layered on top of the signature.

Table 1.1: What happens when a buyer opens the Buyer Dashboard

Step	Who acts	What actually happens
1	Browser	Vite's dev server at `http://localhost:5173` returns `index.html`, the bundled JavaScript and CSS. React mounts the app and React Router selects the buyer route.
2	React + MetaMask	`useWallet` silently calls `eth_accounts` (no popup) to see whether the site is already authorised, then `eth_chainId` and `eth_getBalance` to fill the header.
3	React + RPC	`useEscrows` calls `eth_getLogs` filtered on the escrow address and the `EscrowCreated` topic to discover order IDs that exist.
4	React + RPC	For each discovered ID it calls `getEscrow(orderId)` with `eth_call` and decodes the returned struct: buyer, seller, amount, timestamps, status.
5	React	Filters to escrows where the buyer equals the connected address, formats amounts from 18-decimal integers, and renders cards and an activity feed.
6	React (optional)	For the tracking panel it calls the Express API, e.g. `GET http://localhost:5001/api/shipments/order/<orderId>`, which returns the JSON record shown above.

Notice what is *absent* from that list: no server tells the browser what the buyer owns. The browser asks the chain. That single decision — the contract is the source of truth, the backend is only a courier record — is the backbone of the whole architecture, and it is the answer to "how do you know the dashboard is showing real data?".

1.2 What a blockchain is, and why it exists

Start with the problem rather than the definition. Suppose two strangers want to exchange money for goods over the internet. Somebody has to go first. If the buyer pays first, the seller may vanish. If the seller ships first, the buyer may never pay. The usual answer is an institution in the middle: a bank, a marketplace, an escrow agent. That works, but it means the rule that decides who wins a dispute lives in a private database, applied by staff, visible to nobody.

A blockchain is a **distributed ledger**: a database replicated across many independent computers (**nodes**), where every change is a signed transaction, changes are grouped into **blocks**, and blocks are chained together by cryptographic hashes so that altering an old block invalidates every block after it. Because many nodes hold copies and compare them, there is no single machine whose edit becomes truth. The interesting part is not "many copies"; it is that the copies agree on rules for what counts as a valid change, and that agreement (**consensus**) is what lets strangers rely on the record without trusting each other.

Table 1.2: Traditional database versus blockchain, with ProofPay examples

Concept	Traditional database	Blockchain, in ProofPay
Who writes	Any application with credentials; the operator can edit any row.	Only a signed transaction can change state. No account, not even the owner, can rewrite an escrow's history.
Authority	The operator decides what the rules mean.	The deployed bytecode decides. `confirmDelivery` either satisfies its checks or reverts.
History	Audit logs are optional and mutable.	Every settlement emits events permanently; the receipt, block number and logs are the audit trail.
Failure	Restore a backup; the operator's judgement applies.	Transactions are atomic: they either fully apply or leave no state change.
Cost	Cheap writes, fast.	Every write costs gas, which is why ProofPay stores only what matters on chain.
Privacy	Data is private by default.	Everything on chain is public, which is why the report stores `keccak256(trackingId)` rather than the tracking number.

Hashing is the glue. A hash function takes any input and returns a fixed-length fingerprint: change one character and the fingerprint changes completely, but the same input always gives the same fingerprint. ProofPay uses **Keccak-256**, which returns 32 bytes. You will meet it in three places: block linkage (not your code), order IDs (32 random bytes, treated as an identifier), and report IDs (`keccak256` of the order, outcome, tracking hash, timestamp and metadata). The report ID is what makes replay detectable: the same report hashes to the same 32 bytes, so the receiver can recognise it.

Digital signatures are the other half. A private key signs; anyone holding the matching public key can verify the signature but cannot produce a new one. An Ethereum **address** is derived from the public key, so "verify the signature, recover the signer's address, compare it with the sender field" is enough to prove who authorised a transaction. This is why nobody can move your pUSD by editing a request in the browser: without your key, there is no valid signature, and the node rejects the transaction before any contract runs.

> **ANALOGY, THEN THE REALITY**
>
> Think of a blockchain as a shared notebook kept by a hundred people, where each page's first line is a fingerprint of the previous page. Tearing out a page changes every later fingerprint, so the forgery is visible to everyone holding a copy.
>
> Technically: each block header commits to its parent's hash, so rewriting history requires redoing all subsequent proof-of-work or consensus participation, on the majority of the network, faster than honest nodes extend the chain. On the local Hardhat chain, this security property is *absent by design* — one machine mines instantly and you can restart it and erase everything. That is why this project never claims mainnet security.

Immutability, therefore, is not magic: it is the cost of rewriting history. **Transparency** is the consequence of replication: every participant can read every transaction. **Decentralisation** is the degree to which no single party can unilaterally decide what happens next. ProofPay gets strong immutability and transparency from the chain, but its oracle path is only partially decentralised, and the honest answer to "is this decentralised?" is discussed in Module 11 and rehearsed in Module 14.

1.3 How a blockchain transaction works

Here is the full path of one ProofPay action — a buyer funding a 3,499 pUSD escrow. Read it once as a story, then again as a checklist, because interviewers ask you to narrate exactly this.

1. **The user initiates an action.** The buyer clicks "Fund escrow". React has already validated the form and knows the amount, the seller address and a freshly generated 32-byte order ID.
2. **The wallet prepares a transaction.** The app calls `eth_sendTransaction` through the injected provider. MetaMask builds an unsigned transaction: from, to (the escrow contract), value (0, because pUSD is an ERC-20 not ETH), data (the encoded `createEscrow` call), gas parameters and the next nonce.
3. **The transaction is signed.** MetaMask shows the details and asks for confirmation. On approval it signs the transaction hash with the buyer's private key. The key never leaves the extension.
4. **The transaction is broadcast.** The signed bytes go to the RPC endpoint — here `http://127.0.0.1:8545`, the Hardhat node.
5. **A node validates it.** The node checks the signature, the nonce, the balance for gas, the chain ID and the intrinsic gas; malformed or unauthorised transactions never reach a contract.
6. **It is included in a block.** Hardhat mines it immediately. On a public chain this is where validators or miners compete; here it is deterministic.
7. **The contract executes.** The EVM runs `createEscrow`: validations, struct write, then `paymentToken.safeTransferFrom(buyer, escrow, 3,499e18)`, which calls the token contract, which checks the allowance and moves the balance.

8. **State changes.** The escrow's storage now holds a `Funded` record; the token's balances are updated; events are queued.
9. **A receipt is returned.** The app polls `eth_getTransactionReceipt`, sees `status: 1`, reads the logs, and refreshes the dashboard.

Table 1.3: The vocabulary of a transaction, with real values from this project

Term	Meaning	Where you meet it in ProofPay
Transaction hash	Keccak-256 of the signed transaction; its unique ID.	`0x74d678211c74efda145b9c712adae35bcd5a35616275ec8d76d0dcc3a232f4a6` was the release settlement at block 30.
Block number	Position of the block in the chain.	Settlement at block 24 (release), 29 (refund), 30 (release). Each was exactly one block higher than the previous.
Gas	Unit of computational work; you pay gas × gas price.	`createEscrow` consumed 156,144 gas after the first; `processReport` about 122,470.
Gas limit	Maximum gas you allow; unused gas is refunded.	Hardhat's `localhost` network sets a 16,000,000 block gas limit; the CRE write path configures `gasLimit: "1000000"`.
Gas price	Price per gas unit; irrelevant in value terms on a local chain.	Hardhat defaults; the demonstration never optimises it, and the report says so rather than quoting a mainnet figure.
Nonce	Per-account counter that orders that account's transactions and blocks replays.	A stuck nonce is the classic "transaction pending forever" cause; see Module 12.
Chain ID	Identifier of the network, signed into the transaction to prevent cross-chain replay.	31337 for Hardhat Local; the UI refuses writes unless `isCorrectNetwork` is true. Sepolia would be 11155111, mainnet 1.
RPC	Remote procedure call: the JSON-over-HTTP interface to a node.	`http://127.0.0.1:8545` for the node; viem's `http()` transport in the CRE workflow and the bridge.
EVM	Ethereum Virtual Machine: the deterministic computer that executes bytecode.	Every contract call is EVM execution; "EVM capability" in CRE means "read from that machine".
Account	An address with a balance and, for contracts, code and storage.	Twenty funded accounts exist on the Hardhat node; three roles matter: admin, buyer, seller.
Address	20 bytes; derived from a public key (EOA) or from deployer+nonce (contract).	`0x5FbD…0aa3` pUSD, `0xe7f1…0512` escrow, `0xB7f8…4F5e` receiver.
Private key	256-bit secret that signs transactions. Whoever holds it controls the account.	Never appears in the repository. Hardhat's accounts are unlocked, which is why the bridge can use `eth_sendTransaction` without a key.
Public key	Derived from the private key; used to verify signatures and to derive the address.	You never handle it directly in this project; the wallet does.

What happens when a transaction fails

Failure comes in three flavours, and telling them apart is an interview skill in itself. **Rejected in the wallet**: the user pressed cancel, so nothing was broadcast, no gas was spent, and the app should say "you rejected the request" rather than "transaction failed". **Reverted during execution**: the transaction was mined, gas was consumed, but the contract threw. In Solidity 0.8 a revert rolls back all state changes, which is why a failed settlement leaves the escrow untouched. The receipt shows `status: 0`, and modern tooling can decode the revert reason — ProofPay uses custom errors such as `EscrowNotFunded(orderId)` so the reason is precise and cheap. **Never mined**: the transaction sits in the mempool because the gas price is too low or a nonce gap exists; here it appears as "pending forever".

> **COMMON MISCONCEPTION**
>
> "A reverted transaction still transfers the tokens because the transfer happened before the revert." False. Reverts are atomic: the EVM discards every state change made by the whole call, including nested calls into the token contract. The only thing permanently consumed is gas, which is why a reverted settlement costs the sender fees but moves no pUSD.

1.4 Ethereum and the EVM

Ethereum is a network whose state is a key-value store maintained by thousands of nodes, plus a virtual machine that can modify that store according to code that users deploy. Solidity compiles to EVM bytecode; the bytecode is what actually runs. "EVM-compatible" networks (Polygon, Arbitrum, Base, BNB Chain, and your local Hardhat node) implement the same instruction set and RPC conventions, which is why the same contracts and the same front-end work on any of them — only the chain ID and addresses change.

There are two kinds of account, and ProofPay depends on both. An **externally owned account (EOA)** is controlled by a private key: the buyer, the seller, the admin. A **contract account** has code and storage and no key: the token, the escrow, the receiver. An EOA initiates transactions. A contract can only act when called, either by an EOA or by another contract — which is exactly how the receiver becomes the escrow's oracle: the admin calls the receiver, and the receiver calls `confirmDelivery`, so the escrow sees `msg.sender = receiver`, not the admin.

Table 1.4: EVM data locations, and why they matter here

Location	Lifetime and cost	ProofPay relevance
Storage	Permanent, on chain, expensive (roughly 20,000 gas for a fresh slot).	`_escrows`, `processedReports`, `orderSettled`, token balances and allowances. This is why `createEscrow` costs ~156k gas and why the escrow never stores the tracking ID itself.
Memory	Per-call, expandable, cheap, discarded at the end.	The decoded `Escrow memory` returned by `getEscrow`; string decoding inside the receiver.
Calldata	Read-only input of an external call; cheapest for large bytes.	`processReport(bytes metadata, bytes report)` takes the 256-byte report as calldata, which is why the bridge's transaction is 388 bytes of call data.
Logs / events	Written once, not readable by contracts, cheap-ish, permanent.	`EscrowCreated` is how the frontend discovers orders; `ReportProcessed` is the audit record of a settlement.

Events deserve one more sentence because beginners expect contracts to read them. They cannot. A log is a receipt for humans and indexers: it is stored in the transaction receipt, and only off-chain software (`eth_getLogs`, block explorers, your React app) consumes it. That is precisely why `useEscrows` uses logs to *discover* order IDs and then calls `getEscrow` to *read* the data: discovery is cheap and indexable; truth comes from storage.

1.5 Cryptographic foundations you will actually be asked about

Hashing. Keccak-256 maps any bytes to 32 bytes, deterministically and irreversibly. Properties that matter: the same input always yields the same output (so a report ID can be recomputed by anyone); a one-bit change scrambles the whole output (so tampering is detectable); and you cannot feasibly find two inputs with the same output (so an attacker cannot substitute a different report for the same ID). In this project, hashing appears in the block chain itself, in order IDs, in `keccak256(toHex(trackingId))` inside the report, in report IDs, and in ABI function selectors.

Signatures. Ethereum uses ECDSA over secp256k1. A signature proves that the holder of a private key approved a specific 32-byte digest. The digest for a transaction covers the chain ID, nonce, gas, recipient, value and data, so a signature cannot be replayed on another chain or reused for a different call. When MetaMask shows you a scary-looking hex blob and asks you to confirm, that is the digest you are authorising.

Address derivation. Address = last 20 bytes of `keccak256(publicKey)` for an EOA. For a contract, the address is `keccak256(rlp(sender, nonce))[12:]` — which is why, on a fresh Hardhat node, deployments land at the familiar sequence `0x5FbD…`, `0xe7f1…` from the same account: same deployer, increasing nonce.

Why contracts cannot trust arbitrary external information. This is the seed of Module 7. A contract is deterministic: every node must compute the same result from the same inputs, or consensus breaks. If Solidity could perform an HTTP GET, two nodes could receive different responses (or none), and the chain would fork on the answer. Therefore external facts must be *pushed* into the chain by a transaction, and the only question that matters is who is allowed to push, and what evidence they attach. Everything in this project's oracle design — the receiver's authorisation, the report's provenance fields, the replay protection — exists because of that single constraint.

> **INTERVIEW CHECKPOINT — MODULE 1**
>
> **Q.** "Why can't the escrow contract just call the courier's API when the parcel is delivered?" **Model answer.** Because execution must be deterministic and every node must agree on the same state transition; an HTTP call is non-deterministic and unavailable inside the EVM. So the fact has to arrive as part of a transaction, and something off chain must decide to send it. In ProofPay that something is a Chainlink CRE workflow, and the contract's protection is that it will only accept the report from an authorised caller, only once, and only while the order is still funded.

MODULE 2

METAMASK, EXPLAINED COMPLETELY

MetaMask is where most beginners' understanding collapses into "the wallet thing". This module fixes that. By the end you should be able to explain what MetaMask stores, what it refuses to store, what it does when a site calls it, and why ProofPay needs two separate confirmations to fund one escrow.

2.1 What MetaMask actually is

MetaMask is a browser extension that (a) holds encrypted private keys, (b) signs transactions and messages on your behalf, and (c) injects a JavaScript object into every page at `window.ethereum` so that websites can request those signatures. It is also a lightweight wallet UI and a network switcher.

What it stores: an encrypted vault containing your seed phrase and derived keys, your selected network and account, and a local transaction history. What it does **not** store: your cryptocurrency. The tokens and ETH are entries in the blockchain's state. If you delete the extension and reinstall it with the same seed phrase, your balance reappears instantly, because the balance never lived in the extension. This is the single most useful sentence for explaining wallets to a non-technical audience.

An **address** is your public identifier: `0x7099…79C8`. Anyone can send to it; nobody can spend from it without the key. ProofPay displays addresses shortened for readability (`0x7099…79c8`) but always sends full 20-byte values to the chain, because a shortened address is not a valid address.

2.2 Private keys, seed phrases, importing versus connecting

A **seed phrase** (12 or 24 words) is the master secret. Every account in the wallet is derived from it by a deterministic process (BIP-39 for the mnemonic, BIP-32/BIP-44 for derivation paths). Importing an account means giving the wallet either the seed phrase or a raw private key so that it can control that account. Connecting a site means granting that site permission to *see your address and ask for signatures* — it never gives the site your key.

Table 2.1: Connect versus import — a distinction interviewers test

Action	What you hand over	What the site can then do	Risk
Connect (ProofPay's "Connect wallet")	Nothing secret. The wallet reveals the selected address after you approve.	Read your address and balances; ask you to sign transactions or messages, each of which you must approve.	Low. Revocable from MetaMask's "Connected sites".
Import account	The seed phrase or a private key.	Everything: full control of the account.	Catastrophic if the site is malicious. Legitimate only inside the wallet's own UI.

MetaMask asks for approval on every transaction because a signature is irreversible. There is no "undo" for a mined transaction and no customer-support number to call. The prompt is the last line of defence, which is why ProofPay's UI always shows the amount and the recipient before triggering it, and why the escrow contract independently revalidates everything: never rely on a UI to enforce a rule.

> **LOCAL ACCOUNTS ONLY**
>
> Hardhat prints twenty accounts with their private keys when you run `npx hardhat node`. Those keys are public, published in Hardhat's documentation, and identical every time. They exist only for local development. Never send real funds to them, never reuse them on a public network, and never paste them into a website. This guide deliberately contains no key material.

2.3 Accounts, networks, chain IDs and our local configuration

One wallet can hold many accounts (different addresses derived from the same seed) and can point at many networks (different chains). The **chain ID** is the crucial field: it is included in the signed transaction, so a transaction signed for chain 31337 cannot be replayed on Ethereum mainnet. The RPC URL is where the wallet sends requests for the selected network.

Table 2.2: ProofPay's local network, verified from hardhat.config.js and the running node

Setting	Value in this project
Network name in MetaMask	Hardhat Local (any label works; the chain ID is what matters)
RPC URL	`http://127.0.0.1:8545`
Chain ID	`31337` (hex `0x7a69`)
Currency symbol	ETH (test ETH only; it pays gas, not purchases)
Backend	`http://localhost:5001`
Frontend	`http://localhost:5173`
Accounts (Hardhat defaults)	Admin/owner/manual oracle `0xf39F…2266`; buyer `0x7099…79C8`; seller `0x3C44…93BC`
Why the network must match	Contracts live at fixed addresses on one chain. If MetaMask points at mainnet, the app would be reading a chain where `0xe7f1…0512` is either empty or an unrelated contract. ProofPay guards this by checking `chainId === 31337` before offering any write action.

2.4 How MetaMask talks to React: EIP-1193 in the actual code

EIP-1193 standardises the injected provider: a single object with a `request({ method, params })` method that returns a promise. ProofPay uses exactly six methods. Understanding what each one does, and when it pops up a window, is the difference between debugging in seconds and guessing for an hour.

Table 2.3: The EIP-1193 methods ProofPay uses

Method	Prompts?	Purpose and where it appears
`eth_requestAccounts`	Yes (first time)	Asks the user to connect. Used by `connect()` in `useWallet`; returns an array of addresses.
`eth_accounts`	No	Returns already-authorised accounts. Used on mount for silent rehydration, so a returning user is not asked again.
`eth_chainId`	No	Returns the current chain as a hex string. `parseChainId` converts it and the result drives `isCorrectNetwork`.
`eth_getBalance`	No	Reads native ETH for the header. Note this is gas money, not pUSD.
`eth_call`	No	Executes a contract function read-only. ProofPay uses it for `balanceOf`, `getEscrow`, `manualOracle`.
`eth_sendTransaction`	Yes	Requests a signature and broadcast. Used for approve, createEscrow, processReport and the manual settlements.
`wallet_switchEthereumChain`	Yes	Asks MetaMask to switch networks; if the chain is unknown it fails with code 4902 and the app then calls `wallet_addEthereumChain`.
`accountsChanged` / `chainChanged`	Events	Emitted when the user switches account or network. `useWallet` subscribes and re-reads state, which is why the dashboards follow the active account without a page reload.

Here is the actual core of the project's wallet handling, shortened but not rewritten:

frontend/src/store/useWallet.ts — silent rehydration and event subscriptions

```
// On mount: no popup. If the site is already authorised, adopt the account.
const rehydrate = async () => {
  if (sessionStorage.getItem(MANUAL_DISCONNECT_KEY) === '1') return;
  const accounts = await provider.request<string[]>({ method: 'eth_accounts' });
  if (cancelled || accounts.length === 0) return;
  await applyAddress(accounts[0]);
};

// Live updates: the wallet tells us, we never poll for account changes.
provider.on('accountsChanged', handleAccountsChanged);
provider.on('chainChanged', handleChainChanged);
```

`applyAddress` is the single place that turns an address into UI state: it reads the chain ID, reads the native balance, and sets `isCorrectNetwork` by comparing against `HARDHAT_LOCAL.chainId`. Everything else in the app — dashboards, checkout, oracle panel — consumes that one object. If you are asked "how does the UI know the wallet changed?", the answer is: it subscribes to the provider's events and re-runs `applyAddress`; there is no polling loop.

2.5 The real transaction lifecycle in ProofPay

Funding one escrow involves **two** transactions, and the reason is an ERC-20 design decision, not an accident. A contract cannot move tokens out of your wallet unless you have authorised it to. That authorisation is itself a state change on the token contract, so it needs its own transaction.

Table 2.4: Two transactions, one purchase

#	Transaction	Who executes what	What it means / what you see
1	`approve(escrow, 3499e18)` on pUSD	Buyer → token contract. Storage: `allowance[buyer][escrow] = 3499e18`.	MetaMask shows a call to the token with a spending cap. After mining, the checkout shows "approved". No money has moved yet.
2	`createEscrow(orderId, seller, 3499e18)` on escrow	Buyer → escrow, which calls `transferFrom(buyer, escrow, 3499e18)` on the token.	MetaMask shows a second confirmation. On success the allowance is consumed and the escrow holds 3,499 pUSD in the `Funded` state.

Why not one transaction? Because the token contract is the only thing that may debit your balance, and it will only do so for an address you have approved. Combining them would require the token to trust an arbitrary caller, which would let any contract drain any wallet. The allowance indirection is the security feature; the extra click is its cost. (EIP-2612 `permit` and EIP-3009 `transferWithAuthorization` allow signature-based single-transaction flows for tokens that implement them; pUSD, built on OpenZeppelin's plain ERC-20, does not.)

Signing a message versus signing a transaction. A message signature (EIP-191/712) proves you control an address at a point in time; it changes no state and costs no gas. A transaction signature authorises a state change, is broadcast, costs gas and produces a receipt. ProofPay's flows are all transactions; the only message-ish interaction is the wallet connection itself.

Table 2.5: Four things beginners conflate

Action	What changes	How you verify it
Wallet connection	Nothing on chain. A permission exists inside the extension.	The address appears in the header; no transaction hash exists.
Token approval	Token contract's allowance mapping.	`allowance(owner, spender)`; a receipt from the token address; an `Approval` event.
Token transfer	Balances of two accounts.	`balanceOf`; a `Transfer` event.
Contract interaction	Whatever the contract's code writes.	Contract-specific events (`EscrowCreated`, `ReportProcessed`) plus a changed `getEscrow` result.

2.6 Practical MetaMask walkthrough for this project

Do this once, slowly, with the three terminals running. You will understand more from ten minutes of this than from an hour of reading.

1. **Start the chain.** `cd ~/Desktop/Minor_2/contracts && npx hardhat node`. Leave it running. Note the twenty printed accounts — those are your local identities.
2. **Deploy.** In a second terminal, from `contracts`, run the project's deploy scripts (`deploy.js`, then `deploy-receiver.js`, then `authorize-receiver.js`). This writes `contracts/deployments/cre-receiver-localhost.json`, which is your record of what is deployed where.
3. **Install MetaMask** and create a wallet. Write the seed phrase on paper. It controls only test accounts in this workflow.
4. **Add the network.** Networks → Add network → Add manually. RPC `http://127.0.0.1:8545`, chain ID `31337`, symbol ETH. Save and switch to it.
5. **Import the three roles.** Account menu → Import account → paste the private key that Hardhat printed for account 0 (admin), then repeat for account 1 (buyer) and account 2 (seller). Importing is legitimate here because these keys are public test keys; you are teaching yourself role switching.
6. **Connect.** Open `http://localhost:5173`, click Connect wallet, choose an account. The header should show "Hardhat Local" and a pUSD balance. If it shows a red network warning, you are on the wrong chain.
7. **Import the token** so MetaMask can display it: Import tokens → custom address `0x5FbDB2315678afecb367f032d93F642f64180aa3`, symbol pUSD, decimals 18.
8. **Act as buyer.** Go to Checkout, choose a product, confirm the seller address, submit. Approve the first prompt (allowance), then the second (escrow). Watch the checkout move through "approving" → "approved" → "creating" → confirmed.
9. **Copy the transaction hashes** from the UI. Paste one into a terminal: `cast receipt <hash> --rpc-url http://127.0.0.1:8545` if you have Foundry, or read it via the project's viem/ethers scripts. Confirm `status 1` and note the block number.
10. **Act as seller.** Switch account, open Incoming Escrows, select the funded order, register a shipment with a tracking ID beginning `PPD-`. This writes to the Express API, not the chain.
11. **Act as admin.** Switch account, open the admin dashboard, confirm that `escrow.oracle()` equals the receiver and that `manualOracle()` equals this account. That panel is your proof the wiring is correct.

> **IF SOMETHING LOOKS WRONG**
>
> A zero pUSD balance usually means the token is not imported or the admin never minted to that account. A missing "Fund escrow" button usually means the wrong network. A revert on `createEscrow` with `ERC20InsufficientAllowance` means the first transaction was replaced or never confirmed. Module 12 turns each of these into a diagnosis procedure.

> **INTERVIEW CHECKPOINT — MODULE 2**
>
> **Q.** "If MetaMask doesn't store coins, what happens if I uninstall it?" **A.** The coins are entries in the blockchain; they remain untouched. Reinstalling and restoring the seed phrase recreates the keys and therefore access. What you lose by discarding the seed phrase is the ability to sign, not the asset.
>
> **Q.** "Why does ProofPay ask for two confirmations?" **A.** ERC-20 separates permission (allowance) from movement (transferFrom). The first transaction grants the escrow a spending cap; the second spends it. That indirection is what stops arbitrary contracts from draining wallets.

MODULE 3

SMART CONTRACTS AND SOLIDITY FROM ZERO

This module is the centre of the book. Everything else in ProofPay exists to serve the three contracts: the frontend calls them, the backend checks them, the oracle reports into them, and the tests pin their behaviour. Read this module twice — once for the language, once for the design.

3.1 What a smart contract is, precisely

A smart contract is a program deployed to a blockchain, where it receives an address, keeps its own storage, and can only be changed by sending it a transaction. Three properties distinguish it from a backend function:

- **Deterministic execution.** Every node computes the same result from the same input. No randomness, no clock, no network, no filesystem. `block.timestamp` is the only time available, and it is the block's time, agreed by consensus.
- **Persistent, public state.** Storage survives forever and anyone can read it. That is why ProofPay writes only the escrow's essentials and keeps product names, descriptions and tracking IDs off chain.
- **No owner override.** Once deployed, the code cannot be patched. ProofPay's escrow deliberately has no pause, upgrade or sweep function; the only mutable configuration is the oracle address.

The practical consequence: a bug in a backend can be fixed in ten minutes; a bug in a contract that holds money is permanent. That asymmetry explains the project's conservatism — small functions, explicit checks, OpenZeppelin for anything standard, and 31 tests for 300 lines of Solidity.

3.2 Solidity in one page, using ProofPay's own constructs

Table 3.1: Solidity constructs you must be able to name, with ProofPay examples

Construct	Meaning	In this project
`pragma solidity`	Compiler version constraint.	`^0.8.20` in all three contracts, compiled with 0.8.28.
`contract`	A unit of code plus storage; deployed once, called many times.	`ProofPayUSD`, `ProofPayEscrow`, `ProofPayCREReceiver`.
`constructor`	Runs once at deployment; sets immutable values.	Escrow takes the token address and owner; receiver takes escrow, owner, forwarder, manual oracle.
State variable	Permanent storage slot.	`paymentToken` (immutable), `oracle`, `_escrows`, `processedReports`.
Visibility	`public` (callable outside + getter), `external` (callable outside only), `internal` (this contract and children), `private` (this contract).	`_getFundedEscrow` is private; `_processReport` is internal; settlement functions are external.
`view` / `pure`	Reads state / reads nothing. Costs no gas when called externally with `eth_call`.	`getEscrow` is `view`; `decimals()` is `pure`.
`payable`	Allows ETH to be sent with the call.	Not used: pUSD is the payment asset, ETH is only gas. That absence is a design statement.
`mapping`	Key-value store; every key exists conceptually with a zero value.	`mapping(bytes32 => Escrow)`, `mapping(bytes32 => bool)`.
`struct`	Grouped fields.	`Escrow { orderId, buyer, seller, amount, createdAt, settledAt, status }`.

Construct	Meaning	In this project
`enum`	Named integers, 0..n-1.	`None=0, Funded=1, Released=2, Refunded=3`; the numbers matter because CRE and the bridge compare against `1`.
`modifier`	Reusable pre/post condition wrapped around a function body.	`onlyOracle`, `nonReentrant`, `onlyOwner`, `onlyAuthorized`.
`event`	Append-only log; not readable by contracts.	`EscrowCreated`, `ReportProcessed`, `SettlementExecuted`.
Custom error	Cheap typed revert with arguments.	`error EscrowNotFunded(bytes32 orderId)`; the tests assert on the exact error.
`require` / `revert`	Abort and undo all state changes.	Every validation branch uses `if (...) revert ErrorName();`.
Inheritance / interfaces	Reuse and type-safe external calls.	`is ERC20, Ownable`; `IEscrowSettlement`, `IReceiver`, `IERC165`.
`msg.sender`	The immediate caller — a contract's call has the contract as sender.	The escrow sees the *receiver* as sender during settlement, not the admin. This is the whole trick.
`msg.value`	ETH attached to the call.	Always zero here; no function is payable.
`block.timestamp`	Current block time in seconds.	Stored as `uint64` in `createdAt` and `settledAt` to pack storage.

A 15-LINE CONTRACT, SO THE SYNTAX HAS SOMEWHERE TO LAND

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Vault {
    mapping(address => uint256) public balances;   // storage
    error NotEnough(uint256 have, uint256 want);   // custom error

    function deposit() external payable {          // payable: accepts ETH
        balances[msg.sender] += msg.value;         // state write
    }

    function withdraw(uint256 amount) external {
        uint256 have = balances[msg.sender];
        if (have < amount) revert NotEnough(have, amount);
        balances[msg.sender] = have - amount;      // write BEFORE send
        payable(msg.sender).transfer(amount);      // external call LAST
    }
}
```

The ordering in `withdraw` — check, then write, then transfer — is the "checks-effects-interactions" pattern. ProofPay's escrow follows it too, and additionally wraps each state-changing function in `nonReentrant`.

3.3 ERC-20, from first principles

A token is not a special kind of coin; it is a row in a contract's storage. ERC-20 is a standard interface for such rows so that wallets, exchanges and other contracts can treat every token the same way. The standard requires six functions (`totalSupply`, `balanceOf`, `transfer`, `allowance`, `approve`, `transferFrom`) and two events (`Transfer`, `Approval`).

Table 3.2: ERC-20 concepts, and exactly where ProofPay uses them

Concept	Meaning	ProofPay use

Concept	Meaning	ProofPay use
Fungible	Every unit is identical; 1 pUSD = 1 pUSD.	Correct for money. A courier receipt would be non-fungible (ERC-721), which this project does not use.
Decimals / `decimals()`	Display scaling only. All arithmetic uses integers.	18, so 3,499 pUSD is `3499 * 10^18` = 3499000000000000000000. The UI's `parseTokenAmount` does the multiplication; `formatTokenAmount` reverses it.
`balanceOf`	Balance of one address.	Dashboard header; the evidence script's before/after check (+100 exactly).
`allowance`	How much a spender may still take from an owner.	Read by the checkout before offering "Fund"; consumed inside `createEscrow`.
`approve`	Sets that cap.	Transaction 1 of a purchase; 46,383 gas in this deployment.
`transfer`	Owner moves their own tokens.	Used by the escrow at settlement: `safeTransfer(seller, amount)`.
`transferFrom`	A spender moves tokens within the allowance.	Used at funding: `safeTransferFrom(buyer, escrow, amount)`. This is the only way pUSD enters custody.
`mint` / `burn`	Create / destroy supply. Not in the standard; an extension.	Only `mint` exists, owner-only. No burn: escrowed value must not be destroyable.
Events	Transfer/Approval logs.	How block explorers and the UI show token movement without calling `balanceOf` for every address.

Why are `approve` and `transferFrom` separate? Because otherwise any contract you call could take everything you own. The allowance is a capability you grant deliberately, for a specific spender, up to a specific amount. ProofPay's UI approves the exact order amount rather than an unlimited allowance, so a bug in the escrow cannot drain the wallet beyond one order.

1. Buyer calls `approve(escrow, 3499e18)` on pUSD. Storage: `allowance[buyer][escrow] = 3499e18`.
2. Buyer calls `createEscrow(orderId, seller, 3499e18)` on the escrow.
3. Escrow calls `pUSD.transferFrom(buyer, escrow, 3499e18)`.
4. pUSD checks `allowance[buyer][escrow] >= 3499e18` and `balanceOf(buyer) >= 3499e18`, moves the balance, decrements the allowance, emits `Transfer`.
5. Escrow emits `EscrowCreated` and returns. The buyer's wallet is 3,499 pUSD lighter; the escrow contract is 3,499 pUSD heavier; nobody can spend it.

3.4 ProofPayUSD.sol, line by line

The entire token is sixteen lines. Its restraint is the point: every added feature would be another way to lose money.

contracts/contracts/ProofPayUSD.sol (complete file)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract ProofPayUSD is ERC20, Ownable {
    constructor(address initialOwner) ERC20("ProofPay USD", "pUSD") Ownable(initialOwner) {}

    function decimals() public pure override returns (uint8) {
        return 18;
    }

    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }
}
```

Table 3.3: What each line of ProofPayUSD does, and why it is written that way

Line	Explanation
`contract ProofPayUSD is ERC20, Ownable`	Inherits the whole standard implementation (balances, allowances, transfers, events) and OpenZeppelin's ownership module (an `owner()` address plus `onlyOwner`). Nothing about ERC-20 is hand-written, which eliminates a whole class of bugs.
`constructor(address initialOwner) ERC20("ProofPay USD","pUSD") Ownable(initialOwner) {}`	Sets the display name and symbol, and makes the deployer's chosen address the owner. Empty body: everything needed is done by the two base constructors. A zero owner would revert with `OwnableInvalidOwner` — tested.
`decimals() override returns 18`	Explicit even though 18 is OpenZeppelin's default. Stating it makes the scaling assumption visible in the source and in the report, and it is marked `pure` because it reads nothing.
`mint(address,uint256) external onlyOwner`	The only way supply grows. `external` because it is always called from outside; `onlyOwner` because an open mint would let anyone print money and destroy the escrow's meaning. Delegates to `_mint`, which also rejects the zero address.

What is deliberately missing: no burn (escrowed value must never be destroyable), no pause (a pause would give the owner a way to freeze everyone's funds), no fee-on-transfer (that would break the escrow's accounting, which asserts an exact amount arrives), no permit (not needed by this flow), no cap (the demonstration mints on demand). Because the token is plain ERC-20, the escrow can use `SafeERC20` and behave predictably.

Lifecycle. Deploy with owner = admin (block 1, gas 618,390). Mint 1,000,000 pUSD to the admin (block 4), then distribute to buyer and seller (blocks 5–6). Buyer approves the escrow per order. Escrow pulls funds with `transferFrom`. At settlement the escrow calls `transfer` to the seller or back to the buyer. Total supply at block 32 was 1,010,100 pUSD, which is simply the initial million plus the top-ups used to seed demonstration orders.

3.5 ProofPayEscrow.sol, the heart of the system

The escrow answers one question: who is allowed to move this money, and when? It holds pUSD, records one struct per order, and exposes exactly two terminal transitions. There is no `withdraw`, no admin rescue, no upgrade. If the oracle disappears, the money stays locked — a deliberate failure mode, because "stuck" is recoverable by governance while "drained" is not.

contracts/contracts/ProofPayEscrow.sol — state and types (lines 10–48)

```
enum EscrowStatus { None, Funded, Released, Refunded }

struct Escrow {
    bytes32 orderId;
    address buyer;
    address seller;
    uint256 amount;
    uint64 createdAt;
    uint64 settledAt;
    EscrowStatus status;
}

IERC20 public immutable paymentToken;
address public oracle;
mapping(bytes32 => Escrow) private _escrows;
```

Two details repay attention. `paymentToken` is `immutable`: set once in the constructor and inlined into the bytecode, so it can never be pointed at a malicious token later. The timestamps are `uint64` rather than `uint256`: they fit comfortably until the year 584 billion, and packing two 64-bit values with an address and an enum into fewer storage slots saves gas.

Table 3.4: The escrow's complete state machine

State	Value	How you get here	What is possible next
`None`	0	Default for every order ID that was never used. `getEscrow` reverts rather than returning this.	`createEscrow` (only if the ID is unused).
`Funded`	1	`createEscrow` after validations and a successful `transferFrom`.	`confirmDelivery` or `confirmFailure`, both oracle-only. Nothing else. It can stay Funded indefinitely.
`Released`	2	`confirmDelivery`: seller paid, `settledAt` set.	Nothing. A second settlement reverts `EscrowNotFunded`.
`Refunded`	3	`confirmFailure`: buyer repaid, `settledAt` set.	Nothing. Same protection.

createEscrow — the only way in

ProofPayEscrow.sol lines 79–118

```
function createEscrow(bytes32 orderId, address buyer, address seller, uint256 amount)
    external nonReentrant
{
    if (orderId == bytes32(0)) revert InvalidOrderId();
    if (buyer == address(0) || buyer != msg.sender) revert InvalidBuyer();
    if (seller == address(0)) revert InvalidSeller();
    if (buyer == seller) revert SameBuyerAndSeller();
    if (amount == 0) revert InvalidAmount();
    if (_escrows[orderId].status != EscrowStatus.None) revert EscrowAlreadyExists(orderId);

    _escrows[orderId] = Escrow({ orderId: orderId, buyer: buyer, seller: seller, amount: amount,
        createdAt: uint64(block.timestamp), settledAt: 0, status: EscrowStatus.Funded });

    paymentToken.safeTransferFrom(buyer, address(this), amount);
    emit EscrowCreated(orderId, buyer, seller, amount);
}
```

Table 3.5: createEscrow, function contract

Question	Answer
Purpose	Open an escrow and take custody of the buyer's pUSD.

Question	Answer
Caller	The buyer, from the checkout page, via MetaMask.
Parameters	`orderId` (random 32 bytes generated client-side), `buyer` (must equal `msg.sender`), `seller`, `amount` in the token's smallest unit.
Why `buyer` is a parameter at all	An explicit argument makes the intent readable and lets the contract prove `buyer == msg.sender`. Without that check, someone could fund an escrow in another person's name using their own allowance, producing a confusing ownership record.
Internal flow	Six validations, write the struct, then `safeTransferFrom`, then emit. Note the order: state first, external call last, which is checks-effects-interactions.
Storage changes	`_escrows[orderId]` gains a `Funded` record; the token updates two balances and one allowance.
Events	`EscrowCreated(orderId indexed, buyer indexed, seller indexed, amount)` plus the token's `Transfer`.
Reverts	`InvalidOrderId`, `InvalidBuyer`, `InvalidSeller`, `SameBuyerAndSeller`, `InvalidAmount`, `EscrowAlreadyExists`, and from the token: `ERC20InsufficientAllowance` / `ERC20InsufficientBalance`.
How it protects funds	The only address that can ever move the deposit afterwards is `oracle`, because both settlement functions are `onlyOracle`. `buyer == msg.sender` stops third-party funding; the uniqueness check stops a second deposit overwriting a live one.

confirmDelivery and confirmFailure — the only ways out

ProofPayEscrow.sol lines 119–139

```
function confirmDelivery(bytes32 orderId) external onlyOracle nonReentrant {
    Escrow storage escrow = _getFundedEscrow(orderId);
    escrow.status = EscrowStatus.Released;
    escrow.settledAt = uint64(block.timestamp);
    paymentToken.safeTransfer(escrow.seller, escrow.amount);
    emit EscrowReleased(orderId, escrow.seller, escrow.amount);
}

function confirmFailure(bytes32 orderId) external onlyOracle nonReentrant {
    Escrow storage escrow = _getFundedEscrow(orderId);
    escrow.status = EscrowStatus.Refunded;
    escrow.settledAt = uint64(block.timestamp);
    paymentToken.safeTransfer(escrow.buyer, escrow.amount);
    emit EscrowRefunded(orderId, escrow.buyer, escrow.amount);
}
```

The private helper is where the real invariant lives:

ProofPayEscrow.sol lines 141–158 — read path and funding guard

```
function getEscrow(bytes32 orderId) external view returns (Escrow memory) {
    Escrow memory escrow = _escrows[orderId];
    if (escrow.status == EscrowStatus.None) revert EscrowNotFound(orderId);
    return escrow;
}

function _getFundedEscrow(bytes32 orderId) private view returns (Escrow storage escrow) {
    escrow = _escrows[orderId];
    if (escrow.status == EscrowStatus.None) revert EscrowNotFound(orderId);
    if (escrow.status != EscrowStatus.Funded) revert EscrowNotFunded(orderId);
}
```

Read those two functions together and you see the double-settlement defence at the custody layer: settlement requires `status == Funded`, and settlement immediately writes `Released` or `Refunded`. A second attempt therefore fails at `_getFundedEscrow` before any transfer. That is true even if a future receiver, a buggy script or a stolen key

calls it again. Defence in depth: the receiver also tries to prevent the second call, but the escrow does not depend on it.

> **WHY NOT SEND FUNDS STRAIGHT TO THE SELLER AND REFUND FROM THE SELLER IF LOST?**
>
> Because the seller would have to be trusted to return money. Escrow inverts the trust: the money sits in a contract that neither party controls, and the only question is who can prove the condition. That is also why the escrow has no owner-withdrawal path — an "emergency" admin function is exactly the backdoor users would criticise in an audit.

setOracle and the authority chain

```
function setOracle(address newOracle) external onlyOwner {
    if (newOracle == address(0)) revert InvalidOracle();
    address previousOracle = oracle;
    oracle = newOracle;
    emit OracleUpdated(previousOracle, newOracle);
}
```

One line of configuration decides who may settle every order. In this deployment the owner is the admin EOA and the oracle is the receiver contract, set in block 19 for 30,539 gas. That separation is deliberate: the owner key configures, the receiver executes. When the project moves to a DON, only this address changes.

3.6 ProofPayCREReceiver.sol, the policy boundary

Why does a second contract exist between the oracle and the money? Three reasons. It accepts the exact interface a Chainlink DON calls (`onReport(bytes,bytes)`). It centralises authorisation, decoding, replay protection and duplicate-settlement checks in one auditable place. And it lets the escrow stay dumb: the escrow knows nothing about reports, outcomes or CRE, only "pay seller" and "pay buyer".

ProofPayCREReceiver.sol — state and authorisation (lines 40–96)

```
IEscrowSettlement public immutable escrow;
address public forwarder;        // authorised DON forwarder
address public manualOracle;     // authorised local/backup settler
mapping(bytes32 => bool) public processedReports;
mapping(bytes32 => bool) public orderSettled;

modifier onlyAuthorized() {
    if (msg.sender == forwarder) { _; return; }
    if (manualOracle != address(0) && msg.sender == manualOracle) { _; return; }
    if (owner() == msg.sender && forwarder == address(0)) { _; return; }
    revert UnauthorizedCaller(msg.sender);
}
```

Read the third branch carefully, because it is the one to be honest about. When no forwarder is configured — as in this local deployment, where the forwarder is the zero address — the owner may also settle. That is a convenience for local work and a weakening of the model; a production deployment would set a forwarder and delete that branch. The report and this guide both say so rather than pretending the fallback is a feature.

ProofPayCREReceiver.sol lines 116–168 — entry points and processing

```
function onReport(bytes calldata metadata, bytes calldata report) external override onlyAuthorized {
    _processReport(metadata, report);
}
function processReport(bytes calldata metadata, bytes calldata report) external onlyAuthorized {
    _processReport(metadata, report);
}

function _processReport(bytes calldata metadata, bytes calldata report) internal {
    (bytes32 orderId, string memory outcome, bytes32 trackingIdHash, string memory observedAt)
        = abi.decode(report, (bytes32, string, bytes32, string));

    if (orderId == bytes32(0)) revert InvalidReportData();

    bytes32 reportId = keccak256(
        abi.encodePacked(orderId, outcome, trackingIdHash, observedAt, metadata)
    );
    if (processedReports[reportId]) revert ReportAlreadyProcessed(reportId);
    if (orderSettled[orderId]) revert OrderAlreadySettled(orderId);

    try this._checkEscrowFunded(orderId) returns (bool funded) {
        if (!funded) revert EscrowNotFunded(orderId);
    } catch { revert EscrowNotFound(orderId); }

    bytes32 outcomeHash = keccak256(bytes(outcome));
    if (outcomeHash == keccak256(bytes("DELIVERED"))) {
        escrow.confirmDelivery(orderId);
        processedReports[reportId] = true;
        orderSettled[orderId] = true;
        emit ReportProcessed(orderId, outcome, reportId);
        emit SettlementExecuted(orderId, true);
    } else if (outcomeHash == keccak256(bytes("LOST"))) {
        escrow.confirmFailure(orderId);
        processedReports[reportId] = true;
        orderSettled[orderId] = true;
        emit ReportProcessed(orderId, outcome, reportId);
        emit SettlementExecuted(orderId, false);
    } else {
        revert UnsupportedOutcome(outcome);
    }
}
```

Table 3.6: What each stage of _processReport is defending against

Stage	Attack or mistake it prevents	What happens on failure
`abi.decode(...)`	Garbage bytes, a truncated payload, or a payload with the wrong field types.	Decoding reverts; the transaction fails with no state change.
`orderId == 0`	A default or empty report that would otherwise hash to something and mark an order settled.	`InvalidReportData`.
`reportId` check	Replay: sending identical bytes twice, after a restart or from two bridges.	`ReportAlreadyProcessed(reportId)`; the second transaction is refused, the first stays final.
`orderSettled` check	A *different* report for an order already paid: different timestamp, different metadata, same order.	`OrderAlreadySettled(orderId)`.
Funding self-call	Settling a stale report whose escrow was already closed, or a nonexistent order.	`EscrowNotFunded` / `EscrowNotFound`.
Outcome hash branch	Any outcome other than the two supported strings, including a workflow bug that emitted `MANUAL_REVIEW`.	`UnsupportedOutcome(outcome)`; nothing is paid.

Note what the receiver does **not** do: it does not verify a signature, because authorisation comes from the caller's address; it does not re-derive the outcome from anything, because the outcome is CRE's decision; and it does not know the seller or the amount, because the escrow owns those facts. It is a policy boundary, and keeping it narrow is what makes it reviewable.

3.7 Solidity security, using your own code as the case study

Reentrancy

Reentrancy is an attack where an external call lets the callee call back into your contract before your state is final. With a token that had a malicious hook, `safeTransfer` could re-enter `confirmDelivery` and pay twice. Two defences stop it here. `ReentrancyGuard` sets a storage flag so a nested call reverts with `ReentrancyGuardReentrantCall`. And the state is already terminal before the transfer, so even without the guard a second entry would hit `EscrowNotFunded`. The test suite includes a reentrant `createEscrow` case precisely to keep both defences honest.

SafeERC20

Some tokens return `false` instead of reverting on failure, and some return nothing at all. A plain `transfer` call that ignores the return value can therefore "succeed" without moving anything. `SafeERC20` checks the return data and reverts if the call failed, so the escrow's accounting cannot drift from reality. pUSD itself behaves, but the escrow is written to survive a less cooperative asset.

Access control and the principle of least authority

Table 3.7: Every privileged action in the system

Action	Who can do it	Why that role, and what it cannot do
`mint`	`owner` of the token (admin EOA)	Needed to create the demonstration economy. Cannot move anyone's existing balance.
`setOracle`	`owner` of the escrow	Points settlement at the receiver. Cannot settle directly — no such function exists.
`setForwarder`, `setManualOracle`	`owner` of the receiver	Configures who may deliver reports. Rotation is logged via events.
`confirmDelivery`, `confirmFailure`	`oracle` (the receiver contract)	The only path to money movement. No EOA holds this role.
`processReport`, `onReport`	forwarder, manual oracle, or owner-with-no-forwarder	Delivers a report; still bound by replay, duplicate and funding checks, and still cannot choose the payee.

Integer handling

Solidity 0.8 checks arithmetic by default: overflow and underflow revert instead of wrapping. That is why ProofPay needs no `SafeMath`. Balances are `uint256` in the token's smallest unit, so 3,499 pUSD is 3499×10^{18}, an integer with no rounding. Comparisons such as `amount == 0` are exact; there are no floating-point traps anywhere in the money path, and the UI's formatting is presentation only.

INTERVIEW CHECKPOINT — MODULE 3

Q. "Where exactly do you prevent double payment?" **A.** In three independent places. The escrow requires `status == Funded` and flips it to a terminal state before transferring. The receiver keeps `processedReports` for identical reports and `orderSettled` for any second report on the same order. The bridge refuses to submit a report ID it already recorded. Any one of the three would stop the common case; all three together stop retries after crashes, two competing bridges, and a buggy new caller.

Q. "Can the owner steal the escrowed money?" **A.** Not directly. The owner can point the oracle at a contract that always releases, which is a governance risk, not a code-level drain — and it is logged by `OracleUpdated`. There is no function that transfers escrowed pUSD to an arbitrary address.

MODULE 4

HARDHAT AND LOCAL BLOCKCHAIN DEVELOPMENT

Hardhat is the tool that turns Solidity text into a running system: it compiles, it deploys, it tests, and it can run a complete Ethereum node on your laptop. Everything you demonstrated in this project — a funded escrow, a CRE-mediated settlement at block 30 — happened on the chain that Hardhat started.

4.1 What Hardhat does, and why this project uses it

Hardhat is a development environment for Ethereum. In this repository it provides five things: a Solidity compiler pipeline (`npx hardhat compile`), a test runner built on Mocha with chain-aware assertions, a scripting environment where you write JavaScript that talks to the chain, a local node (`npx hardhat node`) with deterministic accounts, and a network abstraction so the same scripts can target `localhost` or a public network by changing one flag.

contracts/hardhat.config.js (complete)

```
require("dotenv").config();
require("@nomicfoundation/hardhat-ethers");
require("@nomicfoundation/hardhat-chai-matchers");

const SEPOLIA_RPC_URL = process.env.SEPOLIA_RPC_URL || "https://rpc.sepolia.org";
const PRIVATE_KEY = process.env.PRIVATE_KEY;
const LOCAL_RPC_URL = process.env.LOCAL_RPC_URL || "http://127.0.0.1:8545";

module.exports = {
  solidity: { version: "0.8.28", settings: { optimizer: { enabled: true, runs: 200 } } },
  networks: {
    hardhat:   { chainId: 31337 },
    localhost: { url: LOCAL_RPC_URL, chainId: 31337, gas: 16000000 },
    sepolia:   { url: SEPOLIA_RPC_URL, accounts: PRIVATE_KEY ? [PRIVATE_KEY] : [], chainId: 11155111
},
  },
};
```

Three observations that interviewers like. First, the optimizer is on at 200 runs: deployment bytecode is slightly larger, runtime calls are cheaper, which is the right trade for contracts called many times. Second, the `sepolia` block exists but was never used; its presence is configuration, not evidence of deployment, and the report says so. Third, the private key is read from the environment and only wired into the unused network — the local workflow never needs a key at all, because Hardhat's accounts are unlocked.

4.2 The local node, accounts and test ETH

`npx hardhat node` starts a JSON-RPC server on port 8545, mines a block per transaction instantly, and prints twenty accounts with 10,000 test ETH each. The first three are the project's roles:

Table 4.1: The accounts that matter in this project

Index	Address	Role in ProofPay
0	`0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266`	Deployer, owner of all three contracts, escrow owner, receiver `manualOracle`, bridge signer, holder of 1,000,000 minted pUSD initially.

Index	Address	Role in ProofPay
1	`0x70997970C51812dc3A010C7d01b50e0d17dc79C8`	Buyer: approves and funds escrows; receives refunds.
2	`0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC`	Seller: registers shipments; receives releases.

Because the node holds the keys in memory and exposes them as unlocked accounts, a client can send `eth_sendTransaction` and the node signs. That is why the bridge needs no private key. It is also why a restart is so destructive: the chain lives in the node's memory, so stopping it erases every block, every contract and every balance, and the next start redeploys from scratch at the same deterministic addresses with a fresh history.

> **NEVER RESTART CASUALLY**
>
> If you restart the node mid-demonstration, the contract addresses stay the same (same deployer, same nonces) only after you redeploy; until then those addresses are empty. The frontend will show zero balances, the backend's shipment records will reference orders that no longer exist, and the bridge will fail preflight with "No bytecode at escrow". Recovery is: redeploy, re-authorise, re-mint, re-register shipments. Module 12 has the full procedure.

4.3 Compilation, artifacts, ABI and bytecode

`npx hardhat compile` produces two things per contract under `artifacts/`: **bytecode**, which is the deployable machine code, and the **ABI** (application binary interface), a JSON description of every function, argument, return value, event and error. The ABI is what lets JavaScript call a contract: it tells viem or ethers how to encode your arguments into calldata and how to decode the returned bytes.

A function's **selector** is the first four bytes of `keccak256("name(type1,type2)")`. `processReport(bytes,bytes)` therefore always has the same selector everywhere in the world, which is why the 388-byte calldata in the bridge log begins with that constant and why an explorer can label a transaction it has never seen before.

Table 4.2: Four operations beginners confuse

Operation	What it costs / changes	How you do it here
Deploy a contract	One transaction, gas paid, new address created, constructor runs once.	`scripts/deploy.js` (token, escrow), `scripts/deploy-receiver.js` (receiver + JSON record).
Connect to a deployed contract	Free; no chain interaction.	`new ethers.Contract(address, abi, signerOrProvider)` or viem's `readContract({ address, abi, ... })`.
Call a view function	Free, no transaction, no receipt; executed locally by the node via `eth_call`.	`getEscrow`, `balanceOf`, `oracle()`, `manualOracle()`.
Send a state-changing transaction	Costs gas, produces a receipt, changes state permanently.	`createEscrow`, `approve`, `processReport`.

4.4 Deployment scripts and what they prove

Deployment order is forced by the constructors. The token must exist before the escrow, because the escrow's constructor checks that the token address has bytecode. The escrow must exist before the receiver, because the receiver stores it as `IEscrowSettlement`. Then two authorisation calls make the chain of authority real: `escrow.setOracle(receiver)`, and `receiver.setManualOracle(admin)` with the forwarder left at zero.

```
// The essence of scripts/authorize-receiver.js, with read-back verification.
await escrow.setOracle(receiverAddress);
const wired = await escrow.oracle();
if (wired.toLowerCase() !== receiverAddress.toLowerCase()) throw new Error("oracle wiring failed");

// scripts/deploy-receiver.js additionally writes:
// contracts/deployments/cre-receiver-localhost.json
// { network, chainId, deployer, owner, escrow, forwarder, manualOracle, receiver, deployedAt }
```

That JSON file is the project's deployment record, and it is why the report can state addresses with confidence. Its `forwarder` field is `0x000…000`: the honest marker that no DON is attached.

4.5 Testing: how 31 tests are organised, and what they actually assert

Tests are written with Mocha, Chai and `@nomicfoundation/hardhat-chai-matchers`, and use `loadFixture` from the network helpers. A fixture deploys the contracts once and snapshots the chain; every test then restores that snapshot, so tests cannot contaminate each other and the suite runs in under a second.

```
async function deployTokenFixture() {
  const [owner, alice, bob] = await ethers.getSigners();
  const token = await (await ethers.getContractFactory("ProofPayUSD")).deploy(owner.address);
  await token.waitForDeployment();
  return { token, owner, alice, bob };
}

it("allows only the owner to mint", async function () {
  const { token, owner, alice } = await loadFixture(deployTokenFixture);
  await expect(token.connect(alice).mint(alice.address, 1000n))
    .to.be.revertedWithCustomError(token, "OwnableUnauthorizedAccount")
    .withArgs(alice.address);
});
```

Table 4.3: The 31 contract tests by suite, and the failure each suite is designed to catch

Suite	Tests	What the suite pins down (selected)
ProofPayUSD	6	Exact metadata and zero initial supply; owner-only mint with the exact custom error and argument; rejection of minting to the zero address; standard approve/transferFrom semantics with allowance decrement; insufficient balance rejection; zero-owner deployment rejection.
ProofPayEscrow	16	Funding moves tokens and emits `EscrowCreated`; every validation branch reverts with its specific error; both payout paths pay the right party; duplicate order rejection; settlement of an unknown or unfunded order; oracle rotation; multiple escrows settled independently; reentrant creation blocked.
ProofPayCREReceiver	9	Only authorised callers; report decoding; replay of identical bytes rejected; a second, different report for a settled order rejected; unfunded and unknown escrows rejected; unsupported outcome rejected; multiple orders settled independently.

Two tests worth understanding in detail

"prevents duplicate settlement". The test funds an escrow, submits a valid DELIVERED report, then submits the same report again. Without the protection, the seller would be paid twice out of other users' deposits — the escrow's balance is pooled, so the second payment would silently consume someone else's money. The assertion is that the second call reverts with `ReportAlreadyProcessed` and that the seller's balance is unchanged. The reason this test matters more than most: it is the difference between a demo and a system.

"blocks reentrant escrow creation". A malicious token contract calls back into `createEscrow` during `transferFrom`. Without a guard, the attacker could create a second escrow that shares the same deposit, or corrupt the map-

ping. The `nonReentrant` modifier makes the inner call revert. If you removed the modifier, this test fails immediately, which is exactly what you want from a security test.

> **HOW TO RUN AND READ THE SUITE**
>
> `cd contracts && npx hardhat test` prints one line per test and a summary. The expected summary is **31 passing** in well under a second. If a test fails, read the diff: chai-matchers show expected versus actual error names and arguments, which is usually enough to identify the broken invariant without a debugger.

MODULE 5

FRONTEND DEVELOPMENT AND BLOCKCHAIN INTEGRATION

The frontend is a React 19 single-page application built by Vite 8, styled with Tailwind CSS 4, routed with React Router 7, and connected to the chain through MetaMask and viem. Its most important property is architectural: it holds no business truth. Everything it displays is read back from the contracts or the shipment API.

5.1 React in the vocabulary this project uses

Table 5.1: React concepts mapped to real ProofPay files

Concept	Meaning	ProofPay instance
Component	A function returning JSX; the unit of UI.	`BuyerDashboard`, `SellerDashboard`, `CheckoutPage`, `OrderTracking`, `OraclePanel`, `ShipmentSimulatorPanel`.
Props	Read-only inputs from a parent.	`CheckoutPage({ wallet, token, onCreated })`; `OraclePanel({ wallet, orders, onSettled })`.
`useState`	Local mutable state that triggers re-render.	Form fields (`seller`, `amount`), selected product, field errors, copied-state.
`useEffect`	Side effects after render; subscriptions; async reads.	`useWallet` rehydrates the wallet and subscribes to `accountsChanged`/`chainChanged`; `OraclePanel` reads `escrow.oracle()` and `manualOracle()` on mount.
Custom hook	A function that packages stateful logic for reuse.	`useWallet` (connection), `useEscrows` (discovery + reads), `useEscrowActions` (approve/create/settle transactions).
Context	Cross-tree state without prop drilling.	Not used for chain data; wallet state is passed explicitly, which keeps the data flow inspectable.
Event handler	Function bound to a DOM event.	`handleSubmit` in checkout validates the seller address, the amount, and self-payment before any wallet prompt.
Conditional rendering	Render based on state.	Wrong-network banner; "Connect wallet" versus dashboard; transaction phase labels.
List rendering	Map data to elements with keys.	Escrow rows, activity feed, shipment event timeline.

5.2 TypeScript: why the types are not decoration

The project uses TypeScript 5.7 with strict settings. In a blockchain app this matters more than usual because three different representations of the same value meet in the browser: a decimal string typed by a user ("3499"), a `bigint` in the token's smallest unit, and a hex string for calldata. Mixing them silently is how people send 3.499 tokens instead of 3,499.

```
// frontend/src/lib/contracts.ts — the boundary between display and chain units
export function parseTokenAmount(input: string, decimals: number):
  { value: bigint; error: string | null } { /* digits → bigint * 10**decimals */ }

export function formatTokenAmount(value: bigint, decimals: number): string { /* reverse */ }
```

Every amount crossing the boundary goes through these two functions. The UI never does `parseFloat` on money, and the contract never sees a string. Types also catch the common mistake of passing `number` where `bigint` is required, which would otherwise throw at runtime on large amounts.

5.3 ABI, selectors and how the browser actually calls a contract

An ABI is a JSON array describing each function's name, inputs, outputs and mutability. viem uses it in two directions: `encodeFunctionData` turns (function, args) into calldata; `decodeFunctionResult` turns returned hex into typed values. A read becomes `eth_call`; a write becomes `eth_sendTransaction` followed by a receipt wait.

Table 5.2: Read versus write, in the exact terms the code uses

Aspect	Read (`eth_call`)	Write (`eth_sendTransaction`)
Purpose	Evaluate a `view`/`pure` function without changing state.	Execute a state-changing function.
Wallet prompt	None.	One per transaction.
Cost	Free.	Gas, paid in test ETH here.
Result	Return data, decoded immediately.	Transaction hash, then a receipt with status and logs.
ProofPay example	`getEscrow`, `balanceOf`, `manualOracle`.	`approve`, `createEscrow`, `processReport`.

```
// frontend/src/lib/receiver.ts — building a processReport call by hand
export function encodeProcessReportCalldata(orderId, kind, observedAt) {
  const outcome = kind === 'delivery' ? 'DELIVERED' : 'LOST';
  const trackingIdHash = `0x${'0'.repeat(64)}`;           // manual path has no tracking hash
  const report = encodeAbiParameters(
    parseAbiParameters('bytes32 orderId, string outcome, bytes32 trackingIdHash, string
observedAt'),
    [orderId, outcome, trackingIdHash, observedAt]
  );
  return encodeFunctionData({ abi: RECEIVER_ABI, functionName: 'processReport', args: ['0x', report]
});
}
```

This function is worth studying because it mirrors the CRE payload exactly. The receiver decodes four fields; the manual panel must therefore produce four fields, with an empty metadata blob (`0x`) and a zero tracking hash. The zeroed hash is honest: the manual operator has no tracking ID, and the report ID is still unique because the timestamp and order differ. It also shows the trade-off of the manual path — no provenance — which is why automatic settlement is the project's real subject.

5.4 Discovering escrows, and why logs are not the source of truth

```
// frontend/src/store/useEscrows.ts (structure, not the whole file)
const logs = await provider.request({ method: 'eth_getLogs', params: [{
  address: escrowContractAddress(), fromBlock: '0x0', toBlock: 'latest',
  topics: [ESCROW_CREATED_TOPIC]                       // filter on the event signature
}]});
const ids = [...new Set(logs.map(l => l.topics[1]))];  // indexed orderId
for (const id of ids) {
  const escrow = await readEscrow(id);                 // getEscrow via eth_call
  // build the Order object from the struct, never from the log's data
}
```

Two subtleties. The order ID is *indexed* in the event, which is why it appears in `topics[1]` and why `eth_getLogs` can filter on it. And the log is used only to learn which IDs exist; every displayed field is re-read from storage. That protects the UI from stale or partial logs and from the classic bug of trusting an event that a reorg could remove.

5.5 Transaction state in the UI, and why two receipts matter

The checkout does not fire two transactions and hope. It models the sequence explicitly: `approving` → `approved` → `creating` → confirmed, with the buttons disabled while `busy`. The reason is that the second transaction is only valid after the first is mined; sending both immediately would race, and the escrow call could execute before the allowance exists, reverting with `ERC20InsufficientAllowance`.

```
const busy = tx.phase === 'approving' || tx.phase === 'approved' || tx.phase === 'creating';
// ... later, after each receipt:
//   approve  → phase 'approved', then send createEscrow
//   create   → phase 'created', call onCreated() so the app re-reads the chain
```

After success, the component calls `onCreated()`, which triggers the same discovery-and-read path described above. No local state is trusted as a record of the purchase; the chain is re-read, and if the read disagrees with the optimistic UI, the chain wins.

5.6 Error handling that users can act on

Three error layers exist. Wallet errors come back with codes: 4001 means the user rejected the prompt, 4902 means the chain is not configured. Contract errors arrive as revert data; viem decodes them into the custom error name, which the UI can translate ("Escrow already exists for this order"). Application errors are the ones the UI generates before touching the wallet, such as "The seller cannot be your own wallet", validated in `handleSubmit`.

> **INTERVIEW CHECKPOINT — MODULES 4 AND 5**
>
> **Q.** "Why does the UI call `getEscrow` after reading logs instead of decoding the log's data?" **A.** Logs are an index, not a record: they can be missing if you start from the wrong block and they carry only the fields the event declared. Storage is authoritative, so the app uses logs to find IDs and storage to render values.
>
> **Q.** "What happens if the user rejects the second MetaMask prompt?" **A.** The allowance transaction is already mined, so the spender is approved but no escrow exists. Nothing is lost; the UI reports the rejection and the buyer can retry the escrow call without repeating the approval.

MODULE 6

THE BACKEND AND THE SHIPMENT API

The backend is small, deliberately: one Express service, five routes, one JSON file, one validation layer. Its job is to be the off-chain system of record for physical progress, so that the oracle has something independent to read. It never touches the escrow, never holds keys, and never decides a settlement.

6.1 Node, Express and the shape of a REST API

Node.js runs JavaScript outside the browser, with a single-threaded event loop: I/O is asynchronous, so waiting for a file read does not block other requests. Express is a thin HTTP framework on top of Node: you register routes, and middleware functions run in order. This project uses Express 4.21.2 with only two extra dependencies, `cors` and `dotenv`.

```
// backend/src/app.js (structure)
const app = express();
app.use(cors({ origin: config.corsOrigin }));
app.use(express.json({ limit: '32kb' }));          // reject oversized bodies early
app.use('/api/shipments', createShipmentRouter({ shipmentService, config }));
app.use((req, res) => res.status(404).json({ error: 'NOT_FOUND' }));
app.use((error, req, res, next) => { /* ApiError → status, else 500 */ });
```

REST means resources are addressed by URLs and verbs describe the intent: `GET` reads, `POST` creates, `PATCH` partially updates. The API's verbs are not decoration; each maps to a different service method and a different validation path.

Table 6.1: The complete shipment API, as implemented

Endpoint	Handler	Behaviour and status codes
`GET /api/shipments`	`shipmentService.list(query)`	Returns `{ shipments, count, simulated }`, newest first, with optional `orderId` and `status` filters. This is the exact call the CRE workflow makes.
`GET /api/shipments/order/:orderId`	`getByOrderId`	One record by 32-byte order ID, or 404 `SHIPMENT_NOT_FOUND`. Declared before `/:trackingId` so the literal path is not swallowed by the parameterised one.
`GET /api/shipments/:trackingId`	`getByTrackingId`	One record by tracking ID, or 404.
`POST /api/shipments`	`register(body)`	Protected by the mutation secret. 201 with a `Location` header on success; 400 validation, 404 `ESCROW_NOT_FOUND`, 409 `ESCROW_NOT_FUNDED` / `SELLER_MISMATCH` / `DUPLICATE_TRACKING_ID` / `SHIPMENT_ALREADY_EXISTS`.
`PATCH /api/shipments/:trackingId/status`	`updateStatus`	Protected by the mutation secret. 200 with the updated record; 409 `INVALID_STATUS_TRANSITION` with `{ from, to, allowed }`; 404 if the tracking ID is unknown.

Every response carries `X-ProofPay-Data-Source: mock-courier-simulator` and `Cache-Control: no-store`. That is an honesty mechanism visible in the network tab: the data you are reading is simulated, and it must not be cached into a stale screenshot.

6.2 Middleware, validation and the five-layer write path

Middleware is a function that runs before the handler. `requireMutationAccess` is the security middleware: it compares a shared secret header against configuration and returns 401 when it is absent or wrong. It exists because the API is intentionally reachable on localhost by the browser, but only the seller client and the scheduler should be able to change a status.

Validation lives in `lib/validation.js`, not in the route. `normalizeCreateShipment` checks the order ID is 32 bytes of hex, the tracking ID is non-empty and conforms to the expected pattern, the carrier is a string, and the seller address is a valid 20-byte hex address; it lowercases addresses so comparisons with chain data are reliable. `normalizeStatusUpdate` rejects unknown statuses. Rejecting bad input at the edge means every later layer can assume clean data — but each later layer still validates, because a service can be called from somewhere else tomorrow.

6.3 Registering a shipment: the real sequence

1. Seller opens Incoming Escrows, selects a funded order, and enters a tracking ID such as `PPD-MACBOOK-001`.
2. The React app sends `POST /api/shipments` with the order ID, tracking ID, carrier label, its own address and a description, plus the mutation secret header.
3. `register()` normalises the body, then calls `assertNoDuplicates`: no existing shipment with that tracking ID, none for that order.
4. It then calls `chainVerifier.verifyEscrow(orderId)`, a read-only RPC call to `getEscrow`.
5. Three chain-level facts are enforced: the escrow exists (404 if not), its status is 1 = `Funded` (409 if released/refunded), and its seller equals the registered seller address (409 `SELLER_MISMATCH`).
6. The duplicate checks run again — after the async RPC — to close the window in which two concurrent requests could both pass.
7. Finally the record is built with `status: "CREATED"`, `simulated: true`, timestamps from an injectable clock, and the first event in its history.

Step 5 is the answer to "what stops a seller from registering a shipment for an order that does not exist?". The backend refuses, and even if it did not, the CRE workflow independently checks the escrow's status before reporting, and the receiver checks again before paying. Three layers, same fact.

6.4 State transitions, and why the API is strict

```
// backend/src/services/shipmentService.js
export const SHIPMENT_TRANSITIONS = Object.freeze({
  CREATED:    Object.freeze(['SHIPPED']),
  SHIPPED:    Object.freeze(['IN_TRANSIT']),
  IN_TRANSIT: Object.freeze(['DELIVERED', 'LOST']),
  DELIVERED:  Object.freeze(['RETURNED']),   // simulation-only return path
  LOST:       Object.freeze([]),
  RETURNED:   Object.freeze([]),
});
```

Table 6.2: The shipment lifecycle as enforced by the API

Status	Allowed next	Meaning for the oracle
`CREATED`	`SHIPPED`	Registered, not dispatched. CRE reports nothing; the escrow stays funded.
`SHIPPED`	`IN_TRANSIT`	In the network. Still no settlement.
`IN_TRANSIT`	`DELIVERED` or `LOST`	The decision point. CRE now produces a report, and the escrow is still funded, so it is eligible.

Status	Allowed next	Meaning for the oracle
`DELIVERED`	`RETURNED` (simulation only)	CRE maps DELIVERED to RELEASE. If the escrow is already released, a later RETURNED produces no automatic action — the workflow routes it to manual review.
`LOST`	none	CRE maps LOST to REFUND; the escrow becomes Refunded and terminal.
`RETURNED`	none	Manual review only. No automatic report is generated, because a return after delivery is a dispute, not a courier fact.

The transition table is validated inside the store's serialised updater. That detail matters: if the check happened before the async write, two concurrent PATCH requests could both read `IN_TRANSIT` and both apply `DELIVERED` and `LOST`, producing two contradictory events. Serialising the check and the write makes the transition atomic at the application level.

6.5 Persistence: one file, written carefully

`shipmentStore.js` keeps the collection in memory and mirrors it to `backend/data/shipments.json`. Writes go to a temporary file and are then renamed over the target, so a crash mid-write cannot leave a truncated JSON document. Writes are serialised through a promise chain, so concurrent requests queue rather than interleave. Reads are served from memory, which is why the CRE HTTP call is cheap.

This is not production storage, and the guide says so plainly: no indexing, no transactions across entities, no replication, no access control at rest. For a teaching project whose subject is the oracle pipeline, a single atomic file is the right trade — it makes the data source inspectable with `cat`, which is exactly what a viva needs.

6.6 Why seller registration is separate from escrow creation

They are different facts recorded by different parties on different systems. The escrow records a financial obligation: this buyer has locked this amount for this seller. The shipment record describes physical fulfilment: this parcel with this tracking number is moving. If registration were part of `createEscrow`, the buyer would have to know a tracking number that does not exist yet, and the escrow contract would have to store a string, paying permanent storage costs for data that changes six times.

Separating them also creates the cross-check that makes the oracle meaningful: the backend verifies that the order is funded and that the seller matches before accepting a shipment. Neither system can lie alone; the chain supplies the financial fact, the API supplies the physical fact, and CRE is the component that joins them.

> **INTERVIEW CHECKPOINT — MODULE 6**
>
> **Q.** "Why not put shipment status on chain?" **A.** Cost, mutability and privacy. Six status updates per order would cost gas each time, and a mis-scan would be permanent. Keeping it off chain means it can be corrected, while the money stays on chain where corrections are unnecessary.
>
> **Q.** "Who can fake a shipment?" **A.** Anyone holding the API's mutation secret, which is server-side only. But a fake status cannot move money unless the escrow is funded and unspent, and the settlement itself is limited to one payout per order by the receiver and the escrow.

MODULE 7

CHAINLINK, ORACLES AND CRE FROM ZERO

This is the module that turns you from someone who used Chainlink into someone who can explain why it is necessary. Read it slowly.

7.1 The oracle problem, stated simply and then precisely

A smart contract is a sealed room: it can compute, it can read its own storage, and it can call other contracts, but it cannot look outside the blockchain. It has no network access, no clock beyond the block timestamp, and no way to ask "did the parcel arrive?". This is not an oversight; it is what makes consensus possible. Every node must compute the same answer, so every input must already be on chain.

Therefore any fact about the real world has to be carried in by a transaction. The carrier is called an **oracle**. The **oracle problem** is: how do you get a trustworthy external fact into a system that cannot verify it, without reintroducing a single point of trust?

Table 7.1: Off-chain facts versus on-chain facts in ProofPay

Fact	Where it lives	Why there
Buyer locked 3,499 pUSD for seller S	On chain (escrow storage)	It is the money. Must be tamper-proof and verifiable by both parties.
Shipment PPD-MACBOOK-001 is DELIVERED	Off chain (Express API)	It changes often, comes from an external process, and must be correctable.
Report says DELIVERED at 12:55:31Z	On chain (via receiver)	It is the evidence that justified moving money; it must be permanent.
Courier's internal scan history	Off chain	Voluminous, private, and irrelevant to the contract beyond the status.

7.2 What Chainlink is, and which parts this project actually uses

Chainlink is a network of independent node operators that deliver external data and computation to blockchains. A **node** runs software that can fetch APIs, sign results and submit transactions. A **DON** (decentralised oracle network) is a group of nodes that must agree on a result before it is reported, which removes the single-reporter assumption. **Consensus** in that context means the nodes independently fetch and compare, and only an agreed answer is accepted.

Table 7.2: Chainlink products, and whether ProofPay uses them

Product	What it does	Status in this project
Price Feeds	Publishes market prices (ETH/USD, etc.) on chain.	**Not used.** ProofPay has no price dependency; amounts are denominated in pUSD.
Automation (Keepers)	Runs a contract function when a condition is met.	**Not used.** The scheduling here is a local cron inside the CRE workflow and the demo scheduler script.
Functions	Serverless JavaScript that returns a value on chain.	**Not used.** Its model is a request/response callback; ProofPay needs a multi-step workflow with its own decision logic.
VRF / CCIP	Verifiable randomness / cross-chain messaging.	**Not used.**

Product	What it does	Status in this project
CRE (Compute Runtime Environment)	Workflows written in TypeScript that fetch HTTP, read EVM state, compute, and produce signed reports for a receiver contract.	**Used, and executed locally through the official CLI simulator.** The on-chain write path is disabled by `dryRun: true` and no DON is deployed.

Why CRE rather than a price feed? Because the task is not "read a number someone else publishes"; it is "fetch my own API, check my own contract, apply my own rule, and produce a report my contract can verify". CRE is the product that supports arbitrary workflow logic with both HTTP and EVM capabilities, which is exactly the shape of the problem.

7.3 The CRE concepts you must be able to define

Table 7.3: CRE vocabulary, anchored in main.ts

Term	Meaning	Where it appears
Workflow	A TypeScript program with a trigger and a handler; deployed to a DON or run in the simulator.	`shipment-workflow/main.ts`, registered by `initWorkflow()`.
Trigger	What starts a run. Here, a cron schedule.	`new CronCapability()` with `"0 */5 * * * *"` (every five minutes).
HTTP capability	The workflow's way to call the internet, with optional consensus across nodes.	`new HTTPClient()` + `sendRequest` wrapped in `consensusIdenticalAggregation<string>()`.
EVM capability	Reads (and, when enabled, writes) to a chain selected by a CRE chain selector.	`new EVMClient(network.chainSelector.selector);callContract(...)`.
Runtime	The object giving config, logging and `runtime.report()`.	`Runtime<Config>`, used for `runtime.log` and `runtime.config`.
Report	The encoded payload plus a runtime metadata header, produced by the SDK and verifiable on chain.	`runtime.report(prepareReportRequest(payload)).result()`.
`writeReport`	The call that would submit the report to a receiver on a supported chain.	Present in the code but unreachable while `dryRun` is true.
Simulation	Running the workflow locally with the CLI, one node, real code.	`cre workflow simulate shipment-workflow --target staging-settings --trigger-index 0 --non-interactive`.
Deployment	Registering the workflow with a DON so it runs on a schedule with consensus.	**Not performed in this project.**

7.4 main.ts, stage by stage, with the real code

The workflow is one handler. Read the following in the order it executes; each block is quoted from `chainlink/cre/shipment-workflow/main.ts`.

Stage 1 — configuration and the decision table

```
export type Config = {
  shipmentApiBaseUrl: string; shipmentApiPath: string; rpcUrl: string; chainId: number;
  escrowContractAddress: string; receiverContractAddress: string;
  forwarderAddress: string; dryRun: boolean;
};
const ESCROW_STATUS_FUNDED = 1;
```

Configuration is supplied per target by `config.staging.json`, validated by zod in `config.ts`. The staging file sets the API base to `http://localhost:5001`, the chain ID to 31337, the escrow and receiver addresses to the deployed ones, the forwarder to the zero address, and `dryRun: true`. `config.production.json` exists and was left untouched.

```
function shouldSettle(shipment: { status: string }): SettlementDecision {
  switch (shipment.status) {
    case "DELIVERED": return { action: "RELEASE", outcome: "DELIVERED", reason: "..." };
    case "LOST":      return { action: "REFUND",  outcome: "LOST",      reason: "..." };
    case "IN_TRANSIT":
    case "SHIPPED":   return { action: "NONE",    reason: "payment remains locked" };
    case "RETURNED":  return { action: "MANUAL_REVIEW", reason: "requires manual review" };
    case "CREATED":   return { action: "NONE",    reason: "not ready for settlement" };
    default:          return { action: "NONE",    reason: `Unknown shipment status:
${shipment.status}` };
  }
}
```

This function is the project's entire settlement policy, and it lives in exactly one place. The bridge has no copy of it. If an interviewer asks "where is the decision made?", point here.

Stage 2 — fetch the shipment list over HTTP, with consensus aggregation

```
const httpClient = new HTTPClient();
const responseBody = httpClient
  .sendRequest(runtime, fetchShipments, consensusIdenticalAggregation<string>())(runtime.config)
  .result();

const fetchShipments = (sendRequester: HTTPSendRequester, config: Config): string => {
  const url = `${config.shipmentApiBaseUrl}${config.shipmentApiPath}`;
  const response = sendRequester.sendRequest({ url, method: "GET" }).result();
  if (!ok(response)) throw new Error(`HTTP request failed with status: ${response.statusCode}`);
  return text(response);
};
```

Three things to notice. The request is wrapped in `consensusIdenticalAggregation<string>()`, which in a DON deployment means every node fetches and the results must be byte-identical before the workflow continues — that is how a single lying node is outvoted. In the local simulator there is one node, so the consensus wrapper is exercised but proves nothing; say that honestly. Second, failures throw, and the handler catches them and returns an error string, so a dead API results in "HTTP error" and zero settlements rather than a partial run. Third, the body is treated as untrusted text and parsed defensively:

```
let shipments: any[] = [];
if (Array.isArray(parsed)) shipments = parsed;
else if (parsed && typeof parsed === "object" && "shipments" in parsed &&
Array.isArray(parsed.shipments))
  shipments = parsed.shipments;
else return "No shipments";
```

Stage 3 — read the escrow on chain before believing the shipment

```
const network = getNetwork({ chainFamily: "evm", chainSelectorName: "ethereum-testnet-sepolia",
isTestnet: true });
const evmClient = new EVMClient(network.chainSelector.selector);

const callData = encodeFunctionData({ abi: ESCROW_ABI, functionName: "getEscrow", args:
[orderIdBytes32] });
const callMsg = encodeCallMsg({ from: zeroAddress, to: config.escrowContractAddress, data: callData
});
const evmResp = evmClient.callContract(runtime, { call: callMsg, blockNumber:
LAST_FINALIZED_BLOCK_NUMBER }).result();
const escrow = decodeFunctionResult({ abi: ESCROW_ABI, functionName: "getEscrow", data:
bytesToHex(evmResp.data) });
if (escrow.status !== ESCROW_STATUS_FUNDED) { runtime.log("Escrow not funded..."); continue; }
```

This is the cross-check that makes the design sound: the API claims a shipment status, the chain proves whether money is still held. Both must agree before a report exists. `LAST_FINALIZED_BLOCK_NUMBER` is used rather than "latest" so the read is from a settled block, not a tip that could be reorganised.

> **THE CHAIN-SELECTOR LIMITATION, STATED EXACTLY**
>
> `getNetwork` is called with `ethereum-testnet-sepolia` because the installed SDK (1.23.0) exposes no chain selector for a local Hardhat node on chain ID 31337. The EVM read therefore does not point at Hardhat through that abstraction. This is precisely why the local execution bridge exists, and why this project must never be described as having executed a CRE write against Hardhat. CRE's read is performed against the configured RPC in the local workflow path used here, and the write path is disabled entirely.

Stage 4 — build and generate the report

```
const observedAt = shipment.updatedAt || shipment.createdAt || "2026-11-01T00:00:00.000Z";
const trackingIdHash = keccak256(toHex(shipment.trackingId));
const reportPayload = createReportPayload(shipment.orderId, decision.outcome, shipment.trackingId,
observedAt);
const reportRequest = prepareReportRequest(reportPayload);
const report = runtime.report(reportRequest).result();
```

`createReportPayload` does the ABI encoding:

```
const orderIdBytes32 = (orderIdHex.padEnd(66, "0").slice(0, 66)) as Hex;
const trackingIdHash = keccak256(toHex(trackingId));
return encodeAbiParameters(parseAbiParameters("bytes32, string, bytes32, string"),
  [orderIdBytes32, outcome, trackingIdHash, observedAt]);
```

Table 7.4: The four report fields, and why each exists

Field	Type	Why it is in the report
orderId	`bytes32`	Identifies which escrow to pay. The receiver decodes it and passes it to the escrow.
outcome	`string`	"DELIVERED" or "LOST". The receiver accepts only these two and maps them to the two settlement calls.
trackingIdHash	`bytes32`	Evidence of which tracking record produced the decision, without publishing the tracking number. Part of the report ID.
observedAt	`string`	When the courier record was last updated. Distinguishes two reports about the same order and makes the audit trail time-specific.

Stage 5 — extract the exact bytes, and prove they are the right bytes

The SDK prefixes a 109-byte metadata header to the encoded body. The receiver decodes the body only, so sending the raw report would corrupt the first field. The workflow therefore tries both candidates and accepts the one that decodes to the expected order and outcome:

```
const creRawReportHex = bytesToHex(report.rawReport());
const bodyBytes = report.body();
if (bodyBytes && isExpectedAbiPayload(bodyBytes, orderIdBytes32, outcome))
  return { payloadHex: bytesToHex(bodyBytes), payloadSource: "body", creRawReportHex };
if (isExpectedAbiPayload(report.rawReport(), orderIdBytes32, outcome))
  return { payloadHex: creRawReportHex, payloadSource: "rawReport", creRawReportHex };
return { payloadHex: creRawReportHex, payloadSource: "unverified", creRawReportHex };
```

In every observed run the source was `body`, the payload was 256 bytes, the raw report 365 bytes, and 365 − 256 = 109 exactly matches `REPORT_METADATA_HEADER_LENGTH`. That arithmetic is the cleanest evidence that the header was stripped correctly, and it is the kind of detail that makes an interviewer believe you built the thing.

Stage 6 — emit the structured records, then honour dry-run

```
runtime.log(`${REPORT_LOG_PREFIX}${JSON.stringify({
  type: "report", orderId, outcome: decision.outcome, trackingId: shipment.trackingId,
  observedAt, metadata: "0x", report: payloadHex })}`);

runtime.log(`${REPORT_LOG_PREFIX}${JSON.stringify({
  type: "meta", orderId, action: decision.action, trackingIdHash, reportSource: payloadSource,
  reportDecodesToReceiverAbi: payloadSource !== "unverified", creRawReportLength,
  creMetadataHeaderLength: REPORT_METADATA_HEADER_LENGTH, creRawReportHash,
  receiver: config.receiverContractAddress, escrow: config.escrowContractAddress,
  creDryRun: config.dryRun })}`);

if (config.dryRun) { runtime.log("DRY RUN: ... CRE write skipped"); continue; }
```

The sequencing is the single most important repair in the project's history. In the original code the `dryRun` check came *before* `runtime.report`, so with dry-run enabled — the safe setting — no report object was ever produced and nothing could be captured. Moving generation above the branch is what made the entire local pipeline possible: the report exists in both modes, and only its on-chain destination is conditional.

The two log lines exist because the CRE runtime truncates user log lines at roughly one kilobyte. The payload alone is 512 hex characters, so provenance is emitted separately and the bridge rejoins the two records by order ID. That constraint, and the workaround, is a good story about reading your tools' limits instead of fighting them.

> **INTERVIEW CHECKPOINT — MODULE 7**
>
> **Q.** "Why hash the tracking ID?" **A.** Two reasons: the hash ties the settlement to a specific courier record as evidence, and it avoids publishing the tracking number on a public ledger, which would leak shipping details. The receiver stores it only as part of the report ID.
>
> **Q.** "Does consensus aggregation prove your oracle is decentralised?" **A.** No. The code is written for it, but in this deployment the simulator runs one node, so no independent agreement happens. Real decentralisation requires the workflow deployed to a DON with multiple operators, which was out of scope.

MODULE 8

THE LOCAL EXECUTION BRIDGE

This is the component that makes the local demonstration possible, and the one most likely to be misunderstood in an interview. Learn its boundaries before its code: the bridge carries a decision, it never makes one.

8.1 Why a bridge exists at all

A CRE workflow's normal on-chain write path is `evmClient.writeReport(...)`, which targets a chain selected by a CRE chain selector. The installed SDK (1.23.0) has no selector for a local Hardhat node on chain ID 31337. Two consequences follow: the workflow cannot deliver a report to the local escrow by itself, and no DON was available to deploy to anyway.

So the workflow runs in the official CLI simulator with `dryRun: true`, producing a real report but no transaction. Something must carry that report the last metre. That something is `local-runner.ts`, a Node/Bun script that runs the simulator, parses its structured output, validates everything it can, and submits the exact bytes through `processReport`.

> **WHAT THIS ARCHITECTURE IS, AND IS NOT**
>
> **It is:** genuine CRE workflow execution (real HTTP fetch, real EVM read, real decision, real ABI encoding) plus a local relay.
>
> **It is not:** a Chainlink DON, decentralised consensus, or DON-signed delivery. One machine runs the simulation and one unlocked local account submits. Anyone who says this is "oracle decentralisation" is wrong, and the receiver's checks — not the bridge — are what protect the funds.

8.2 The seventeen steps, and where each lives in the code

Table 8.1: The bridge's execution path

#	Step	Implementation detail
1	Load configuration	`loadConfig()` reads RPC URL, expected chain ID, receiver, escrow, admin, allowlist, dry-run flag, timeout, state file, CRE command.
2	Acquire a single-instance lock	`acquireLock(LOCK_FILE)` with `open(file, "wx")`; a stale lock from a dead pid is detected with `isProcessAlive` and removed.
3	Preflight the chain	Chain ID, bytecode at receiver and escrow, unlocked admin, `receiver.manualOracle == admin`, `escrow.oracle == receiver`, admin ETH balance.
4	Start the CRE simulation	`spawn("cre", ["workflow","simulate","shipment-workflow","--target","staging-settings","--trigger-index","0","--non-interactive"])` with a timeout.
5	Capture stdout/stderr	Line-by-line consumption; ANSI escapes stripped with `ANSI_PATTERN`.
6	Identify structured records	Only lines starting with `PROOFPAY_REPORT_V1:` are parsed; all other output is passed through as human log.
7	Join `report` and `meta` records	Matched by order ID. A report without its meta record is rejected rather than guessed at.
8	Extract bytes	`line.report` is already the receiver-compatible payload; the bridge does not decode or strip anything.

#	Step	Implementation detail
9	Validate fields	Order ID matches `/^0x[0-9a-fA-F]{64}$/`; outcome is DELIVERED or LOST; report and metadata are hex.
10	Cross-check provenance	`meta.receiver` and `meta.escrow` must equal configuration; `meta.reportDecodesToReceiverAbi` must not be false; `meta.creDryRun` must not be false.
11	Decode independently	`decodeAbiParameters` on the bytes; the four decoded fields must match the line's own claims.
12	Recompute the report ID	`keccak256(encodePacked([...]))` mirroring the receiver exactly, so both agree on the replay key.
13	Apply the allowlist	If `BRIDGE_ORDER_ALLOWLIST` is set, only those order IDs may proceed; others are logged as "NOT IN ALLOWLIST".
14	Check on-chain state	`getEscrow(orderId)`: must exist, must be `Funded`; seller and amount are read for the log.
15	Check duplicates	State file's report IDs and `processedReports` must not contain this report ID; `orderSettled` must be false.
16	Prepare and submit	`encodeFunctionData(processReport(metadata, report))`, then `walletClient.sendTransaction({ to, data })` — sent as an unlocked account, no key.
17	Verify and record	Wait for the receipt, require `status === "success"`, write `{orderId,outcome,reportId,txHash,blockNumber,from,submittedAt}` to `.local-bridge-state.json`.

8.3 Preflight: the part that saves you from yourself

```
const chainId = await withRetry("chain id", () => publicClient.getChainId());
if (chainId !== config.expectedChainId) failures.push(`Chain ID ${chainId} is not
${config.expectedChainId}.`);

const receiverCode = await publicClient.getCode({ address: config.receiver });
if (!receiverCode || receiverCode === "0x") failures.push(`No bytecode at receiver
${config.receiver}.`);

const unlocked = (await walletClient.getAddresses()).map(getAddress);
if (!unlocked.some(a => a.toLowerCase() === config.admin.toLowerCase()))
  failures.push(`Admin ${config.admin} is not an unlocked RPC account.`);

const manualOracle = await publicClient.readContract({ address: config.receiver, abi: RECEIVER_ABI,
functionName: "manualOracle" });
if (getAddress(manualOracle).toLowerCase() !== config.admin.toLowerCase())
  failures.push(`receiver.manualOracle() is ${manualOracle}, but the bridge signer is
${config.admin}.`);

const oracle = await publicClient.readContract({ address: config.escrow, abi: ESCROW_ABI,
functionName: "oracle" });
if (getAddress(oracle).toLowerCase() !== config.receiver.toLowerCase())
  failures.push(`escrow.oracle() is ${oracle}, expected the receiver.`);
```

Every one of those six checks corresponds to a real failure mode: wrong network, wrong addresses after a redeploy, an account MetaMask switched, a receiver whose manual oracle was rotated, an escrow still pointing at the admin EOA, and an empty gas balance. The observed output prints each line with "ok" or "MISMATCH", so a failing pre-flight tells you which fact is wrong without any further debugging. In the dry-run shown in the report, the block read `chainId 31337 ok`, `receiver code 3583 bytes ok`, `escrow code 3274 bytes ok`, `admin unlocked ok (rpc accounts: 20)`, `receiver.oracle … ok (== signer)`, `escrow.oracle … ok (== receiver)`.

8.4 Validation: the bridge repeats the contract's checks, in TypeScript

```
if (!ORDER_ID_PATTERN.test(line.orderId)) problems.push("orderId is not a bytes32 hex string");
if (!SUPPORTED_OUTCOMES.includes(line.outcome))
  problems.push(`outcome "${line.outcome}" is not one of ${SUPPORTED_OUTCOMES.join(", ")}`);
if (meta.reportDecodesToReceiverAbi === false)
  problems.push("CRE reported that its bytes are not the receiver ABI payload");
if (meta.creDryRun === false)
  problems.push("CRE dryRun is false: CRE itself may submit reports. Refusing to bridge to avoid a
duplicate settlement.");

const decoded = decodeAbiParameters(SETTLEMENT_REPORT_ABI, line.report);
if (decoded[0].toLowerCase() !== line.orderId.toLowerCase())
  problems.push("bytes decode to a different orderId than the line advertises");
if (decoded[1] !== line.outcome) problems.push("bytes decode to a different outcome");
if (decoded[2] === undefined || decoded[3] !== line.observedAt) problems.push("bytes decode to a
different observedAt");
```

The `creDryRun === false` check is subtle and worth memorising: if someone flips CRE's own dry-run off, both CRE and the bridge could deliver the same report, and the second delivery would revert on replay. Rather than rely on that revert, the bridge refuses to run at all — a fail-closed design. The independent decode is the other notable choice: the bridge does not trust CRE's summary of what it produced; it decodes the bytes itself and requires them to agree with the advertised fields.

Report-ID recomputation mirrors the Solidity exactly:

```
const reportId = keccak256(encodePacked(
  ["bytes32", "string", "bytes32", "string", "bytes"],
  [line.orderId, line.outcome, decodedTrackingIdHash, decodedObservedAt, line.metadata]
));
```

Compare this with the receiver's `keccak256(abi.encodePacked(orderId, outcome, trackingIdHash, observedAt, metadata))`. Same fields, same order, same packing — so the two independently compute the same 32 bytes. If they diverged, the bridge's duplicate check would be useless and the contract's replay protection would still hold, but the logs would lie. Matching them is a correctness requirement, not a nicety.

8.5 Submission without a private key

```
const walletClient = createWalletClient({ account: config.admin, transport });
// Passing the account as an address makes viem use eth_sendTransaction,
// i.e. the Hardhat node's unlocked account. No key material is involved.
const hash = await walletClient.sendTransaction({ to: config.receiver, data: calldata });
const receipt = await publicClient.waitForTransactionReceipt({ hash });
```

This is the security highlight of the whole component: there is no `CRE_ADMIN_PRIVATE_KEY` anywhere, because the Hardhat node already holds the key and exposes the account as unlocked. A private key in an environment file would have been the most dangerous artefact in the repository; not having one removes a whole class of leak.

8.6 Idempotency, retries and state

The state file is a JSON document listing successful submissions. On startup the bridge loads it; before submitting it checks the report ID against it, and also against the chain's `processedReports` and `orderSettled`. On success it appends a record. On failure it does **not** retry: a reverted submission is reported, the cause is decoded from the revert data, and the process exits non-zero. Blind retries after an ambiguous failure are exactly how duplicate settlements happen, so the bridge refuses to guess.

Table 8.2: Responsibilities, split cleanly between CRE and the bridge

Concern	CRE workflow	Local bridge
Fetch shipment data	Yes — HTTP capability	Never
Read escrow state	Yes — EVM capability	Yes, but only as a preflight/duplicate check
Decide RELEASE / REFUND	Yes — `shouldSettle`	Never; it only accepts DELIVERED or LOST
Encode the report	Yes — ABI encoding + `runtime.report`	Never; it relays bytes unchanged
Trim SDK metadata header	Yes — `extractAbiPayload`	No; it trusts only the reported `report` field
Authorise the report on chain	No	No — the receiver does
Prevent duplicates	No	Yes, in addition to the receiver's own mappings
Sign / submit the transaction	No (`dryRun`)	Yes, via an unlocked account

8.7 What would change for a real DON deployment

1. The workflow would be deployed to a DON with `dryRun: false`, and the shipment API would have to be publicly reachable over HTTPS with authentication, not `localhost`.
2. The receiver's `forwarder` would be set to the DON's forwarder address, and the owner fallback branch in `onlyAuthorized` would be removed.
3. The bridge would be deleted. `writeReport` would deliver the report; the receiver's replay and duplicate checks would remain the safety net.
4. `manualOracle` would stay as a documented break-glass path with a multisig, or be removed.
5. The EVM read would target a supported chain selector, and the escrow would be deployed on that same chain.

> **INTERVIEW CHECKPOINT — MODULE 8**
>
> **Q.** "Isn't the bridge just re-implementing the oracle?" **A.** No: an oracle decides what the outside world says. The bridge never looks at a shipment; it reads CRE's output, refuses anything malformed, and submits. Remove the bridge and the decision still exists in CRE; remove CRE and the bridge has nothing to submit.
>
> **Q.** "What stops the bridge from submitting a report twice?" **A.** Three things: its own state file, the receiver's `processedReports`, and its refusal to retry after any failure. The second is the only one that matters if the bridge's disk is wiped.

MODULE 9

THE AUTOMATIC COURIER SCHEDULER

`courier-scheduler.ts` is the component that makes the demonstration self-driving. Its header comment states its limits better than any paraphrase: it pushes shipments through a timeline, then runs the existing CRE simulation and the existing bridge. It never decides a settlement, never touches the contracts, and never submits a transaction itself.

9.1 Configuration and timeline

Table 9.1: Scheduler environment variables and defaults

Variable	Default	Effect
`COURIER_API_BASE`	`http://localhost:5001`	Which API it patches.
`COURIER_MANAGE_PREFIX`	`PPD-`	Only shipments whose tracking ID starts with this are advanced, so legacy records are never touched.
`COURIER_SHIPPED_AFTER_MS`	10,000	CREATED → SHIPPED delay.
`COURIER_IN_TRANSIT_AFTER_MS`	10,000	SHIPPED → IN_TRANSIT delay.
`COURIER_FINAL_AFTER_MS`	20,000	IN_TRANSIT → DELIVERED or LOST delay.
`COURIER_POLL_MS`	3,000	Loop interval.
`COURIER_RUN_BRIDGE`	true	Whether to run CRE + bridge after an update.
`COURIER_RUN_SECONDS`	0 (forever)	Bounded demo runs.
`COURIER_ONCE`	false	Single tick, useful in tests.

9.2 Status progression and the LOST convention

```
case "CREATED":    return "SHIPPED";
case "SHIPPED":    return "IN_TRANSIT";
case "IN_TRANSIT": return trackingId.toUpperCase().endsWith("-LOST") ? "LOST" : "DELIVERED";
```

The outcome of a shipment is decided by its tracking ID suffix: `PPD-MACBOOK-001` ends in `DELIVERED`; `PPD-DEMO-LOST` ends in `LOST`. This is a courier fact generator, not a settlement rule, and the file says so. CRE still reads the resulting status and decides what it means.

9.3 One tick, in order

```
const due = shipments.filter(s => isDue(s, config));         // prefix + elapsed delay
if (due.length === 0) { logIdleOnce(`waiting:${waiting}`, ...); return 0; }

let updates = 0;
for (const shipment of due) {
  const target = nextStatus(shipment.status, shipment.trackingId);
  if (!target) continue;
  const updated = await patchStatus(config, shipment.trackingId, target);   // authenticated PATCH
  log(`[courier] ${shipment.trackingId}: ${shipment.status} -> ${updated.status}`);
  updates++;
}
if (updates > 0 && config.runBridge) {
  log(`[courier] ${updates} update(s); starting CRE workflow + local bridge`);
  await runBridge(config);
}
```

Three properties are worth naming. Updates are applied one at a time and awaited, so two shipments reaching a terminal state together cannot start two overlapping CRE executions. The bridge is started only after at least one successful update, so idle ticks cost nothing. And the bridge inherits `LOCAL_BRIDGE_DRY_RUN`, which defaults to true, so the scheduler cannot accidentally send money unless someone explicitly opted in.

A lock file (`.courier-scheduler.lock`) prevents two scheduler instances, using the same create-with-`wx` technique as the bridge and the same stale-pid recovery. The scheduler also logs its mode at startup: "DRY RUN - no transaction will be sent" versus "LIVE", plus the reminder that CRE's dry-run stays true and the bridge decides nothing.

9.4 The MacBook demonstration, step by step

1. Buyer funds an escrow of 3,499 pUSD for the seller. The escrow exists and is `Funded`.
2. Seller registers tracking ID `PPD-MACBOOK-001` against that order. The API validates the escrow is funded and the seller matches, then stores status `CREATED`.
3. Scheduler tick at t+10s: patches status to `SHIPPED`. An event is appended. The scheduler then starts CRE + bridge; CRE reports nothing yet because SHIPPED maps to NONE, and the bridge has no report to parse.
4. Tick at t+20s: `IN_TRANSIT`. Same outcome: no report, escrow untouched.
5. Tick at t+40s: `DELIVERED`, because the tracking ID does not end in `-LOST`. Now CRE's `shouldSettle` returns RELEASE, the escrow is still funded, and a 256-byte report is emitted.
6. The scheduler's bridge run parses that report, validates it, and — if dry-run is off — submits `processReport`. The receiver checks authorisation, replay, duplicate and funding, calls `confirmDelivery`, and the seller receives 3,499 pUSD.

> **WHAT WAS ACTUALLY DEMONSTRATED**
>
> The three settlements executed on chain were 100 pUSD orders: `0x4d07…aaf3` (release, block 24), `0x0c34…1152` (refund, block 29) and `0xf694…c26434` (release, block 30). A 3,499 pUSD escrow was observed funded at block 32 but was not settled during the recorded work. So the MacBook narrative above describes the mechanism exactly as implemented, while the measured evidence comes from the 100 pUSD orders. Distinguishing the two is precisely the honesty an examiner rewards.

9.5 What changes for a LOST shipment

Only the final status. A tracking ID ending in `-LOST` transitions `IN_TRANSIT → LOST`, and from there everything is symmetric: CRE maps LOST to REFUND, the report carries outcome LOST, the receiver calls `confirmFailure`, and the escrow pays the *buyer* instead of the seller. The observed refund at block 29 did exactly this for 100 pUSD, with the buyer's balance rising from 4,007 to 4,107 and the seller's unchanged.

MODULE 10

THE COMPLETE WALKTHROUGH: 3,499 pUSD FOR A MACBOOK

This module is the story you will tell in an interview. Each stage is given twice: first in plain English, as you would explain it to a non-technical friend, then technically, as you would defend it to an engineer.

Stage 1 — Purchase

Plain. The buyer chooses a MacBook, connects their wallet, and pays into a locked box that neither they nor the seller can open. The seller can see that the money exists but cannot touch it.

Technical. React generates a random 32-byte `orderId`, validates the seller address and the amount, then sends `approve(escrow, 3499e18)`. After that receipt, it sends `createEscrow(orderId, seller, 3499e18)`. The escrow validates six conditions, writes a `Funded` struct, pulls the tokens with `safeTransferFrom`, and emits `EscrowCreated`. Two receipts, two confirmations, 156k and 46k gas respectively. The buyer's balance drops by exactly 3,499 pUSD; the escrow's balance rises by the same amount.

Stage 2 — Seller registration

Plain. The seller says "I have shipped it, here is the tracking number". This goes into the courier's system, not the blockchain, because it will change several times.

Technical. `POST /api/shipments` with the mutation secret. The service normalises the input, rejects duplicates by tracking ID and by order, calls `getEscrow` over RPC, and requires `status === 1` and `escrow.seller === sellerAddress`. It stores the record with status `CREATED` and the first event, and re-runs the duplicate check after the async call to close the race.

Stage 3 — Courier progress

Plain. Over the next minute the parcel moves: dispatched, in transit, delivered. Each step is recorded with a timestamp.

Technical. The scheduler polls `GET /api/shipments`, filters to `PPD-` records with elapsed delays, and sends authenticated `PATCH` requests. The service validates against `SHIPMENT_TRANSITIONS` inside the serialised store update and appends an event per change. A manual alternative exists in the admin dashboard's simulator; both paths hit the same endpoint and the same rules.

Stage 4 — CRE evaluation

Plain. An independent program reads the courier's record and the locked box, and decides whether the delivery condition is satisfied.

Technical. The cron-triggered workflow fetches `GET /api/shipments` through the HTTP capability with consensus aggregation, parses defensively, and for each terminal shipment encodes `getEscrow` and reads it through the EVM capability at the last finalised block. It skips anything not `Funded`, applies `shouldSettle`, and builds the four-field payload. The report is generated, the payload source is verified, and two structured log records are emitted.

Stage 5 — Local relay

Plain. Because the workflow cannot post to our local test chain directly, a small relay carries its signed-off decision the last step and submits it.

Technical. The bridge runs the simulation, parses only `PROOFPAY_REPORT_V1:` lines, joins report and meta by order ID, validates the outcome, decodes the bytes independently, recomputes the report ID, applies the allowlist, checks the escrow is still funded, and encodes `processReport(0x, report)`. It sends the transaction from the unlocked admin account and waits for a successful receipt, then records the submission.

Stage 6 — Settlement

Plain. The receiver checks that the report is legitimate and unused, then tells the locked box to pay. Delivered means the seller is paid; lost means the buyer is refunded.

Technical. `_processReport` decodes, derives the report ID, checks `processedReports`, `orderSettled` and the funding self-call, then branches on the outcome hash. `confirmDelivery` writes `Released` and transfers to the seller; `confirmFailure` writes `Refunded` and transfers to the buyer. Both emit settlement events; the receiver marks both mappings and emits `ReportProcessed` and `SettlementExecuted`.

Table 10.1: The three settlements that were actually executed and verified

Test	Outcome	Order	Block	Gas	Verified effect
TC-1	DELIVERED → RELEASE	`0x4d07…aaf3`	24	122,471	Escrow Released; seller +100 pUSD; `orderSettled true`.
TC-2	LOST → REFUND	`0x0c34…1152`	29	122,425	Escrow Refunded; buyer +100 pUSD; seller unchanged.
TC-3	DELIVERED → RELEASE	`0xf694…c26434`	30	122,471	Escrow Released; seller +100 pUSD, total 5,993.

One detail from the refund run is worth memorising, because it demonstrates the safety design in a single log line: the CRE simulation reported *two* settlements — the lost order as eligible, and the delivered order as "NOT IN ALLOWLIST" — and the bridge submitted exactly one. CRE legitimately found two candidates; the allowlist and the skip logic ensured only the intended order moved money.

> **INTERVIEW CHECKPOINT — MODULES 9 AND 10**
>
> **Q.** "Walk me through one order end to end." **A.** Use the six stages above, and end with the numbers: two wallet confirmations to fund, three courier updates, one CRE report of 256 bytes inside a 365-byte raw report, one `processReport` transaction at ~122k gas, receipt status 1, and exactly one balance change of the escrowed amount.

MODULE 11

SECURITY AND DESIGN DECISIONS, DEFENDED

Interviewers rarely ask "what does this function do?" — that is in the code. They ask "why is it built this way, and what did you give up?". This module answers that question thirteen times, using a fixed structure so the answers become habit: what we chose, why, what the alternatives were, why we rejected them, and what the trade-off costs.

11.1 Threat model first

Table 11.1: Who might attack what, and which control answers it

Adversary	Capability	Control that limits them
Buyer	Funds orders, controls their own key, can call any contract function.	Cannot withdraw after funding; cannot settle, because settlement is `onlyOracle`; cannot fund in someone else's name.
Seller	Registers shipments, controls the courier record through the API secret.	Can mark a parcel delivered, but cannot move money; settlement still requires a funded escrow, one payout, and the receiver's checks.
Admin / oracle operator	Owns contracts, configures addresses, holds the manual-oracle role.	Can rotate the oracle (visible via events) but has no function that transfers escrowed funds to an arbitrary address.
Bridge operator	Runs the relay, chooses which reports to submit.	Cannot invent a valid report that the workflow did not produce without also forging the fields the receiver hashes; cannot replay; cannot settle twice.
Network observer	Reads everything on chain.	Sees amounts and addresses, but only a hash of the tracking ID — a deliberate privacy choice.

11.2 Thirteen decisions, each defended

1. An escrow contract instead of paying the seller directly

Chosen. Buyer deposits to `ProofPayEscrow`; the seller is paid only on an oracle-authorised delivery. **Why.** The product being sold is protection; without custody there is nothing to protect. **Alternatives.** Direct payment with a refund process; a marketplace holding funds; a payment channel. **Rejected because.** Direct payment makes the buyer chase a refund from an unwilling seller; a marketplace reintroduces the trusted intermediary; channels solve streaming, not conditional sales. **Trade-off.** Funds can be stuck if the oracle never fires, and the buyer's capital is illiquid for the order's duration — accepted, because "stuck" is recoverable and "drained" is not.

2. A dedicated ERC-20 (pUSD) instead of ETH or a real stablecoin

Chosen. A 16-line ERC-20 with 18 decimals and owner-only mint. **Why.** It separates the value under protection from gas; it makes amounts exact in integers; it lets the demonstration mint balances on demand. **Alternatives.** Pay in ETH; integrate USDC/DAI; accept multiple tokens. **Rejected because.** ETH-denominated escrow conflates fees with principal and makes the demonstration's numbers unstable; integrating a real stablecoin adds external dependencies and regulatory claims the project cannot support; multi-token support multiplies accounting paths. **Trade-off.** pUSD is not money — no reserve, no peg, no redemption — so every claim about it must be scoped to "prototype token", which the report and this guide both do.

3. A separate receiver contract in front of the escrow

Chosen. `ProofPayCREReceiver` holds authorisation, decoding, replay protection and outcome mapping; the escrow only pays. **Why.** The receiver is the standard interface a DON calls, and isolating report handling keeps the custody contract small enough to review line by line. **Alternatives.** Put `onReport` directly on the escrow; let the escrow accept (orderId, outcome) from an EOA. **Rejected because.** Merging them would force the escrow to import ABI-decoding and replay logic, enlarging the attack surface of the contract that holds money; the EOA variant collapses the oracle back to a single trusted key with no provenance. **Trade-off.** One more deployment, one more address to configure, and about 51,000 extra gas per settlement (122,471 versus 71,186 for a direct call) — a fixed cost that buys auditable report handling.

4. Three roles: owner, forwarder, manualOracle

Chosen. Ownership configures; the forwarder is the intended automated reporter; `manualOracle` is a deliberate local and break-glass path. **Why.** Separating configuration from settlement means the key that can change addresses is not the key that moves money in normal operation. **Alternatives.** A single admin role; no manual path at all. **Rejected because.** One role concentrates risk; no manual path makes the system untestable locally and unrecoverable if the oracle dies. **Trade-off.** A local deployment with a zero forwarder lets the owner settle through the third branch of `onlyAuthorized` — a real weakening, documented in the code, the report and Module 12, and removed by setting a forwarder in production.

5. Chainlink CRE instead of a custom Node script that settles orders

Chosen. The decision lives in a CRE workflow with HTTP and EVM capabilities, report encoding and structured output. **Why.** The workflow is the component designed for exactly this: fetch external data, read chain state, compute, produce a report a contract can act on — and it is portable to a DON. **Alternatives.** A cron job holding a key and calling `confirmDelivery`; Chainlink Automation; Chainlink Functions. **Rejected because.** A key-holding script is a single trusted signer with no report artefact and no path to decentralisation; Automation triggers existing functions rather than running arbitrary multi-source logic; Functions is a request/response value fetch, not a workflow with its own decision table and report. **Trade-off.** CRE's chain-selector support and DON availability constrain the local deployment, which is the sole reason the bridge exists.

6. A local execution bridge instead of waiting for DON access

Chosen. Run the official simulator with `dryRun: true` and relay the resulting report locally. **Why.** It lets the whole pipeline be built, tested and measured now, while keeping CRE as the only decision-maker. **Alternatives.** Skip on-chain settlement entirely; fake the settlement in the UI; write a mock oracle that decides on its own. **Rejected because.** Skipping settlement makes the project a diagram; faking it in the UI would be dishonest; a self-deciding mock oracle would move the policy out of CRE and invalidate the architecture's central claim. **Trade-off.** The trust model is weaker than a DON: one machine runs the simulation and one account submits. That limitation is stated in the code header, the report and Module 8 rather than hidden.

7. The four-field report payload

Chosen. `(bytes32 orderId, string outcome, bytes32 trackingIdHash, string observedAt)`. **Why.** The order identifies the escrow; the outcome selects the payout; the tracking hash ties the decision to evidence without publishing the tracking number; the timestamp distinguishes two reports and dates the observation. **Alternatives.** Just `(orderId, outcome)`; include the full tracking ID; include the courier's raw JSON. **Rejected because.** Two fields give no provenance and make replay IDs ambiguous; the raw tracking ID leaks shipping data on a public ledger; the raw JSON is unbounded, expensive and unparseable on chain. **Trade-off.** The receiver cannot independently reconstruct the decision — it trusts CRE's outcome and can only check authorisation and uniqueness. That is inherent to any oracle design; the mitigation is the report's provenance record plus on-chain replay protection.

8. Two mappings: processedReports and orderSettled

Chosen. One keyed on a hash of the full report, one on the order. **Why.** They block different attacks: identical replay versus a fresh-looking report on an already-paid order. **Alternatives.** Only the order mapping; only the report mapping. **Rejected because.** With only the report mapping, an attacker who changes one character of the timestamp gets a new ID and a second payout attempt (stopped only by the escrow); with only the order mapping, you lose the audit record of which report settled which order. **Trade-off.** Two storage writes per settlement, roughly 40,000 gas — trivial against the value protected.

9. Terminal states with no exit

Chosen. `Released` and `Refunded` are final; no function reverses them; no owner rescue. **Why.** Reversibility is the property an attacker wants and the property a payment system must not have. **Alternatives.** An admin `reverseSettlement`; a pause function; an upgrade proxy. **Rejected because.** Each creates a privileged path to move money and a governance question the project cannot answer; an upgrade proxy would make "code is law" conditional. **Trade-off.** Mistakes are permanent. The mitigation is that mistakes are prevented upstream: preflight, allowlist, dry-run defaults and 31 tests.

10. Shipment state kept off chain

Chosen. Express + JSON persistence, read by CRE over HTTP. **Why.** Status changes six times per order, comes from an external process, and must be correctable. **Alternatives.** Store status on chain via an oracle write per update; use IPFS; use an event-sourced log on chain. **Rejected because.** Six paid writes per order is wasteful and every misscan becomes permanent; IPFS is content-addressed and awkward for mutable state; on-chain event sourcing has the same cost problem. **Trade-off.** The API becomes the trust boundary, mitigated by validating it against the escrow on chain before any payout.

11. The `-LOST` tracking-ID convention in the scheduler

Chosen. A demo scheduler derives the terminal status from the tracking ID suffix. **Why.** It makes a self-driving demo deterministic with no extra configuration. **Alternatives.** A random outcome; a config file per shipment; an admin click. **Rejected because.** Randomness makes demonstrations unrepeatable; per-shipment configuration is clumsy; an admin click defeats the point of automation. **Trade-off.** It is a courier-fact convention, not a settlement rule, and could be mistaken for one — hence the explicit comment in the file and this paragraph.

12. Dry-run defaults and the order allowlist

Chosen. `LOCAL_BRIDGE_DRY_RUN=true` by default; `BRIDGE_ORDER_ALLOWLIST` optionally restricts a run to named orders; CRE's own `dryRun` stays true. **Why.** Two independent keys before money moves, and a scope control that prevents a legitimate multi-report run from settling orders you did not intend. **Alternatives.** One flag; no allowlist; a confirmation prompt. **Rejected because.** One flag is one mistake away from an unintended transaction; without an allowlist the lost-order run would have settled two orders instead of one; a prompt cannot be used in a scheduler. **Trade-off.** More configuration to explain, and a user who forgets to set the flag sees a "planned" transaction instead of a real one — which the logs state plainly.

13. No private key in the repository

Chosen. The bridge sends `eth_sendTransaction` from Hardhat's unlocked account. **Why.** A settlement key on disk is the single most valuable file an attacker could find. **Alternatives.** A `CRE_ADMIN_PRIVATE_KEY` in `.env`; a keystore file; a hardware wallet. **Rejected because.** An env key is trivially leaked by a screenshot, a commit or a log; a keystore still needs a password on disk; hardware signing is impossible for an unattended scheduler. **Trade-off.** The design only works on a node whose accounts are unlocked, i.e. a local development chain. A production deployment would use a KMS or a dedicated signer service, and the guide says so rather than implying the local pattern is production-ready.

11.3 The limitations to volunteer, not hide

Table 11.2: Honest limitations and the precise wording to use

Limitation	How to say it
Local chain only	"Everything runs on Hardhat Local, chain ID 31337; no public testnet or mainnet deployment exists, so there is no explorer history to point at."
No DON	"The CRE workflow runs in the official CLI simulator with dry-run enabled; it is real workflow execution, not DON execution."
Simulated courier	"The courier is a script; there is no DHL or FedEx integration. The API has the shape a real carrier integration would need."
Bridge trust	"One machine relays the report. It cannot change the decision, but it could suppress or forge one; the receiver's checks bound the damage to one payout per funded order."
Prototype token	"pUSD has no reserve or peg. It exists so the escrow has a unit of account separate from gas."
No audit	"No third-party audit or formal verification. The mitigations are a small contract surface, OpenZeppelin primitives and 31 tests."
Owner fallback in the receiver	"When no forwarder is configured, the owner can settle. That is a local convenience and would be removed in production."
Infrastructure	"JSON file storage, a shared-secret header instead of real authentication, unlocked accounts. Development-grade by design."

MODULE 12

DEBUGGING AND TROUBLESHOOTING HANDBOOK

Every entry below is a failure you can actually hit in this project, in roughly the order you will hit them. The pattern is always the same: identify which layer owns the failure, then read the exact error rather than guessing.

12.1 How to read an error

Four sources of truth, in this order. The browser console shows the raw provider error object with a `code` and often a decoded revert. The MetaMask activity tab shows what was signed and its status. The node console shows the transaction and any revert reason. The receipt, fetched by hash, tells you definitively whether it succeeded (`status: 1`) or reverted (`status: 0`). Never diagnose from a toast notification.

12.2 Wallet, network and gas

Table 12.1: Wallet and network problems

Symptom	Cause	How to confirm	Fix
"MetaMask was not detected"	Extension missing or disabled; page loaded before injection.	In console: `typeof window.ethereum` is `undefined`.	Install/enable, reload the page. Note the app also reports this state in `hasProvider`.
Connect does nothing	User rejected the request, or the tab was never focused.	Provider error code `4001`.	Click connect again; treat 4001 as a rejection, not a bug.
Red "wrong network" banner; no write buttons	Wallet on mainnet or another chain.	Header shows a chain other than Hardhat Local; `wallet.chainId !== 31337`.	Use the app's switch button, or add chain 31337 manually with RPC `http://127.0.0.1:8545`.
Switch fails with code 4902	The chain is not configured in MetaMask.	Error code 4902.	`useWallet.switchToHardhatLocal` then calls `wallet_addEthereumChain`; approve it.
"Insufficient funds for gas"	Account has no test ETH.	Header ETH balance is 0.	Use a Hardhat account; they start with 10,000 ETH. Never send real ETH.
Transaction pending forever	Nonce gap or gas price too low.	Pending transaction visible in the wallet; node never mined it.	Restart the local node (which erases state — read 12.7 first) or reset the account's activity in MetaMask.

12.3 Token and escrow problems

Table 12.2: Funding and settlement errors, decoded

Error	Meaning	Fix
`ERC20InsufficientAllowance(spender, allowance, needed)`	The escrow was not approved for this amount, or the approval was consumed by an earlier order.	Run `approve(escrow, amount)` again. ProofPay approves the exact amount, so a second purchase needs a second approval.
`ERC20InsufficientBalance(sender, balance, needed)`	Not enough pUSD.	Mint to the buyer from the admin account, or fund from an account that has pUSD. Check decimals: 3,499 pUSD is 3499e18.

Error	Meaning	Fix
`EscrowAlreadyExists(orderId)`	That 32-byte ID is already used.	Generate a fresh random order ID; never reuse one from a previous run.
`InvalidBuyer()`	The `buyer` argument is not `msg.sender`.	Pass the connected wallet address, not a hard-coded one.
`SameBuyerAndSeller()`	Buyer and seller are the same address.	Use the seller account `0x3C44…93BC`; the checkout also blocks self-payment in the UI.
`EscrowNotFound(orderId)`	No escrow for that ID on this chain.	Usually a restart-wiped chain or a wrong order ID; redeploy and re-fund (12.7).
`EscrowNotFunded(orderId)`	Already released or refunded.	Expected behaviour on a duplicate attempt. Verify with `getEscrow` before retrying anything.
`UnauthorizedOracle(caller)`	Caller is not `escrow.oracle()`.	Do not call settlement directly. Configure `escrow.setOracle(receiver)` and go through the receiver.

12.4 Receiver and oracle problems

Table 12.3: Receiver-side failures

Error / symptom	Cause	Fix
`UnauthorizedCaller(caller)`	Sender is neither forwarder, manual oracle, nor owner-with-no-forwarder.	Use the admin account, or set `manualOracle` to the account you are using. The admin panel shows both addresses so this is checkable before sending.
`ReportAlreadyProcessed(reportId)`	Identical bytes were already submitted.	Correct behaviour. Check the state file and the receiver mapping; do not resubmit.
`OrderAlreadySettled(orderId)`	A different report for an already-paid order.	Also correct behaviour. The order is terminal; investigate why CRE produced a second report (usually a stale shipment record).
`UnsupportedOutcome(outcome)`	Outcome string is not DELIVERED or LOST, e.g. MANUAL_REVIEW.	The workflow should never emit that; if it does, fix `shouldSettle` usage rather than widening the receiver.
Decode revert / `InvalidReportData`	Wrong bytes: the SDK metadata header was not stripped, or fields are in the wrong order.	Verify with the bridge's own decode check; confirm the payload is the 256-byte body, not the 365-byte raw report.
Receiver address is null in the UI	`VITE_RECEIVER_CONTRACT_ADDRESS` missing or malformed.	Set it in `frontend/.env` to `0xB7f8BC63BbcaD18155201308C8f3540b07f84F5e` for this deployment and restart Vite.

12.5 Shipment API problems

Table 12.4: Backend failures and their status codes

Response	Meaning	Fix
401 from POST or PATCH	Mutation secret missing or wrong.	Send the configured secret header; the seller client already does. Check the backend's config, never commit the value.
409 `INVALID_STATUS_TRANSITION`	Illegal move, e.g. CREATED → DELIVERED.	Follow `SHIPMENT_TRANSITIONS`; the response body lists `allowed`.
409 `ESCROW_NOT_FUNDED`	The on-chain escrow is released or refunded.	Register against a funded order; check `getEscrow`.
409 `SELLER_MISMATCH`	The seller address in the request is not the escrow's seller.	Connect the seller account that actually owns the order.

Response	Meaning	Fix
409 `DUPLICATE_TRACKING_ID` / `SHIPMENT_ALREADY_EXISTS`	Tracking ID or order already registered.	Use a new tracking ID per shipment; one shipment per order.
404 `ESCROW_NOT_FOUND`	Order ID has no escrow on chain.	Usually after a node restart; redeploy and re-fund.
API unreachable (network error)	Backend not running or wrong port.	Start it on 5001; confirm with `curl http://localhost:5001/api/shipments`.
Scheduler says "no shipments in flight"	All managed shipments are terminal, or their tracking IDs do not start with `PPD-`.	Register a new `PPD-` shipment against a funded order; the scheduler only advances matching records.

12.6 CRE and bridge problems

Table 12.5: Workflow and bridge diagnostics

Symptom	Cause	Fix / command to check
"HTTP error" and zero settlements	Shipment API down, or wrong base URL in `config.staging.json`.	Check the backend is on 5001 and the config path is `/api/shipments`.
"JSON parse error"	API returned HTML (proxy error page) or malformed JSON.	`curl` the endpoint and inspect the body, not just the status.
EVM read fails	Wrong RPC, wrong escrow address, or chain not reachable from the workflow.	Confirm `rpcUrl`, `chainId` and the escrow address in the staging config.
"Escrow not funded (status 2)" in CRE logs	Order already settled.	Nothing to fix; this is the skip working correctly.
No `PROOFPAY_REPORT_V1` lines	No eligible shipments, or report generation placed after the dry-run branch (the historical bug).	Confirm `runtime.report` runs before `if (config.dryRun)`.
"report bytes do not decode as (bytes32,string,bytes32,string)"	Raw report used instead of the body, so the 109-byte header is present.	Use the `report` field from the log record; the workflow already selects the correct slice.
Bridge: "No bytecode at receiver"	Node restarted; contracts gone, or wrong address.	Redeploy and re-authorise, then update `.env`.
Bridge: "Admin … is not an unlocked RPC account"	Signer is not one of the node's accounts.	Point `BRIDGE_ADMIN_ACCOUNT` at account 0, or start the node with defaults.
Bridge: "receiver.manualOracle() is …, but the bridge signer is …"	Wiring mismatch after a redeploy.	Run the authorisation script again: `setManualOracle` must equal the signer.
Bridge: "outcome … is not one of the CRE settlement outputs"	Report carried MANUAL_REVIEW or an unexpected string.	Do not widen the bridge; fix the workflow so it emits no report for review cases.
Bridge: "CRE dryRun is false"	Someone enabled CRE writes while also running the bridge.	Keep CRE `dryRun: true` when using the bridge; the check exists to prevent double delivery.
Bridge refuses to start: lock exists	Previous run crashed, or another instance is alive.	The lock file stores a pid; a dead pid is removed automatically. Check for a running process before deleting manually.

12.7 "I restarted Hardhat and everything vanished"

Why. The chain state lives in the node process. Restarting it produces a fresh chain at the same deterministic addresses, but with no deployments, no balances and no escrows. The backend's JSON still lists shipments that reference orders the new chain has never heard of.

Order of recovery matters. Start the node, redeploy the token and escrow, deploy the receiver, run the authorisation script, mint and distribute pUSD, then re-register shipments for the new order IDs, then run the bridge preflight. If

you restore in a different order, the preflight will fail on address bytecode or on oracle wiring — which is the system telling you what is missing.

What you lose. Every transaction hash previously quoted, including the three settlement hashes. That is why the report records block numbers and hashes with their block heights, and why it says they are historical records of one chain instance rather than permanent addresses.

> **INTERVIEW CHECKPOINT — MODULES 11 AND 12**
>
> **Q.** "What is the biggest security limitation?" **A.** The oracle path. Locally, one machine runs the simulation and one unlocked account submits; the external data source is a script with a shared secret. The contracts bound the damage — one payout per funded order, replay-blocked reports, authorised callers only — but a compromised API plus a compromised bridge could produce a wrong-but-well-formed settlement. A DON with multiple operators and a real carrier API is what removes that.

MODULE 13

YOUR PROJECT, FILE BY FILE

This module is the map. For each important file: what it is for, how it is structured, which functions matter, what it talks to, and the execution order when it runs. Use it as a reference while reading the code, and as a checklist before an interview.

13.1 The repository at a glance

Table 13.1: Where everything lives

Path	Responsibility
`contracts/contracts/ProofPayUSD.sol`	The pUSD ERC-20: standard token plus owner-only mint.
`contracts/contracts/ProofPayEscrow.sol`	Custody, order records, the two payout functions, oracle rotation.
`contracts/contracts/ProofPayCREReceiver.sol`	Report authorisation, decoding, replay and duplicate protection, outcome routing.
`contracts/scripts/deploy.js`, `deploy-receiver.js`, `authorize-receiver.js`	Deployment sequence and the two authorisation calls, with read-back verification.
`contracts/deployments/cre-receiver-localhost.json`	Deployment record: network, chain ID, addresses, forwarder, manual oracle, timestamp.
`contracts/test/*.test.js`	31 tests across the token, escrow and receiver.
`frontend/src/store/useWallet.ts`	EIP-1193 connection, chain check, account events, network switching.
`frontend/src/store/useEscrows.ts`	Discovers order IDs from logs, reads every escrow from storage, builds activity feed.
`frontend/src/store/useEscrowActions.ts`	Transaction sequencing for approve, create, and settlement; exposes `tx.phase`.
`frontend/src/lib/contracts.ts`, `escrow.ts`, `wallet.ts`, `receiver.ts`	Unit conversion, contract reads, address helpers, report encoding.
`frontend/src/pages/*`, `components/admin/*`	Buyer, seller, admin, checkout, order tracking, oracle panel, shipment simulator.
`backend/src/app.js`, `routes/shipments.js`, `services/shipmentService.js`, `services/shipmentStore.js`	Express app, routes, business rules, atomic JSON persistence.
`backend/src/lib/validation.js`, `lib/errors.js`, `middleware/mutationAccess.js`, `services/escrowChain.js`	Input normalisation, `ApiError`/`asyncHandler`, shared-secret guard, on-chain escrow verification.
`chainlink/cre/project.yaml`, `workflow.yaml`, `shipment-workflow/*`	CRE project, workflow, config, decision logic, report export, bridge, demo scheduler.

13.2 ProofPayUSD.sol

Purpose. Define the unit of account. **Architecture.** Two OpenZeppelin bases and three members: constructor, `decimals`, `mint`. **Interactions.** Called by the escrow (`transferFrom`, `transfer`) and by the admin (mint); read by the

frontend and the evidence scripts. **Flow.** Deploy → mint to admin → transfer to buyer/seller → approve escrow → escrow pulls funds → escrow pays out.

Two details to be able to defend: decimals are explicit even though 18 is the default, and `mint` is the only privileged function. There is no burn, pause, blacklist or fee, so the escrow's "exact amount arrives" assumption always holds.

13.3 ProofPayEscrow.sol

Purpose. Hold money and release it exactly once, to exactly one party, on the oracle's instruction. **Architecture.** One enum, one struct, one mapping, an immutable token, a mutable oracle, five external functions and two private helpers. **Interactions.** Reads the token; is called by buyers (fund) and by the receiver (settle); read by the frontend and by CRE.

Table 13.2: Execution flow of each escrow entry point

Entry point	Sequence
`createEscrow`	Guard reentrancy → validate orderId, buyer==sender, seller, distinctness, amount, uniqueness → write struct as Funded → `safeTransferFrom` → emit `EscrowCreated`.
`confirmDelivery`	Guard → `onlyOracle` → `_getFundedEscrow` → write Released and `settledAt` → `safeTransfer` to seller → emit `EscrowReleased`.
`confirmFailure`	Same shape, writes Refunded, transfers to buyer, emits `EscrowRefunded`.
`getEscrow`	Read struct; revert `EscrowNotFound` if status is None; return memory copy.
`setOracle`	`onlyOwner` → reject zero → write → emit `OracleUpdated(previous, new)`.

13.4 ProofPayCREReceiver.sol

Purpose. Turn an authorised report into exactly one settlement call. **Architecture.** Immutable escrow reference, two configurable authorised callers, two boolean mappings, two entry points that share one internal routine, and an ERC-165 advertisement. **Interactions.** Called by the forwarder (a DON) or the manual oracle (local/admin); calls the escrow's two settlement functions.

```
// The essential shape, in five lines:
decoded = abi.decode(report, (bytes32, string, bytes32, string));
reportId = keccak256(abi.encodePacked(orderId, outcome, trackingIdHash, observedAt, metadata));
require(!processedReports[reportId] && !orderSettled[orderId]);
require(escrowIsFunded(orderId));
outcome == "DELIVERED" ? escrow.confirmDelivery(orderId) : escrow.confirmFailure(orderId);
```

Flow, in words. Receive bytes → decode → build report ID → reject replay → reject duplicate order → verify funding via a self-call (so a missing escrow becomes a typed error) → hash the outcome string and branch → mark both mappings → emit two events. Anything unexpected reverts, and a revert changes nothing.

13.5 chainlink/cre/shipment-workflow/main.ts

Purpose. Fetch shipments, verify escrows, decide, and produce a report — and nothing else. **Architecture.** One cron handler; a pure decision function; an HTTP fetch helper; an ABI-encoding helper; a payload-extraction helper; two structured log emissions. **Interactions.** Calls the Express API over HTTP, the escrow over EVM, and writes to stdout for the bridge.

```
// Execution order inside onCronTrigger:
1. runtime.config                        → typed staging configuration
2. HTTPClient.sendRequest(consensusIdenticalAggregation)  → shipments JSON
3. JSON.parse with array-or-{shipments} tolerance
4. for each shipment:
     shouldSettle(status) → NONE / MANUAL_REVIEW ? log & continue
     encodeFunctionData(getEscrow) → EVMClient.callContract(LAST_FINALIZED_BLOCK_NUMBER)
     decodeFunctionResult → status !== 1 ? log & continue
     createReportPayload → prepareReportRequest → runtime.report
     extractAbiPayload → verify slice decodes to (orderId, outcome)
     log PROOFPAY_REPORT_V1 {type:"report"} and {type:"meta"}
     if (config.dryRun) { log "DRY RUN"; continue; }
     evmClient.writeReport(...)          // unreachable in this deployment
```

The two subtleties that mark this as real work rather than a tutorial: report generation is deliberately above the `dryRun` branch, and the payload extraction verifies which slice actually decodes rather than assuming a fixed 109-byte offset.

13.6 chainlink/cre/shipment-workflow/local-runner.ts

Purpose. Relay CRE's report to the receiver, safely and exactly once. **Architecture.** Roughly 1,180 lines in nine sections: constants and ABIs; configuration; logging and retry helpers; simulation spawner and line consumer; validation; state and locking; chain context and preflight; escrow reads and transaction preparation; the main loop and summary. **Interactions.** Spawns the CRE CLI, reads the HTTP-served chain through viem, and sends one transaction per eligible report.

Table 13.3: local-runner.ts by section, with the function to look for

Section	Key symbols	What to read first
Constants / ABIs	`REPORT_LOG_PREFIX`, `SETTLEMENT_REPORT_ABI`, `RECEIVER_ABI`, `ESCROW_ABI`	The error definitions are declared so reverts can be humanised.
Configuration	`loadConfig`, `envFlag`, `parseOrderAllowlist`	`dryRun: envFlag("LOCAL_BRIDGE_DRY_RUN", true)` — the default that protects you.
Simulation	`runCreSimulation`, `consumeLine`	How stdout is split into report and meta records.
Validation	`validateReport`	The `creDryRun === false` refusal.
State / lock	`loadState`, `saveState`, `acquireLock`	Idempotency and single-instance guarantees.
Preflight	`createClients`, `preflight`	Six checks, each mapping to a real failure mode.
Execution	`readEscrow`, `prepareTransaction`, `submitTransaction`, `processReport`, `main`	The summary counters: submitted / planned / skipped / failed.

13.7 chainlink/cre/shipment-workflow/courier-scheduler.ts

Purpose. Advance demo shipments on a timeline and optionally run CRE + bridge afterwards. **Architecture.** Config loader; `nextStatus`; due-checking with per-shipment delays; `patchStatus`; `runBridge`; a lock; a tick loop with idle-log suppression. **Interactions.** PATCHes the Express API; spawns the bridge. It never calls a contract.

The two functions to know: `nextStatus(status, trackingId)` which encodes the timeline and the `-LOST` convention, and `tick(config)` which filters due shipments, applies updates sequentially, and runs the bridge only if something changed.

13.8 The shipment service (backend)

Purpose. Be the authoritative off-chain record with strict transitions. **Architecture.** Router → service → store, with validation and errors as libraries. **Interactions.** The router is called by React and the scheduler; the service calls `escrowChain.verifyEscrow` over RPC; the store writes JSON.

```
router.post('/', mutationAccess, asyncHandler(async (req, res) => {
  const shipment = await shipmentService.register(req.body);
  res.status(201).location(`/api/shipments/${shipment.trackingId}`).json(shipment);
}));

// service.register():
normalizeCreateShipment(body) → assertNoDuplicates → verifyEscrow(orderId)
→ 404 ESCROW_NOT_FOUND | 409 ESCROW_NOT_FUNDED | 409 SELLER_MISMATCH
→ assertNoDuplicates again → build record with status CREATED + first event → store.add
```

Notice the deliberate double duplicate check around the async RPC call, and that the record's first event is created together with the record — there is no window in which a shipment exists without history.

13.9 The checkout page

Purpose. Turn a product selection into a funded escrow with as few failure modes as possible. **Architecture.** Controlled form state; validation in `handleSubmit`; transaction sequencing delegated to `useEscrowActions`; presentation only otherwise. **Interactions.** Reads token balance and allowance; sends approve then createEscrow; calls `onCreated` to trigger re-reads.

```
const busy = tx.phase === 'approving' || tx.phase === 'approved' || tx.phase === 'creating';
// validation before any wallet prompt:
if (!isValidAddress(sellerAddress)) next.seller = 'Enter a 0x address with 40 hex characters.';
else if (sellerAddress.toLowerCase() === ZERO_ADDRESS) next.seller = 'The zero address cannot
receive funds.';
else if (sellerAddress.toLowerCase() === (wallet.address ?? '').toLowerCase())
  next.seller = 'The seller cannot be your own wallet.';
if (parsed.error) next.amount = parsed.error;
```

That block is worth quoting in an interview: it shows the UI enforcing the same invariants as the contract, before spending the user's time on a doomed signature. The contract still enforces them; the UI just fails fast.

13.10 The oracle panel (admin)

Purpose. Prove the wiring and allow a manual, receiver-mediated settlement. **Architecture.** Two `useEffect` reads (`escrow.oracle()` and `receiver.manualOracle()`), derived booleans `receiverAuthorized` and `walletAuthorized`, and a submit path that encodes a report client-side. **Interactions.** Reads through EIP-1193 `eth_call`; writes `processReport` from the connected account.

```
const receiverAuthorized = receiverDeployed && escrowOracleState?.toLowerCase() ===
addr.toLowerCase();
const walletAuthorized  = wallet.address?.toLowerCase() === receiverManualOracle?.toLowerCase();
// Both must be true before the panel offers "Confirm delivery" / "Confirm failure".
```

This panel is the manual counterpart to the automated pipeline, and it is genuinely useful: it is how the receiver path was validated before CRE existed, and it remains the diagnostic tool when the automated path misbehaves.

13.11 lib/receiver.ts

Purpose. Build a receiver call without any framework magic. **Architecture.** A minimal ABI (two entries), an environment reader, a mode flag, and one encoder. **Interactions.** Used by the oracle panel; mirrors the CRE payload exactly.

The important line is the zero tracking hash: the manual path has no tracking record, so it passes `0x0…0`. The report ID remains unique because the order and timestamp differ. If asked why the manual path is weaker, this is the evidence: no tracking hash means no link to a courier record, which is precisely what the automated path provides.

> **INTERVIEW CHECKPOINT — MODULE 13**
>
> **Q.** "Which file would you open first to understand the system?" **A.** `ProofPayEscrow.sol`, because it defines what money does; then `ProofPayCREReceiver.sol` for who may trigger it; then `main.ts` for how the decision is produced. The frontend and backend are consumers of those three.

MODULE 14 · PART A

INTERVIEW AND VIVA BANK: LEVELS 1–3

Each answer follows the same rhythm: a short answer you can say in one breath, then the reasoning, then the ProofPay instance. Say the short answer first even in a viva — examiners grade clarity, then probe.

14.1 Level 1 — foundations

Q1. What is a blockchain?

Short: a database replicated across independent computers where changes are signed transactions grouped into hash-linked blocks. **Reasoning:** replication plus agreed validity rules means no single party can rewrite history unnoticed. **In ProofPay:** the escrow's state, every deposit and every settlement live in such a ledger, so neither buyer nor seller can alter the record of who was paid.

Q2. What is Ethereum?

Short: a blockchain whose nodes also run a virtual machine that executes programs called smart contracts. **Reasoning:** Bitcoin stores value transfers; Ethereum stores arbitrary state transitions, which is what makes conditional escrow possible. **In ProofPay:** the payment rules are code, not policy.

Q3. What is a smart contract?

Short: a program deployed to an address on the blockchain, with persistent public storage, executed deterministically by every node. **Reasoning:** determinism is what allows consensus; persistence is what makes it a record; no upgrade path is what makes it credible. **In ProofPay:** `ProofPayEscrow` holds funds and can only pay out through two functions.

Q4. What is Solidity?

Short: the main high-level language for EVM contracts, compiling to bytecode. **Reasoning:** it adds contract-specific constructs — modifiers, events, custom errors, storage mappings — on top of a JavaScript-like syntax. **In ProofPay:** all three contracts are Solidity 0.8.28, with OpenZeppelin for standard behaviour.

Q5. What is MetaMask?

Short: a browser extension wallet that stores keys, signs transactions, and injects an EIP-1193 provider at `window.ethereum`. **Reasoning:** it separates key custody from the application, so a website can request signatures without ever seeing a key. **In ProofPay:** every write — approve, createEscrow, processReport — is signed there.

Q6. What is an ERC-20 token?

Short: a standard interface for fungible tokens: balances, transfers, allowances and two events. **Reasoning:** standardisation means wallets and contracts can handle any token without custom code. **In ProofPay:** pUSD implements it, so the escrow can use `SafeERC20` and treat it like any compliant asset.

Q7. What is gas?

Short: a unit measuring computational work, paid for in the chain's native currency. **Reasoning:** it prices storage and computation, preventing infinite loops and spam. **In ProofPay:** `createEscrow` costs about 156,000 gas and `processReport` about 122,000; the amounts were read from receipts, not estimated.

Q8. What is an escrow?

Short: a neutral hold on funds until an agreed condition is met. **Reasoning:** it removes the "who goes first" problem by making neither party able to spend prematurely. **In ProofPay:** the contract is the neutral holder, and the condition is an oracle report about delivery.

Q9. What is a transaction hash?

Short: the Keccak-256 digest of a signed transaction, used as its unique identifier. **Reasoning:** it is deterministic, so anyone can look it up on any node and see the same receipt. **In ProofPay:** the refund hash `0x835890…a5288` identifies the block-29 settlement.

Q10. What is a block number?

Short: the position of a block in the chain. **Reasoning:** ordering is what gives finality; a receipt's block number is when its effects became part of history. **In ProofPay:** settlements at blocks 24, 29 and 30, each exactly one higher than the last, which also proves no hidden extra transaction.

Q11. What is a nonce?

Short: a per-account counter that orders that account's transactions and prevents replay. **Reasoning:** a transaction with nonce n cannot be mined before n−1, and cannot be mined twice. **In ProofPay:** a stuck nonce is the usual cause of a "pending forever" transaction on the local node.

Q12. What is a chain ID?

Short: the network identifier that is signed into every transaction. **Reasoning:** it prevents a transaction signed for one chain from being replayed on another. **In ProofPay:** 31337 for Hardhat Local, and the UI refuses writes when the wallet reports anything else.

Q13. What is an RPC endpoint?

Short: an HTTP (or WebSocket) URL that a client uses to talk to a node. **Reasoning:** it exposes methods such as `eth_call`, `eth_sendTransaction` and `eth_getLogs`. **In ProofPay:** `http://127.0.0.1:8545` is used by the wallet, the frontend, the backend verifier, the workflow and the bridge.

Q14. What is the EVM?

Short: the deterministic virtual machine that executes contract bytecode. **Reasoning:** determinism plus metered execution (gas) is what makes contract results agree across nodes. **In ProofPay:** the escrow's validations and the token's balance updates are EVM execution.

Q15. What is the difference between an address and an account?

Short: an address is the 20-byte identifier; an account is the thing it identifies, with a balance and possibly code and storage. **Reasoning:** every address is an account, but only EOAs can initiate transactions. **In ProofPay:** the receiver has an address, an account, code and two mappings, but no key — it can only act when called.

Q16. What is a private key, and who holds the project's?

Short: the 256-bit secret that signs transactions. **Reasoning:** possession is authority; there is no password reset. **In ProofPay:** no key is stored in the repository. The local Hardhat node holds test keys in memory and exposes the accounts as unlocked, so the bridge signs via `eth_sendTransaction`.

Q17. What is a hash function and why Keccak-256?

Short: a one-way function mapping any input to a fixed 32-byte fingerprint, with avalanche behaviour. **Reasoning:** Ethereum uses Keccak-256 for block linkage, addresses, selectors and event topics, so using it keeps your code consistent with the platform. **In ProofPay:** order IDs, tracking hashes and report IDs are all Keccak-256.

Q18. What is a digital signature?

Short: a proof, verifiable with a public key, that the holder of the matching private key approved specific bytes. **Reasoning:** signatures bind authorisation to content, so changing any field invalidates them. **In ProofPay:** that is why editing a request in the browser cannot move funds.

Q19. What does immutability actually mean here?

Short: rewriting history requires redoing all later blocks faster than honest participants extend the chain. **Reasoning:** immutability is economic and social, not absolute. **In ProofPay:** strong on chain, absent on the local node — restarting Hardhat erases everything, so no mainnet-style guarantee is claimed.

Q20. Why is the blockchain public a privacy problem, and how did you handle it?

Short: every address, amount and call is visible forever. **Reasoning:** payment transparency is the feature; leaking shipping details is the risk. **In ProofPay:** the report stores `keccak256(trackingId)` rather than the tracking number, so settlements are auditable without publishing parcel identifiers.

Q21. What is a testnet, and did you use one?

Short: a chain with valueless currency for development. **Reasoning:** it lets you rehearse without risking assets while still being public. **In ProofPay:** no. The project runs entirely on Hardhat Local; a Sepolia network entry exists in the config but was never used, and the report says so.

Q22. What is a wallet, if it does not hold coins?

Short: software that holds keys and signs; the coins are ledger entries. **Reasoning:** restoring a seed phrase restores access, not data. **In ProofPay:** deleting the extension changes nothing on chain; the escrow still holds the pUSD.

14.2 Level 2 — mechanics

Q23. How does MetaMask communicate with React?

Short: through the injected EIP-1193 provider's `request()` method and its events. **Reasoning:** the provider is a transport, not a library, so any framework can use it. **In ProofPay:** `useWallet` calls `eth_requestAccounts`, `eth_chainId`, `eth_getBalance`, and subscribes to `accountsChanged` and `chainChanged`.

Q24. What is EIP-1193?

Short: the standard describing the injected provider object and its request/response protocol. **Reasoning:** before it, each wallet invented its own API; after it, one code path works across wallets. **In ProofPay:** every chain interaction in the browser goes through it.

Q25. What is an ABI?

Short: a JSON description of a contract's functions, arguments, returns, events and errors. **Reasoning:** the EVM only sees bytes, so the ABI is how tooling encodes calls and decodes results. **In ProofPay:** the receiver's ABI has two functions in the frontend and seven entries in the bridge, including errors it decodes.

Q26. What is ABI encoding?

Short: the canonical byte layout for arguments: 32-byte words, offsets for dynamic types, lengths for strings and bytes. **Reasoning:** both sides must agree byte-for-byte or decoding fails. **In ProofPay:** the four-field report is 256 bytes, and its layout is why the receiver can decode it with one call.

Q27. What is a function selector?

Short: the first four bytes of `keccak256("name(types)")`, prefixed to calldata. **Reasoning:** it tells the contract which function to run without sending names. **In ProofPay:** the bridge's 388-byte transaction begins with the `processReport(bytes,bytes)` selector.

Q28. What is the difference between `call`, `staticcall` and `delegatecall`?

Short: `call` can change state, `staticcall` forbids it, and `delegatecall` runs another contract's code in the caller's storage. **Reasoning:** the distinction is what makes read-only RPC safe. **In ProofPay:** reads use `eth_call`, which is a static call; nothing in the project uses `delegatecall`, and no proxy pattern is used.

Q29. Why do you need `approve` before `transferFrom`?

Short: because a token contract will only debit an owner who has authorised that specific spender. **Reasoning:** without allowances, any contract you interact with could take everything. **In ProofPay:** transaction 1 sets `allowance[buyer][escrow]`; transaction 2 spends it inside `createEscrow`.

Q30. Why does the allowance get consumed?

Short: ERC-20 decrements the allowance on every `transferFrom`. **Reasoning:** an allowance is a spending limit, not a permanent grant. **In ProofPay:** a second purchase needs a fresh approval, which is why the checkout always runs the approve step rather than assuming it is still valid.

Q31. What is ReentrancyGuard and why is it there?

Short: a modifier that sets a storage flag so a function cannot be re-entered before it finishes. **Reasoning:** external calls hand control to code you do not control. **In ProofPay:** `createEscrow` and both settlement functions are `nonReentrant`, and a test attempts the attack.

Q32. What is SafeERC20?

Short: a wrapper that checks the return value of token calls and reverts on failure. **Reasoning:** some tokens return `false` instead of reverting, so ignoring the return value can silently lose funds. **In ProofPay:** all escrow transfers use it.

Q33. What is an event, and can a contract read it?

Short: a log entry stored in the transaction receipt; contracts cannot read logs. **Reasoning:** logs are cheap, indexable, and for off-chain consumers. **In ProofPay:** `EscrowCreated` lets the frontend discover orders, but every value shown is re-read from storage.

Q34. What is a custom error, and why use it?

Short: a typed revert with optional arguments, defined with the `error` keyword. **Reasoning:** cheaper than strings and precisely testable. **In ProofPay:** the tests assert `revertedWithCustomError(token, "ERC20InsufficientBalance").withArgs(...)`.

Q35. What is an oracle?

Short: a component that delivers external facts to a blockchain. **Reasoning:** contracts cannot fetch, so facts must be pushed in with evidence. **In ProofPay:** the CRE workflow is the oracle, and the receiver is the contract that decides whether to believe its report.

Q36. Why can't a smart contract call an API directly?

Short: because execution must be deterministic and identical on every node; HTTP is neither. **Reasoning:** one node getting a different response would fork consensus. **In ProofPay:** this constraint is the reason the whole oracle pipeline exists.

Q37. What is the oracle problem?

Short: how to get a trustworthy external fact into a system that cannot verify it. **Reasoning:** any oracle reintroduces a trust assumption; the design question is how to bound it. **In ProofPay:** bounded by authorisation, replay protection, one settlement per funded order and a documented trust boundary.

Q38. What is a DON?

Short: a decentralised oracle network: independent nodes that must agree before reporting. **Reasoning:** agreement removes the single-reporter assumption. **In ProofPay:** the code is written for it, but no DON is deployed, so this is future work, not a claim.

Q39. What is Chainlink CRE?

Short: a runtime for TypeScript workflows with triggers, HTTP and EVM capabilities, and signed reports for on-chain receivers. **Reasoning:** it packages the whole fetch-verify-decide-report loop. **In ProofPay:** it holds the only settlement decision in the system.

Q40. What is a cron trigger?

Short: a schedule-based trigger, here the expression `0 */5 * * * *`. **Reasoning:** settlement is periodic, not event-driven from the chain's perspective. **In ProofPay:** a deployed workflow would evaluate every five minutes; the simulator fires once per invocation.

Q41. What does the HTTP capability do in your workflow?

Short: performs `GET /api/shipments` and returns the body as text, with consensus aggregation available. **Reasoning:** this is the only place external data enters the decision. **In ProofPay:** a failure here ends the run with no settlement rather than a partial one.

Q42. What does the EVM capability do?

Short: performs a contract read (and, in principle, a report write) on a chain selected by a CRE chain selector. **Reasoning:** it lets the workflow verify on-chain state before deciding. **In ProofPay:** it reads `getEscrow` at the last finalised block and skips anything not `Funded`.

Q43. What is a CRE report?

Short: the SDK object holding your ABI-encoded payload plus a runtime metadata header. **Reasoning:** the header is provenance; the body is what the receiver decodes. **In ProofPay:** 109-byte header, 256-byte body, 365-byte raw report — and the extraction code proves which slice it used.

Q44. What is a receipt, and what does status 1 mean?

Short: the node's record that a transaction was included and what it did; status 1 means it succeeded. **Reasoning:** a hash alone proves broadcasting, not success. **In ProofPay:** every settlement was verified by receipt status, block number, gas used and the emitted events.

Q45. Why are reads free but writes costly?

Short: reads are executed locally against current state and change nothing; writes must be replicated and stored by every node. **Reasoning:** cost prices permanent storage. **In ProofPay:** this is why shipment status is off chain and only the escrow's essential facts are stored.

14.3 Level 3 — your architecture

Q46. Explain the ProofPay architecture in one minute.

Short: React talks to MetaMask and to an Express API; three contracts on Hardhat hold the money and the rules; a CRE workflow reads the API and the escrow, decides, and emits a report; a local bridge relays that report to the receiver, which instructs the escrow to pay. **Reasoning:** each layer owns one fact and one authority. **Instance:** the buyer's escrow, the seller's shipment record, CRE's decision, and the receiver's authorisation are four separate things that must all agree.

Q47. Why a custom token instead of ETH?

Short: to separate protected value from gas and to keep amounts exact. **Reasoning:** ETH-denominated escrow conflates fees with principal and makes demonstrations unstable. **Trade-off:** pUSD is a prototype with no reserve, and every claim must say so.

Q48. Why can't the seller access the funds immediately?

Short: because the escrow has no withdrawal function at all. **Reasoning:** protection requires that no one — seller, buyer, owner — can move the deposit before the condition. **Instance:** the only two paths are `confirmDelivery` and `confirmFailure`, both `onlyOracle`.

Q49. Why is there a separate receiver contract?

Short: to keep report handling out of the custody contract and to present the interface a DON expects. **Reasoning:** smaller money contracts are easier to audit; report policy changes more often than custody. **Trade-off:** one more address and ~51,000 extra gas per settlement.

Q50. What does `manualOracle` mean?

Short: an address the receiver will also accept reports from, intended for local testing and break-glass operation. **Reasoning:** without it, nothing could be settled before CRE worked, and an oracle outage would freeze funds. **Trade-off:** a real weakening when set to an EOA; production should use a multisig or remove it.

Q51. How does shipment registration work?

Short: the seller POSTs order ID, tracking ID, carrier and address to the API, which validates duplicates, verifies the escrow is funded on chain and that the seller matches, then stores a record with an event history. **Reasoning:** registration is the seller's claim; the chain verification is what makes the claim meaningful.

Q52. How does CRE read escrow data?

Short: it ABI-encodes `getEscrow(orderId)` with viem, sends it through `EVMClient.callContract` at the last finalised block, and decodes the returned struct. **Reasoning:** reading at a finalised block avoids deciding on a reorg-prone tip. **Instance:** status must be 1, otherwise the shipment is skipped and logged.

Q53. How does the local execution bridge work?

Short: it runs the CRE simulator, parses the structured report lines, validates and cross-checks them, then submits the exact bytes to `processReport` from an unlocked local account and verifies the receipt. **Reasoning:** the decision stays in CRE; the bridge only carries it. **Instance:** the block-29 refund.

Q54. Why does the bridge not just call `confirmFailure` itself?

Short: because only the oracle can, and the oracle is the receiver. **Reasoning:** routing through the receiver is what enforces replay and duplicate protection; a direct call would bypass the entire policy layer. **Instance:** the admin EOA cannot call the escrow's settlement functions at all.

Q55. What is the difference between the manual and automatic paths?

Short: manual builds a report in the browser with a zeroed tracking hash; automatic uses a CRE report with a real tracking hash and provenance. **Reasoning:** both pass the same receiver checks, but only the automatic path has evidence linking the decision to a courier record. **Instance:** the oracle panel is the manual path; the bridge is the automatic one.

Q56. Why is shipment status not stored on chain?

Short: it changes six times, comes from outside, and must be correctable. **Reasoning:** on-chain writes are permanent and cost gas; a mis-scan would be unfixable. **Instance:** the JSON store is the record; the chain stores only the settlement and its report ID.

Q57. How is a settlement prevented from happening twice?

Short: three layers: the escrow's Funded check, the receiver's two mappings, and the bridge's state file. **Reasoning:** each layer covers a different failure — a buggy caller, a replayed report, a retried process. **Instance:** blocks 24, 29 and 30 each advanced the chain by exactly one, so no duplicate was hidden.

Q58. What does the courier scheduler do, and what does it not do?

Short: it advances demo shipments through the timeline and optionally runs CRE and the bridge; it never decides settlement or touches contracts. **Reasoning:** separating the fact generator from the decision keeps CRE's authority intact. **Instance:** `-LOST` suffix decides the courier outcome, not the payout.

Q59. Why does the receiver store a hash of the tracking ID?

Short: to tie the settlement to a specific shipment record without publishing the tracking number. **Reasoning:** evidence plus privacy; it also contributes to the report ID. **Instance:** the manual path uses a zero hash, which is exactly why it is weaker.

Q60. What happens if CRE never runs?

Short: the escrow stays Funded; nothing is lost but nothing settles. **Reasoning:** the system fails safe, not open. **Instance:** the manual oracle path or a redeployed workflow is the recovery, and the funds remain claimable because they are still in custody.

Q61. Why does the checkout validate the seller address before prompting the wallet?

Short: to avoid wasting a signature on a call that must revert. **Reasoning:** the contract enforces the same rules, but failing in the UI is cheaper and clearer. **Instance:** zero address, self-payment and malformed addresses are blocked in `handleSubmit`.

Q62. How does the buyer dashboard know which orders are theirs?

Short: it discovers all order IDs from `EscrowCreated` logs, reads each escrow, and filters where `buyer == connected address`. **Reasoning:** chain state cannot be queried "by buyer" without an index, so filtering happens client-side. **Trade-off:** fine for a local demo, but a production app would index events in a database.

Q63. Why does the backend verify the escrow before accepting a shipment?

Short: a shipment for an unfunded order would have no money to settle. **Reasoning:** it also prevents a seller from registering an order that belongs to someone else. **Instance:** 404 `ESCROW_NOT_FOUND`, 409 `ESCROW_NOT_FUNDED`, 409 `SELLER_MISMATCH`.

Q64. What would break if you removed the allowlist from the bridge?

Short: a run intended for one order would settle every eligible order CRE reported. **Reasoning:** the allowlist scopes a demonstration without changing the decision. **Instance:** the refund run reported two shipments; without the allowlist, two settlements would have been submitted.

Q65. What is the most important line in main.ts, and why?

Short: `const report = runtime.report(reportRequest).result();` placed *before* the dry-run branch. **Reasoning:** generation must happen regardless of whether the write is attempted, otherwise the safe configuration produces nothing to capture. **Instance:** that reordering is what made the entire local bridge possible.

MODULE 14 · PART B

INTERVIEW AND VIVA BANK: LEVELS 4–5

14.4 Level 4 — advanced technical

Q66. Walk me through CRE report generation.

Short: encode the four fields, wrap in `prepareReportRequest`, call `runtime.report(...).result()`, then extract the body. **Reasoning:** the SDK adds a metadata header the receiver must not see. **Instance:** 256-byte payload, 365-byte raw report, source reported as `body`.

Q67. How exactly is the report encoded?

Short: `abi.encode(bytes32, string, bytes32, string)` — the order ID word, then a dynamic string, then a word, then a dynamic string. **Reasoning:** dynamic types use offsets, which is why the fixed part is four 32-byte words and the strings trail behind. **Instance:** the receiver uses one `abi.decode` with the same signature.

Q68. Why 109 bytes, and why does it matter?

Short: that is `REPORT_METADATA_HEADER_LENGTH`, the SDK's provenance prefix; the receiver decodes only the body. **Reasoning:** passing the raw report would shift every field by 109 bytes and fail to decode. **Instance:** 365 − 256 = 109, which is the arithmetic proof the strip was correct.

Q69. How does replay protection actually work?

Short: the receiver hashes the whole report and refuses a hash it has seen. **Reasoning:** identical bytes always produce the same digest, so a replay is detectable without storing the report. **Instance:** `keccak256(abi.encodePacked(orderId, outcome, trackingIdHash, observedAt, metadata))`.

Q70. Why do you need a second mapping for duplicate settlements?

Short: because a different report on a settled order produces a different report ID. **Reasoning:** without `orderSettled`, changing the timestamp would bypass replay protection. **Instance:** the escrow's Funded check would still block payment, but the receiver would waste gas and emit a misleading attempt.

Q71. Why can't the CRE SDK target local Hardhat?

Short: the installed SDK has no chain selector for chain ID 31337. **Reasoning:** selectors are an allow-list of supported networks, and local dev chains are not in it. **Instance:** the code calls `getNetwork` with a Sepolia selector because that is what compiles and runs; this is exactly why the bridge exists and why no CRE write is claimed.

Q72. Simulation versus DON deployment — what is the real difference?

Short: simulation runs your workflow in the CLI on one machine; deployment registers it with a DON where multiple operators execute it and agree. **Reasoning:** consensus, availability and signing authority all change. **Instance:** locally `dryRun` is true and a bridge relays; on a DON the workflow would write the report itself.

Q73. What exactly would you change for a DON deployment?

Short: deploy the workflow, flip `dryRun` to false, expose the API over HTTPS, set the receiver's `forwarder`, remove the owner fallback, delete the bridge. **Reasoning:** each change moves one responsibility from the local machine to the network. **Instance:** the escrow needs no change at all, because its oracle is already the receiver.

Q74. How does the receiver communicate with the escrow?

Short: it holds an immutable `IEscrowSettlement` reference and calls `confirmDelivery` or `confirmFailure`. **Reasoning:** because the receiver is the escrow's `oracle`, those calls satisfy `onlyOracle` without any EOA having that power. **Instance:** the escrow sees `msg.sender = receiver`.

Q75. What happens if report submission fails?

Short: the bridge reports the failure, decodes the revert, exits non-zero and records nothing. **Reasoning:** no state file entry means the submission is not falsely considered done; no retry means no duplicate risk. **Instance:** the operator fixes the cause (funding, wiring, authorisation) and runs again deliberately.

Q76. What if the transaction reverts after gas was spent?

Short: all state changes are undone; only gas is consumed. **Reasoning:** the EVM is atomic across the whole call tree. **Instance:** a reverted `processReport` leaves the escrow Funded and the mappings untouched, so a corrected retry is safe.

Q77. How would this scale to production?

Short: deploy the contracts to a public chain, run the workflow on a DON, replace the JSON store with a database, index events for the UI, use a KMS-backed signer for any manual path. **Reasoning:** each change addresses a specific local shortcut, not the architecture. **Instance:** the contract interfaces and the report format stay identical.

Q78. Where does the gas go in a settlement?

Short: mostly the token transfer, the two mapping writes, the escrow state write and the events. **Reasoning:** storage writes dominate; logs are cheaper. **Instance:** ~122,000 gas for `processReport` versus ~71,000 for a direct `confirmDelivery`; the ~51,000 difference is the policy layer's price.

Q79. Why terminal states instead of a reversible flag?

Short: because reversibility is the property an attacker needs. **Reasoning:** a reversible payout invites governance capture and makes balances non-final. **Instance:** the escrow's enum has no path back to Funded, and there is no admin rescue.

Q80. No upgradeability — is that a strength or a flaw?

Short: both, and you should say both. **Reasoning:** strength: no proxy admin can rewrite the rules; flaw: a bug is permanent and migration means a new deployment and manual fund movement. **Instance:** the project mitigates with a small surface, audited primitives and 31 tests rather than an upgrade path.

Q81. How would you add a second payment token?

Short: make the escrow's token per-order rather than immutable, or deploy one escrow per token. **Reasoning:** the immutable token is a deliberate simplification; multi-token support means storing the token address in the Escrow struct and validating it against a registry. **Instance:** the receiver would not change, because the escrow still owns payouts.

Q82. How would partial refunds work?

Short: a new settlement function that splits the amount and a report outcome that carries a basis-points field. **Reasoning:** the current payload has no room for an amount, and adding one changes the receiver's ABI. **Instance:** this is future work, listed in the report, not implemented.

Q83. What if the user switches accounts mid-checkout?

Short: `accountsChanged` re-runs `applyAddress`, the address changes, and the pending transaction is signed by the new account. **Reasoning:** the wallet is the authority, so the UI must follow it rather than cache an address. **Instance:** that is why `useWallet` exposes a `refresh` and subscribes to events.

Q84. Two orders for the same seller — any interference?

Short: none: state is keyed by order ID, and settlements are independent. **Reasoning:** the escrow mapping is per order, so one release cannot affect another. **Instance:** a test named "settles multiple escrows independently" pins exactly this.

Q85. How do you test the oracle without a real courier?

Short: with a deterministic simulated API and a scheduler that advances statuses. **Reasoning:** the oracle's job is to read a source, not to trust it, so a fake source exercises the same code path. **Instance:** the workflow's logs show it processing real records from the Express API, including skip and error branches.

Q86. What does idempotency mean in this system?

Short: running the pipeline twice has the same effect as running it once. **Reasoning:** settlement is the only irreversible effect, so it must be exactly-once, while report generation is safely repeatable. **Instance:** CRE may emit the same report again; the receiver and bridge both refuse it.

Q87. How would an auditor verify a settlement afterwards?

Short: take the transaction hash, read the receipt, decode `ReportProcessed` for the report ID and outcome, read `getEscrow` for the final state, and check token balances. **Reasoning:** the chain is self-describing; nothing depends on the app. **Instance:** that is precisely what the verification script produced for blocks 24, 29 and 30.

Q88. Why `uint64` for timestamps?

Short: to pack two timestamps with an address and an enum into fewer storage slots. **Reasoning:** storage is charged per slot, so smaller types reduce gas when they share a slot. **Instance:** `createdAt` and `settledAt` are both `uint64` and will not overflow for billions of years.

Q89. Why does the receiver take a `metadata` parameter that is always `0x`?

Short: because the CRE/Chainlink receiver interface requires it, and the local path passes empty bytes. **Reasoning:** keeping the signature identical means a DON can call `onReport` without code changes. **Instance:** metadata still contributes to the report ID, so two otherwise identical reports with different metadata are distinct.

Q90. What happens to the allowance after settlement?

Short: it was already consumed at funding; settlement uses `transfer` from the escrow's own balance, not an allowance. **Reasoning:** the escrow owns the tokens, so it needs no permission to move them. **Instance:** after a purchase the buyer's allowance to the escrow returns to zero for that amount.

14.5 Level 5 — the challenging questions

Q91. Why should I trust the courier API?

Short: you should not trust it blindly — you should bound what a lie can achieve. **Reasoning:** a false status can cause a wrong settlement, but only for a funded, unsettled order, only once, and only through an authorised report. **Instance:** the design mitigations are the on-chain funding re-check, replay protection, and the audit trail that records which report justified the payout. A production system would add a real carrier's signed webhooks or multiple independent sources.

Q92. What if the seller manipulates shipment status?

Short: they can make a parcel look delivered, but they cannot make money appear. **Reasoning:** settlement requires a funded escrow and a valid report, and the seller cannot call the escrow or the receiver. **Instance:** the worst case is early release on an order the buyer actually funded — a fraud that leaves a permanent, attributable on-chain record rather than an untraceable one.

Q93. The courier says DELIVERED but the buyer never received the parcel. What then?

Short: this is the system's hardest case, and the honest answer is that it is a dispute, not a bug. **Reasoning:** the oracle reports what the data source says; a wrong data source yields a wrong but valid settlement. **Instance:** mitigation today is the manual-review path and off-chain recourse; the report lists arbitration and multi-source verification as future work. Never claim the design solves physical fraud.

Q94. What stops a seller from generating a false delivery report?

Short: the report is not produced by the seller — it is produced by CRE from the API record, and accepted only from an authorised caller. **Reasoning:** the seller's leverage is limited to the API's write access, which is protected by a server-side secret and validated transitions. **Instance:** even a forged API record still needs a funded escrow and passes the receiver's single-settlement checks.

Q95. What if Chainlink returns incorrect information?

Short: the report would be wrong, and the contracts cannot detect that — they can only enforce authorisation and uniqueness. **Reasoning:** an oracle is a trust boundary; a DON reduces the chance of error or collusion but does not eliminate it. **Instance:** locally, one simulator runs the workflow, so the trust is even more concentrated; that is stated explicitly rather than glossed.

Q96. What if the bridge is compromised?

Short: it could suppress reports or submit a forged one, and the receiver would accept the forgery because the caller is authorised. **Reasoning:** authority is by address, not by content signature. **Instance:** the damage is bounded to one payout per funded order, and every submission is recorded on chain. A DON removes the bridge entirely, which is the real fix.

Q97. What if an admin key is stolen?

Short: the thief can reconfigure the oracle, mint pUSD, and settle through the manual path — but cannot directly drain escrowed funds to an arbitrary address. **Reasoning:** there is no sweep function; the escrow only pays the recorded buyer or seller. **Instance:** the mitigations are owner separation, event logging of every configuration change, and the recommendation of a multisig in production.

Q98. Why is a local execution bridge needed at all?

Short: because the SDK cannot address Hardhat and no DON was available, yet the settlement still had to be demonstrated for real. **Reasoning:** the alternative was either no on-chain settlement or a fake UI claim; the bridge keeps CRE as the decision-maker and produces real transactions. **Instance:** three verified settlements exist because of it.

Q99. Is your system actually decentralised?

Short: no — the custody and rules are on chain, but the oracle path is centralised in this deployment. **Reasoning:** one simulator, one bridge machine, one authorised account, one simulated data source. **Instance:** the honest phrasing is "trust-minimised custody with a centralised oracle, architected for a DON". Claiming more would be false and easily challenged.

Q100. Why didn't you deploy the workflow on the DON?

Short: access to a deployed DON was not available in this environment, and the SDK lacks a local chain selector. **Reasoning:** both are external constraints, not design choices. **Instance:** the workflow was still written to DON conventions — consensus aggregation, report interface, forwarder role — so deployment is a configuration change rather than a rewrite.

Q101. How is this different from traditional escrow?

Short: custody and rules are code, decisions are evidenced, and settlement is automatic. **Reasoning:** a traditional agent holds money and a human judges delivery; here no one holds the money and the judgement is a report you can audit. **Instance:** the buyer can verify the escrowed balance and the settlement transaction without asking anyone.

Q102. What would happen if Hardhat were restarted?

Short: every deployment, balance and escrow disappears; the addresses are reusable only after redeploying. **Reasoning:** the chain is in memory. **Instance:** the recovery order is redeploy → authorise → mint → re-register shipments; the bridge's preflight reports exactly which step is missing.

Q103. What happens if the API becomes unavailable?

Short: CRE logs an HTTP error and settles nothing; the system fails safe. **Reasoning:** no data means no decision, which is preferable to guessing. **Instance:** escrows remain Funded and can be settled later or manually, so availability affects liveness, not safety.

Q104. What if a shipment is returned after delivery?

Short: if it was already released, nothing automatic happens; if it is still funded, CRE routes RETURNED to manual review and emits no settlement report. **Reasoning:** a return after delivery is a dispute about facts, not a courier status. **Instance:** `shouldSettle` returns `MANUAL_REVIEW`, and the receiver would reject such an outcome anyway.

Q105. What is the single biggest limitation?

Short: the trust boundary at the oracle: a simulated data source and a single local relay. **Reasoning:** every other component is either enforced by code or verified by tests; this one rests on operational honesty. **Instance:** the contracts bound the consequences to one payout per funded order, which is why the limitation is survivable but must be stated first, not last.

14.6 Rapid fire

Q106. `msg.sender` versus `tx.origin`?

Short: `msg.sender` is the immediate caller; `tx.origin` is the EOA that started the chain. **Reasoning:** authorising on `tx.origin` enables phishing through an intermediate contract. **Instance:** ProofPay uses `msg.sender` everywhere, which is why the escrow sees the receiver as its caller.

Q107. Why not approve an unlimited allowance?

Short: because the approved spender could take everything at any time. **Reasoning:** least privilege; an exact approval limits blast radius to one order. **Instance:** ProofPay approves the order amount and the allowance is consumed at funding.

Q108. What is front-running, and are you exposed?

Short: observing a pending transaction and submitting one that profits from it. **Reasoning:** on a public mempool, transaction content and ordering are visible. **Instance:** ProofPay's settlement is keyed to a specific order and payee, so there is no price to manipulate; on a local chain there is no mempool race at all.

Q109. Calldata, memory and storage — one line each?

Short: calldata is immutable function input, memory is temporary working space, storage is permanent chain state. **Instance:** the report arrives as calldata, is decoded into memory, and only the two booleans and the escrow struct reach storage.

Q110. Why is the escrow's mapping private?

Short: because a getter that reverts on unknown orders is a better API than one returning a zeroed struct. **Reasoning:** silent zero values are a classic source of bugs. **Instance:** `getEscrow` reverts with `EscrowNotFound`.

Q111. Why does the report ID include the metadata?

Short: so two reports that differ only in provenance are treated as distinct. **Reasoning:** the metadata is part of what the caller asserted. **Instance:** locally metadata is `0x`, so it contributes nothing, but the interface stays DON-compatible.

Q112. What if two reports arrive with identical fields in the same second?

Short: the second is rejected — identical bytes give an identical report ID. **Reasoning:** the tick is part of the report only if the source's timestamp is; identical timestamps mean a true duplicate. **Instance:** that is the behaviour you want; a genuine second settlement attempt must fail.

Q113. What if the bridge's state file is deleted?

Short: replay protection moves entirely to the chain, which already covers it. **Reasoning:** the state file is a convenience for idempotent local runs, not the security boundary. **Instance:** a resubmission would revert with `ReportAlreadyProcessed` or `OrderAlreadySettled`.

Q114. What if two bridges run at once?

Short: the lock file prevents it; if forced, the chain prevents a double payout. **Reasoning:** both would compute the same report ID and only one transaction could succeed. **Instance:** the observed runs show one submission per order and one block of advancement.

Q115. What if CRE emits MANUAL_REVIEW as a settlement?

Short: the bridge refuses it and the receiver would revert with `UnsupportedOutcome`. **Reasoning:** the system only has two payouts, and neither means "a human should look at this". **Instance:** `shouldSettle` never produces an outcome for RETURNED, so this is belt and braces.

Q116. Why is the seller's address checked in the backend as well as the escrow?

Short: defence in depth and better errors. **Reasoning:** the escrow check would only fail at settlement, long after the mistake. **Instance:** `SELLER_MISMATCH` is returned at registration time.

Q117. What is the role of `block.timestamp` in your design?

Short: recording when an escrow was created and settled. **Reasoning:** it is consensus-agreed and monotonic enough for audit timestamps, but miners can nudge it slightly, so it must never gate security. **Instance:** no authorisation or amount logic depends on it.

Q118. Why not store the tracking ID on chain?

Short: cost and privacy. **Reasoning:** arbitrary-length strings cost storage and leak parcel identity. **Instance:** the hash gives evidence without exposure, and the receiver only needs it for the report ID.

Q119. How do you know the frontend is not lying?

Short: because it reads the chain rather than trusting a server, and every displayed value can be re-derived with an independent RPC call. **Reasoning:** the UI could still be malicious, but it cannot fabricate a balance that the chain disagrees with. **Instance:** the verification script reproduces the dashboard's numbers from `getEscrow` and `balanceOf`.

Q120. What is the point of the `EscrowCreated` event if the UI re-reads state?

Short: discovery and indexing. **Reasoning:** there is no enumerable list of mapping keys on chain, so an event is the only efficient way to learn which IDs exist. **Instance:** `useEscrows` filters logs on the event topic and the escrow address.

Q121. Why does the receiver use a self-call for the funding check?

Short: so a missing escrow becomes `EscrowNotFound` instead of a raw revert from the external call. **Reasoning:** typed errors are easier to test and to explain. **Instance:** `try this._checkEscrowFunded(orderId) ... catch { revert EscrowNotFound(orderId); }`.

Q122. Why does the bridge read the escrow again if CRE already did?

Short: because time passed and state can change between the simulation and the submission. **Reasoning:** the read is a preflight, not a second decision; it prevents wasting gas on an order that closed. **Instance:** the dry-run log prints the seller and the 100 pUSD amount from that read.

Q123. What would you do differently if you started again?

Short: design the report export before the first settlement, and keep the bridge's validation and the contract's checks generated from one schema. **Reasoning:** the dry-run ordering bug cost real time and was only visible once exports were attempted. **Instance:** a shared report schema would also have made the report ID derivation impossible to drift.

Q124. Which part of this project are you proudest of, technically?

Short: the demonstration that both outcomes can be executed and independently verified, with the decision provably outside the relay. **Reasoning:** the interesting engineering is the boundary design, not the contracts. **Instance:** the allowlist run where CRE reported two shipments and exactly one transaction resulted.

Q125. If I gave you one week, what would you add?

Short: a Sepolia deployment with a verified receiver and a DON-hosted workflow, plus one real carrier sandbox integration. **Reasoning:** those three changes convert the honest local caveats into production-relevant evidence. **Instance:** the escrow and receiver need no redesign; only addresses, configuration and the removal of the bridge.

MODULE 15

OWNERSHIP, RESUME AND DEFENCE

The goal of this module is not to make the project sound bigger. It is to make your explanation precise enough that an interviewer believes you built it, because precision is the signature of authorship.

15.1 Resume entry

ACCURATE 3–4 BULLET DESCRIPTION

- Built ProofPay, an escrow-based payment-protection DApp: a Solidity ERC-20 (pUSD), a custody escrow with two terminal settlement paths, and a report-authorising receiver contract deployed on a local Hardhat network (chain ID 31337).
- Implemented a Chainlink CRE (TypeScript) workflow that fetches shipment data over HTTP, verifies escrow state over EVM, applies a six-state settlement policy, and emits ABI-encoded reports with full provenance.
- Engineered a local execution bridge that relays CRE's exact report bytes to the receiver, with preflight validation, order allowlisting, idempotent state tracking and receipt verification — settling both release and refund paths on chain.
- Delivered a React/TypeScript frontend with MetaMask (EIP-1193), buyer/seller/admin dashboards and an Express shipment API; validated with 31 contract tests, 17 backend tests and a clean production build.

15.2 The 30-second introduction

"ProofPay is a blockchain payment-protection system for high-value online purchases. The buyer locks pUSD in a smart-contract escrow, a Chainlink CRE workflow reads the courier's shipment record and the escrow state, decides whether the delivery condition is met, and produces a signed-off report; a receiver contract enforces authorisation and replay protection and tells the escrow to pay the seller or refund the buyer. I built the three contracts, the CRE workflow, the local relay and the React front end, and verified both settlement paths on a local Hardhat chain."

15.3 The two-minute version

Structure it as problem, design, evidence, limitation. Problem: in high-value e-commerce, whoever acts first carries the risk, and existing protections replace that risk with an institution you cannot audit. Design: custody moves into a contract with no withdrawal path; the delivery fact comes from an off-chain API read by CRE rather than asserted by an administrator; the report carries order, outcome, tracking hash and timestamp; the receiver accepts it only from an authorised caller, only once, and only while the order is funded. Evidence: 31 contract tests and 17 API tests pass, and three settlements executed with receipts at blocks 24, 29 and 30 — a release, a refund and a second release, each moving exactly 100 pUSD and each advancing the chain by one block. Limitation: the workflow runs in the official CRE simulator with dry-run enabled and a local bridge relays the report, so this is real workflow execution, not DON consensus; and the courier is simulated.

15.4 The five-minute architecture answer

Walk the four layers and name one artefact per layer. Presentation: React reads chain state through MetaMask; the checkout sequences approve then createEscrow, waiting for each receipt. Contracts: the token, the escrow (Funded, Released, Refunded, oracle-only settlement), and the receiver (authorise, decode, de-duplicate, route). Oracle: CRE fetches the API, reads `getEscrow` at a finalised block, applies `shouldSettle`, encodes a four-field report, and logs structured records. Relay: the bridge validates those records, re-derives the report ID, checks the allowlist and fund-

ing, submits `processReport` from an unlocked account, and records the receipt. Then say what each layer cannot do — that is what makes the answer senior rather than descriptive.

15.5 Defending each component

Table 15.1: The one-sentence defence for every component

Component	What you say
pUSD	"A 16-line ERC-20 so the protected amount is an exact integer separate from gas; mint is owner-only, and there is no burn or pause that could destroy or freeze escrowed value."
Escrow	"Custody with exactly two exits, both oracle-only, both requiring Funded and flipping to a terminal state before transferring."
Receiver	"A policy boundary: it authorises the caller, decodes the report, blocks replay and duplicates, and routes two outcomes to two functions."
CRE workflow	"The only decision-maker: it fetches the courier record, verifies the escrow, applies the six-state policy, and encodes the report with provenance."
Bridge	"A relay with no policy: it validates, allowlists, submits CRE's bytes unchanged, and records the receipt; it cannot choose a payee."
Shipment API	"The off-chain system of record with strict transitions and a chain check at registration, so the oracle always reads a fact that corresponds to funded money."
Scheduler	"A courier-fact generator for demos; it advances statuses and triggers the pipeline but never settles anything."
Frontend	"A view over chain state: it discovers orders from logs, re-reads each escrow from storage, and sequences transactions only after receipts."
Tests	"31 contract tests including duplicate settlement and reentrancy, plus 17 API tests; the negative cases are the ones that matter."

15.6 Explaining your contribution honestly

You used AI coding agents on this repository. That is normal in 2026 and it is not a weakness, but the way you describe it determines whether it sounds like authorship or delegation. Say what you decided, not what you typed: "I chose to keep the decision in CRE and make the bridge policy-free, which is why the bridge has no copy of `shouldSettle`. I rejected putting settlement directly on the escrow because it would have put ABI decoding and replay logic into the contract that holds the money." Then demonstrate understanding of a detail only an author would know — the dry-run/report ordering fix, the 109-byte header, the allowlist run where two reports produced one transaction.

> **THREE SENTENCES NEVER TO SAY**
>
> "It is deployed on Chainlink's decentralised network." "It is production-ready and audited." "The courier integration works with real carriers." Each is false, and each invites a question that ends the interview badly. The accurate versions are stronger because they show judgement.

MODULE 16

HANDS-ON EXERCISES AND MOCK INTERVIEW

Do these with the repository closed. If you need the code, you have not learned it yet — read the relevant module again and come back. Answers follow the exercise list; do not read them first.

16.1 The exercises

1. Explain, in five sentences, what happens from the moment a buyer clicks "Connect wallet" to the moment the header shows a pUSD balance.
2. Write out the token approval flow from memory, naming the two contracts, the two transactions and the storage each one changes.
3. Explain why the escrow locks funds, and what would break if the buyer paid the seller directly.
4. Trace one purchase from the React form to the Solidity storage write, naming every file on the way.
5. Describe a complete ERC-20 transaction: who signs, what is encoded, what events are emitted, what changes in storage.
6. Trace one shipment API request end to end, including validation, the chain check and the response code.
7. Explain how CRE fetches shipment status, including which capability, which method, and what happens on failure.
8. Decode an example settlement report by hand: given a 256-byte payload, name the four fields in order and what each contains.
9. Explain how `processReport` works, in order, including every check and every revert.
10. Explain the refund flow without looking at this guide.
11. Diagnose an intentionally wrong network: the user is on mainnet, every button is disabled, and no error is shown. What is happening and where?
12. Diagnose an unauthorised oracle: settlement reverts with `UnauthorizedCaller`. Give three possible causes and the exact read that distinguishes them.
13. Explain how to verify a transaction receipt, and what you would check to prove a release actually paid the seller.

16.2 Answers and expected reasoning

1. Connection. `useWallet` first calls `eth_accounts` (no prompt) to see whether the site is already permitted; if not, the button calls `eth_requestAccounts`, MetaMask prompts, and the chosen address is passed to `applyAddress`. That function reads `eth_chainId` and `eth_getBalance`, then writes one state object with the address, chain name, ETH balance and an `isCorrectNetwork` flag. A separate token hook reads `balanceOf` for pUSD. Nothing is stored in a database; deletion of the extension would not change any of these values on chain.

2. Approval. Transaction 1: buyer → pUSD `approve(escrow, amount)`, which writes `allowance[buyer][escrow]` and emits `Approval`. Transaction 2: buyer → escrow `createEscrow`, which calls `pUSD.transferFrom(buyer, escrow, amount)`; that decrements the allowance, moves balances, and emits `Transfer`. The escrow also writes its `Escrow` struct and emits `EscrowCreated`. The allowance exists so that the token, not the escrow, remains the authority over the buyer's balance.

3. Why escrow. Because the only alternative is trusting the counterparty to reverse a bad outcome. If the buyer pays directly, a lost parcel leaves them asking the seller for money; if the seller ships first, they are unsecured. Escrow

makes the money conditional and the condition auditable. Removing it would also remove the thing the project sells.

4. Purchase trace. `CheckoutPage.tsx` validates the form and calls `useEscrowActions`, which uses viem to encode `approve` and sends it through the injected provider; `MetaMask` signs; `ProofPayUSD.sol` stores the allowance; then `createEscrow` is encoded and sent; `ProofPayEscrow.sol` validates, writes the struct, calls the token, emits the event; back in React, `onCreated()` triggers `useEscrows` to re-read state.

5. A complete transfer. The owner signs a transaction calling `transfer(to, amount)`; the data is the 4-byte selector plus two 32-byte words; the token checks the balance, subtracts from the sender, adds to the recipient, emits `Transfer(from, to, amount)`, and returns true. Two storage slots change; nothing else can alter them afterwards, and the receipt's logs are the permanent evidence.

6. Shipment request. React sends `POST /api/shipments` with the mutation secret; Express parses JSON (32 kB limit), the middleware authorises, the router calls `register`; validation normalises fields; duplicates are rejected; `verifyEscrow` performs an RPC `getEscrow` and requires status 1 and matching seller; the store appends the record atomically; the response is 201 with a `Location` header, or 400/404/409 on failure.

7. CRE fetch. `HTTPClient.sendRequest` with method GET against `baseUrl + /api/shipments`, wrapped in `consensusIdenticalAggregation<string>()`; `ok(response)` is checked, the body is read as text and parsed defensively. On a non-2xx response or a parse error the handler logs and returns an error string, so the run produces no settlements rather than a partial set.

8. Report decoding. Word 1: `bytes32 orderId`. Word 2: offset to the outcome string. Word 3: `bytes32 trackingIdHash`. Word 4: offset to the timestamp string. The two strings follow dynamically with their lengths. In the observed run, the total was 256 bytes, and the raw report was 365 because of the 109-byte SDK header.

9. processReport. `onlyAuthorized` → decode four fields → reject zero order ID → compute report ID → reject if `processedReports` → reject if `orderSettled` → self-call to confirm funding → hash outcome → call `confirmDelivery` or `confirmFailure` → set both mappings → emit `ReportProcessed` and `SettlementExecuted`. Any other outcome reverts with `UnsupportedOutcome`.

10. Refund. Shipment becomes LOST after `IN_TRANSIT`; CRE's `shouldSettle` returns REFUND with outcome LOST; the escrow is read and must be Funded; the payload is encoded and the report generated; the bridge validates the outcome and the funding, submits `processReport`; the receiver checks authorisation, replay, duplicate and funding, then calls `confirmFailure`, which writes Refunded and transfers to the buyer. Verified at block 29: buyer +100 pUSD, seller unchanged.

11. Wrong network. `applyAddress` compared `parseChainId(eth_chainId)` against 31337 and set `isCorrectNetwork = false`. The dashboard's write actions are gated on that flag, and the banner explains it — if the user sees nothing, the banner is above the fold or the component is reading a stale wallet object. Check the header chip and the console value of `chainId`.

12. UnauthorisedCaller. Possible causes: (a) the sending account is neither forwarder nor manual oracle, (b) the receiver's manual oracle was rotated and no longer matches the wallet, (c) the escrow's oracle was pointed elsewhere so the receiver's call fails downstream — but note that (c) produces `UnauthorizedOracle` from the escrow, not `UnauthorizedCaller`. Distinguish with two reads: `receiver.manualOracle()` and `escrow.oracle()`, compared against the connected address and the receiver address respectively.

13. Receipt verification. Fetch by hash, require `status === 1`, check `to` is the receiver, decode the logs for `EscrowReleased(orderId, seller, amount)` and `SettlementExecuted(orderId, true)`, then read `getE-`

`scrow(orderId)` for status 2 and a non-zero `settledAt`, and finally compare the seller's `balanceOf` before and after — the delta must equal the escrowed amount exactly.

16.3 Final mock interview: 25 questions

Answer aloud, in under ninety seconds each. The expected shape is in the brackets; if you cannot fill the bracket, revisit the module named there.

1. What problem does ProofPay solve, in one sentence? *(Module 1)*
2. Why is the escrow a contract rather than a bank account? *(M3)*
3. Why does the buyer need two transactions? *(M2)*
4. What exactly does MetaMask store, and what does it not? *(M2)*
5. Why can a smart contract not call the courier API? *(M7)*
6. What is a report, and what is inside yours? *(M7)*
7. Why is there a receiver contract between CRE and the escrow? *(M3)*
8. How do you prevent a double payment, precisely? *(M3, M8)*
9. What does the bridge decide? *(M8)*
10. What does the bridge deliberately not decide? *(M8)*
11. Why 109 bytes, and what would happen if you ignored them? *(M7)*
12. What is the difference between simulation and DON execution? *(M7)*
13. Why is the CRE dry-run still true after settlement worked? *(M8)*
14. Where is the settlement policy implemented, and how many copies exist? *(M7)*
15. Why is shipment status off chain? *(M6)*
16. What prevents a seller from registering someone else's order? *(M6)*
17. What are the three error layers in the frontend? *(M5)*
18. How does the dashboard discover orders without an indexer? *(M5)*
19. Which tests would fail if you removed ReentrancyGuard? *(M4)*
20. What happens if Hardhat restarts mid-demo, and in what order do you recover? *(M12)*
21. What is your biggest security limitation? *(M11)*
22. Why is pUSD not a stablecoin, and why does that matter? *(M11)*
23. What would you change for a real DON deployment? *(M8)*
24. What did you personally decide that a template would not have given you? *(M15)*
25. What would you build next, and why that? *(M15)*

MODULE 17

GLOSSARY

Every term this project uses, in plain language. Where a term has a specific meaning in this codebase, that meaning is given rather than the general one.

Table 17.1: Glossary of terms used in ProofPay

Term	Meaning
ABI	JSON description of a contract's functions, arguments, returns, events and errors; the contract between bytes and JavaScript.
ABI encoding	The canonical byte layout (32-byte words, offsets for dynamic types) used to pass arguments and decode results.
Account	An address with a balance; may also have code and storage if it is a contract.
Address	20-byte identifier, written as 40 hex characters with a 0x prefix.
Allowance	How much a spender may still take from an owner under ERC-20.
Allowlist (bridge)	`BRIDGE_ORDER_ALLOWLIST`: optional set of order IDs a bridge run may submit.
Approval	ERC-20 event emitted when an allowance is set.
Approve	ERC-20 function that grants a spending cap to a spender.
Atomic write	Writing to a temporary file then renaming, so a crash cannot leave a half-written JSON store.
Bigint	JavaScript's arbitrary-precision integer type, used for all token amounts.
Block	A batch of transactions with a header that commits to the previous block's hash.
Block number	The position of a block in the chain; recorded with every settlement in this project.
Bridge (local execution bridge)	`local-runner.ts`: runs the CRE simulator, validates its report, submits it to the receiver, verifies the receipt.
Bytecode	Compiled contract code deployed to an address; `eth_getCode` returning `0x` means nothing is deployed.
Calldata	Read-only bytes passed into a function call; where the 256-byte report arrives at the receiver.
Chain ID	Network identifier signed into transactions; 31337 here, preventing cross-chain replay.
Checks-effects-interactions	Pattern of validating, writing state, then making external calls — used by the escrow before transfers.
Consensus	Agreement among nodes on the next valid state; in oracle networks, agreement on a reported fact.
Consensus aggregation	CRE wrapper requiring nodes to produce identical results before proceeding; present in the HTTP call but only single-node in simulation.
Contract account	An account with code, controlled only by being called.
Courier scheduler	`courier-scheduler.ts`: advances demo shipment statuses and optionally triggers the pipeline.
Cron trigger	Schedule-based workflow trigger; `0 */5 * * * *` here.
Custom error	Typed Solidity revert with optional arguments, cheaper and more testable than strings.
Decentralisation	Degree to which no single party can unilaterally decide outcomes; partial in this project, by design and by disclosure.
Decimals	Display scaling for a token; 18 for pUSD, so 1 pUSD = 10^{18} base units.
Deterministic execution	Every node computes the same result from the same inputs; the reason HTTP cannot run inside the EVM.

Term	Meaning
DON	Decentralised oracle network: independent nodes that must agree before reporting. Not deployed here.
dryRun	CRE configuration that generates reports but skips `writeReport`; true in this project.
ECDSA	The signature scheme (over secp256k1) used by Ethereum accounts.
EIP-1193	Standard for the injected wallet provider at `window.ethereum`.
EIP-165	Interface detection standard; the receiver advertises `IReceiver` through `supportsInterface`.
EOA	Externally owned account: controlled by a private key and able to start transactions.
ERC-20	Standard fungible token interface: balances, transfers, allowances, two events.
Escrow	Conditional custody of funds by a neutral party or contract; here, `ProofPayEscrow`.
Event (log)	Permanent record in a transaction receipt; readable off chain, not by contracts.
EVM	Ethereum Virtual Machine: the deterministic computer that executes contract bytecode.
EVM capability	CRE facility for reading (and, when enabled, writing) chain state via a chain selector.
Express	Minimal Node.js HTTP framework used for the shipment API.
Forwarder	Address authorised to deliver CRE reports to the receiver; zero in this local deployment.
Function selector	First four bytes of `keccak256("name(types)")`; identifies the function in calldata.
Gas	Unit of computational work; priced in the native currency.
Gas limit	Maximum gas a transaction may consume; unused gas is refunded.
Gas price	Price per gas unit offered by the sender.
Hardhat	Ethereum development environment: compiler, test runner, scripting and local node.
Hardhat node	The local JSON-RPC chain on port 8545 with deterministic unlocked accounts.
Hash	Fixed-length fingerprint of arbitrary data; Keccak-256 here.
HTTP capability	CRE facility for making web requests, with optional consensus across nodes.
Idempotency	Property that repeating an operation has the same effect as doing it once; required for settlement.
Immutability	Practical infeasibility of rewriting history; absent on a restarted local chain.
Interface	A Solidity declaration of function signatures without implementation, used to type external calls.
JSON	Text data format used by the API, configuration and deployment records.
JSON-RPC	Protocol used to call node methods over HTTP, e.g. `eth_call`.
Keccak-256	Ethereum's hash function producing 32 bytes; used for selectors, report IDs and tracking hashes.
Lock file	File created exclusively to prevent two instances of the bridge or scheduler running at once.
Manual oracle	Receiver role allowing a fallback reporter, used here for local and admin settlement.
Mapping	Solidity key-value storage; every key exists with a zero default.
Mempool	Pending transactions waiting to be included in a block.
Metadata (report)	SDK-provided 109-byte header in a raw CRE report, stripped before the receiver sees the payload.
MetaMask	Browser wallet storing keys, signing transactions and injecting the provider.
Mint	Creating new token supply; owner-only in pUSD.
Modifier	Reusable condition wrapped around a function body; `onlyOracle`, `nonReentrant`.
msg.sender	The immediate caller of a function; the receiver, when it settles an escrow.

Term	Meaning
msg.value	Native currency sent with a call; always zero here, since no function is payable.
Nonce	Per-account transaction counter preventing replay and enforcing order.
Off-chain / on-chain	Computed and stored outside the ledger versus inside it.
Opcode	A single EVM instruction; the target of Solidity compilation.
Oracle	Component delivering external facts to a blockchain.
Oracle problem	The difficulty of getting trustworthy external data into a system that cannot verify it.
Ownable / onlyOwner	OpenZeppelin module granting a single address configuration rights.
Preflight	The bridge's read-only checks (chain, bytecode, signer, wiring, gas) before any submission.
Private key	256-bit secret that signs transactions; never stored in this repository.
Provenance	Information about how a report was produced: source slice, lengths, hashes, addresses.
pUSD	ProofPay USD, the prototype ERC-20 used for escrowed value.
Receipt	Node's record of an executed transaction: status, gas used, logs, block number.
Reentrancy	Attack where an external call re-enters the caller before state is final.
ReentrancyGuard	OpenZeppelin modifier preventing reentrant entry.
Refund	Returning escrowed funds to the buyer when the shipment is lost.
Release	Paying the seller when delivery is confirmed.
Report (CRE)	ABI-encoded payload plus SDK metadata header, produced by `runtime.report`.
Report ID	Hash of order, outcome, tracking hash, timestamp and metadata; the replay-protection key.
Replay attack	Resubmitting a valid message a second time to repeat its effect.
REST	API style using URLs for resources and HTTP verbs for intent.
Revert	Aborting execution and undoing all state changes, optionally with a reason.
RPC	Remote procedure call: the HTTP interface to a node.
SafeERC20	Wrapper checking token call results so failures cannot be ignored.
Seed phrase	12–24 words from which all wallet keys are derived.
Settlement	The irreversible payout of an escrow, to seller or buyer.
Signature	Cryptographic proof that a key holder approved specific bytes.
Simulation (CRE)	Running a workflow locally through the CLI, one node, real code.
Smart contract	Program on a blockchain with persistent storage and deterministic execution.
Solidity	The contract language used here, version 0.8.28.
State file (bridge)	`.local-bridge-state.json`: records successful submissions to prevent duplicates.
State variable	Permanent storage declared at contract level.
struct	Grouped fields under one type; the `Escrow` record.
Testnet	Chain with valueless currency for development; not used in this deployment.
Tracking ID	Courier reference such as `PPD-DEMO-LOST`; stored off chain, hashed on chain.
Transaction	Signed instruction that changes chain state.
Transaction hash	Keccak-256 of the signed transaction; its unique identifier.
transfer / transferFrom	ERC-20 functions moving your own balance, or a spender's allowance.

Term	Meaning
TypeScript	Typed superset of JavaScript used in the frontend, scripts and workflow.
Upgradeability	Ability to replace contract code behind a proxy; deliberately absent here.
viem	TypeScript Ethereum library used for ABI encoding, reads and transaction sending.
Wallet	Software holding keys and signing; it does not hold the assets.
Workflow (CRE)	TypeScript program with a trigger and handler, run in simulation or on a DON.
writeReport	CRE call that submits a report to a receiver on a supported chain; unreachable while dry-run is true.
Zero address	`0x000…000`: a burn address; used here to mean "no forwarder configured".

17.1 How to keep learning after this book

Three habits will carry you further than any tutorial. Re-derive, do not re-read: when you forget how the receiver works, write its checks from memory and compare. Change one thing in the repository and predict the failure before running it — set `dryRun` false on CRE while the bridge runs and see which check refuses; remove the allowlist and see two orders become eligible. And explain it to someone who does not code: the moment you can describe an oracle without the word "oracle", you understand it.

Finally, keep the honesty rule. The most impressive thing you can say about ProofPay is not that it is decentralised, but that you know exactly where it is not, why that choice was made, and what would have to change. That is what separates an engineer who used a tool from one who understands it.

End of the ProofPay Master Guide. Every code excerpt is from the repository; every transaction figure was read from the local Hardhat chain; every limitation named here is named in the project report as well.